RKNN3 SDK Development Guide

File ID: RK-JC-YF-416

Release Version: V1.2.0

Date: 2026-10-22

Confidentiality Level: □Top Secret □Secret □Internal ■Public

Disclaimer

This document is provided on an "as is" basis. Rockchip Electronics Co., Ltd. ("the Company", hereinafter the same) makes no express or implied statements or warranties regarding the accuracy, reliability, completeness, merchantability, fitness for a particular purpose, or non-infringement of any statements, information, and content contained in this document. This document serves only as a reference for usage guidance.

Due to product version upgrades or other reasons, this document may be updated or modified from time to time without prior notice.

Trademark Notice

"Rockchip", "瑞芯微", and "瑞芯" are registered trademarks owned by Rockchip Electronics Co., Ltd.

All other registered trademarks or trademarks that may be mentioned in this document are the property of their respective owners.

Copyright © 2026 Rockchip Electronics Co., Ltd. All rights reserved.

Beyond the scope of fair use, no organization or individual may excerpt or copy any part or all of this document without prior written permission from the Company, nor distribute it in any form.

Rockchip Electronics Co., Ltd.

Address: No. 18, Zone A, Software Park, Tongpan Road, Fuzhou, Fujian, China

Website: www.rock-chips.com

Customer Service Hotline: +86-4007-700-590

Customer Service Fax: +86-591-83951833

Customer Service Email: fae@rock-chips.com

Preface

Overview

This document is intended for users of the RKNN3 SDK. It systematically introduces the SDK's model conversion, quantization, evaluation, and on-device deployment methods for both coprocessor mode (RK1820 / RK1828) and non-coprocessor mode (RK3572). The document is organized into five parts: fundamentals, conversion and evaluation, on-device deployment, advanced features, and diagnostics and optimization.

Intended Audience

This document is primarily intended for the following engineers:

- Technical support engineers
- Software development engineers

Revision History

Version	Modified By	Date	Description of Changes	Approved By
V1.0.0	HPC / NN	2026-01-01	1. Initial release (RK182X)	Vincent
V1.0.4	HPC / NN	2026-05-12	1. Added RK3572 support 2. Added LLM inference pause and resume 3. Added KV Cache import and export 4. Supported LoRA 5. Supported encrypted model deployment	Vincent
V1.1.0	HPC / NN	2026-08-08	1. Restructured the document 2. Updated rkllm3-server usage 3. Corrected quantization descriptions and API consistency	Vincent
V1.2.0	HPC / NN	2026-10-22	1. Added Paged Attention usage instructions 2. Added MatMul C API usage instructions 3. Added RK182X multi-card LLM deployment instructions 4. Added MoE model conversion instructions 5. Added LLM structured output (Grammar) instructions	Vincent

Table of Contents

RKNN3 SDK Development Guide

1. Overview and Development Preparation

1.1 RKNN3 SDK Overview

1.1.1 SDK Positioning and Typical Application Scenarios

1.1.2 Supported Hardware Platforms and Deployment Modes

1.1.3 Software Stack and Component Responsibilities

1.1.4 Relationship Among Toolkit / Runtime / Toolkit Lite / Server

1.1.5 Terms, Abbreviations, and Document Conventions

1.2 SDK Components and Environment Constraints

1.2.1 Version Compatibility Matrix

1.2.2 Host System Requirements

1.2.3 On-Device Environment Requirements

1.2.4 Environment Verification

1.3 Development Workflow and Interface Selection

1.3.1 General Model Development Workflow

1.3.2 LLM Development Workflow

1.3.3 MLLM Development Workflow

1.3.4 Interface Selection

1.3.5 Model Conversion and Deployment Outputs

2. Model Conversion, Quantization, and Evaluation

2.1 General Model Conversion

2.1.1 Model Preparation

2.1.2 Conversion Configuration

2.1.3 Model Loading and Building

2.1.4 RKNN Model Export

2.1.5 Conversion Result Verification

2.2 LLM and MLLM Model Conversion

2.2.1 Hugging Face Model Preparation

2.2.2 ONNX, Config, Tokenizer, and Embedding Export

2.2.3 LLM and MLLM Model Conversion

2.2.3.1 LLM Model Conversion

2.2.3.2 MLLM Model Conversion

2.2.4 Model and Tokenizer Adaptation

2.2.5 Conversion Result Verification

2.2.6 Segmented Model Export
2.2.7 MoE Model Conversion
2.3 Model Quantization
2.3.1 Quantization Basics
2.3.1.1 Quantization Definition
2.3.1.2 Quantization Computation
2.3.1.3 Quantization Granularity
2.3.1.4 Quantization Algorithms
2.3.2 Toolkit Quantization Flow Overview
2.3.3 Internal Quantization Flow
2.3.3.1 Data Types
2.3.3.2 Internal Quantization Algorithms
2.3.3.3 Calibration Data Preparation
2.3.3.4 Specific Examples
2.3.4 RKQuantizer Quantization Flow
2.3.4.1 Data Types
2.3.4.2 RKQuantizer Quantization Algorithms
2.3.4.3 Calibration Data Preparation
2.3.4.4 Mixed-Precision Quantization Settings
2.3.4.5 Specific Examples
2.3.5 Other Quantized Model Conversion
2.4 Model Evaluation
2.4.1 Model Inference Verification
2.4.2 Accuracy Evaluation
2.4.3 Performance Evaluation
2.4.4 Memory Evaluation
3. On-device Deployment
3.1 Runtime Deployment Basics
3.1.1 Model and Context Lifecycle
3.1.2 Tensors and Data Layout Formats
3.1.3 Input and Output Memory
3.1.4 Synchronous and Asynchronous Inference
3.1.5 Error Handling and Resource Release
3.2 General Model C API Deployment
3.2.1 Complete Invocation Flow
3.2.2 Model Initialization

3.2.3 Input and Output Preparation
3.2.4 Inference Execution
3.2.5 Output Processing
3.2.6 Complete Example
3.2.7 Debugging Methods
3.3 LLM and MLLM Session Deployment
3.3.1 Complete Invocation Flow
3.3.2 Session Lifecycle
3.3.3 Input and Callbacks
3.3.4 Inference Execution and Interruption
3.3.5 Chat Template
3.3.6 Multimodal Input
3.3.7 Complete Example
3.4 Toolkit Lite Python Deployment
3.4.1 Python Runtime Environment
3.4.2 RKNN3Lite Object Initialization and Release
3.4.3 Loading and Initializing the Model
3.4.4 Tensor and Memory Management
3.4.5 Executing Inference
3.4.6 Complete Example
3.5 RKLLM3 Server Integration
3.5.1 Server Applicable Scenarios
3.5.2 Service Deployment Overview
3.5.3 Client Invocation Methods
3.5.4 Choosing Between Native C/Python Interfaces
3.5.5 Server Documentation Entry
3.6 RK182X Multi-Card LLM Deployment
3.6.1 Multi-Card Topology and Runtime Boundaries
3.6.2 Runtime Parameters and Resource Constraints
3.6.3 Runtime Initialization for Each Stage
3.6.4 Host-Side Data Forwarding and Inference Control
3.6.5 External Inputs and Callbacks
3.6.6 Command-Line Deployment Example
3.7 MatMul C API Usage
3.7.1 Interfaces and Runtime Environment
3.7.2 Data Layout, Memory, and Call Flow

3.7.3 W16A16 Core Call Example
3.7.4 Preparing Quantization Parameters
3.7.5 Multiple Shapes Sharing Fixed B
3.7.6 Return Values, Resource Lifetime, and Further Documentation
4. Advanced Features and Extensions
4.1 Runtime Advanced Features
4.1.1 NPU Multi-Core
4.1.2 Dynamic Shape
4.1.3 Cacheable Memory Coherence
4.1.4 Internal Memory Reuse
4.2 LLM and MLLM Advanced Features
4.2.1 Prompt, Token, and Embedding Inputs
4.2.2 Multimodal and Aux Inputs
4.2.3 Callback Mechanism
4.2.4 Function Calling
4.2.5 KVCache Management
4.2.6 Paged Attention
4.2.7 KVCache Checkpoint
4.2.8 LoRA
4.2.9 Tie Word Embedding
4.2.10 Pause, Resume, and State Query
4.2.11 Structured Output (Grammar)
4.3 Extensions and Security
4.3.1 Custom Operators
4.3.1.1 Post-Process Operators
4.3.1.1.1 Plugin Implementation Steps
4.3.1.1.2 Plugin Compilation, Loading, and Usage
4.3.1.1.3 Examples
4.3.1.2 Custom CPU Operators
4.3.1.2.1 Plugin Implementation Steps
4.3.1.2.2 Plugin Compilation, Loading, and Usage
4.3.1.2.3 Examples
4.3.2 Encrypted Model Deployment
5. Issue Diagnosis and Optimization
5.1 Accuracy Issue Diagnosis

5.1.1 Internal Quantization Accuracy Diagnosis

5.1.2 RKQuantizer Quantization Accuracy Diagnosis

5.2 Performance Analysis and Optimization

5.3 Memory Analysis and Optimization

5.4 FAQ and Error Handling

1. Overview and Development Preparation

1.1 RKNN3 SDK Overview

1.1.1 SDK Positioning and Typical Application Scenarios

The RKNN3 SDK is an AI model deployment development kit from Rockchip for the RK1820 / RK1828 coprocessors and the RK3572 SoC. It provides a complete toolchain covering model conversion, quantization and evaluation, and on-device deployment, and supports efficient NPU inference for various model types such as CNN / LLM / MLLM.

Typical application scenarios include:

- **Visual recognition and detection**: Deploy CNN models such as MobileNet / YOLO on RK182X or RK3572 for tasks such as image classification and object detection.
- **Large language model inference**: Deploy large language models (LLMs) such as Qwen, with support for multi-turn dialogue, streaming generation, KVCache acceleration, LoRA fine-tuning adaptation, and more.
- **Multimodal understanding**: Deploy vision-language models such as Qwen2.5-VL, with support for image-text understanding, video understanding, and more.
- **OpenAI-compatible services**: Provide OpenAI-compatible APIs through rkllm3-server to integrate with the existing application ecosystem.

The SDK supports two deployment modes:

- **Coprocessor mode (RK182X)**: The host SoC (such as RK3588 / RK3576) connects to the RK1820 / RK1828 coprocessors via PCIe / USB / Ethernet. Applications run on the host SoC and control the coprocessors to perform AI inference through the RKNN3 API.
- **Non-coprocessor mode (RK3572)**: The RK3572 SoC integrates a high-performance NPU. Applications run directly on the RK3572 and control the local NPU to perform AI inference through the RKNN3 API.

1.1.2 Supported Hardware Platforms and Deployment Modes

This document applies to the following hardware platforms:

- **RK1820 / RK1828 (coprocessor mode)**: Serve as AI computing acceleration units. They are scheduled by the host SoC and share the computation workload via PCIe / USB / Ethernet interfaces.
- **RK3572 (non-coprocessor mode)**: An SoC platform with an integrated high-performance NPU, where the NPU shares memory space with the CPU / GPU.

The main differences between the two modes:

Comparison Item	Coprocessor Mode (RK182X)	Non-coprocessor Mode (RK3572)
Host SoC	RK3588 / RK3576, etc.	None (RK3572 itself is the host)
NPU	RK1820 / RK1828 coprocessors	RK3572 integrated NPU

Comparison Item	Coprocessor Mode (RK182X)	Non-coprocessor Mode (RK3572)
Communication method	PCIe / USB / Ethernet	Unified memory architecture
Communication service	rknn3_transfer_proxy	rknn3-server
Supported systems	Android / Linux / Windows	Android / Linux (aarch64 only)

1.1.3 Software Stack and Component Responsibilities

The overall RKNN3 software stack architecture is divided into three layers: the PC side, the device-side API and application services, and the Runtime and NPU driver.

> **Coprocessor mode architecture**

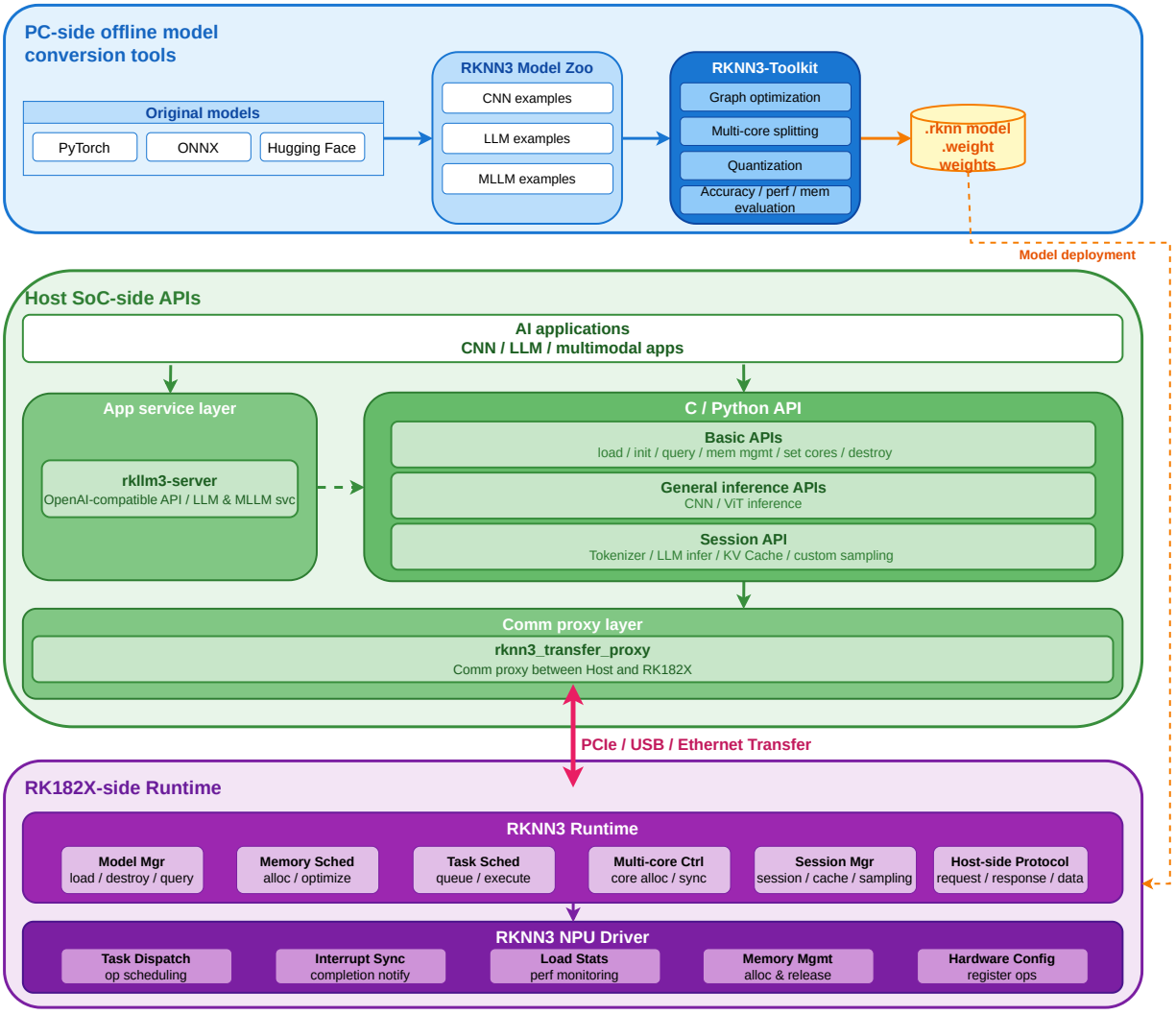


Figure 1-1 Coprocessor mode architecture

Non-coprocessor mode architecture

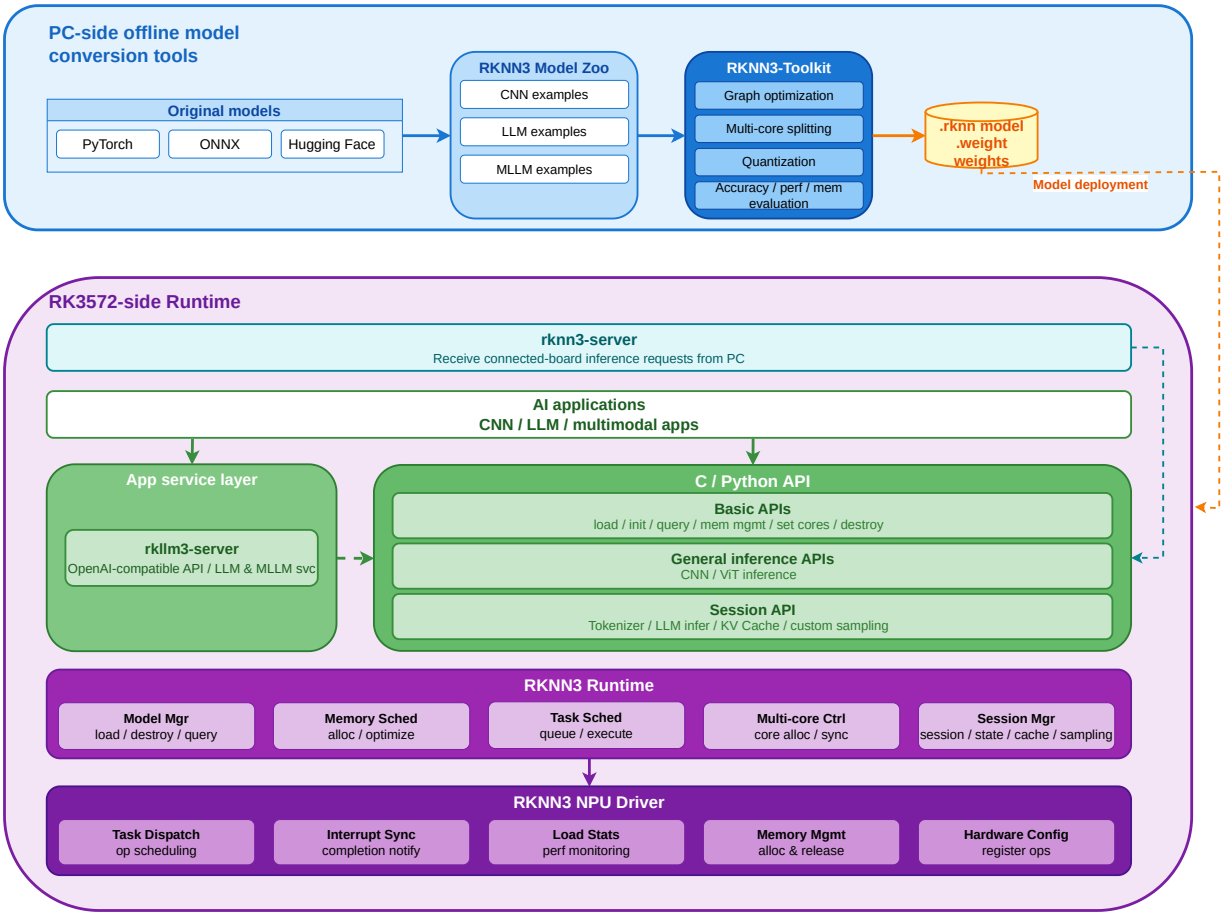


Figure 1-2 Non-coprocessor mode architecture

The responsibilities of each component are as follows:

1. PC side

- **RKNN3 Model Zoo**: Provides model conversion and deployment examples covering multiple model types, including CNNs (MobileNetV2 / ResNet50 / YOLOv5/v6/v8, etc.), LLMs (Qwen2.5 / Qwen3, etc.), and MLLMs (FastVLM / Qwen2.5-VL / Qwen3-VL, etc.).
- **RKNN3-Toolkit**: The core model conversion tool. It converts ONNX models into `.rknn` and `.weight` formats and performs graph optimization, multi-core partitioning, quantization, accuracy analysis, and visualization analysis. For LLM / MLLM models, first use the export scripts from the Model Zoo to convert the model to ONNX, and then use the Toolkit to convert it to RKNN.

2. Device-side API and application services

- **RKNN3 API**: The core interface layer, available in both C and Python, categorized by function into:
- **Basic API**: Fundamental operations such as model loading, initialization, query, memory management, setting the running cores, and destruction.
- **General inference API**: Inference interfaces for general vision models such as CNN / ViT.

- **Session API**: Inference interfaces for LLMs, supporting KVCache, custom sampling strategies, and other features.
- **rkllm3-server**: Provides an OpenAI-compatible API service with support for text, image, and audio input, simplifying the integration and invocation of large models.

3. Communication services

- **Coprocessor mode**: Uses **rknn3_transfer_proxy**, which provides a communication proxy between the host SoC and the RK1820 / RK1828 coprocessors and supports three connection types: PCIe / USB / Ethernet.
- **Non-coprocessor mode**: Uses **rknn3-server**, which runs on the RK3572 side, handles connected-board inference requests from the PC, and manages NPU resources.

4. Runtime and NPU driver

- **RKNN3 Runtime**: Responsible for model management, memory scheduling, inference task scheduling, multi-core parallel control, and Session management.
- **RKNN3 NPU driver**: Responsible for low-level operations such as task dispatch, interrupt synchronization, and load statistics.

In coprocessor mode, applications run on the host SoC and control model loading and inference execution on the RK1820 / RK1828 coprocessors through the RKNN3 API and rknn3_transfer_proxy. In non-coprocessor mode, applications run directly on the RK3572 and control the local NPU through the RKNN3 API. Users can perform model inference directly through the RKNN3 API, or invoke large model services through the OpenAI-compatible interface provided by rkllm3-server.

1.1.4 Relationship Among Toolkit / Runtime / Toolkit Lite / Server

The RKNN3 SDK provides four core components, each corresponding to a different stage of the model lifecycle. Users choose among them based on the use case:

Component	Running Location	Main Responsibilities	Typical Scenarios
RKNN3-Toolkit	PC	Model conversion, quantization, evaluation, connected-board debugging	Model conversion and evaluation stage
RKNN3 Runtime	On-device (RK182X / RK3572)	Model execution, memory management, multi-core scheduling	On-device inference (C API)
RKNN3-Toolkit Lite	On-device	Python inference interface	Quick on-device verification in Python
rkllm3-server	On-device	OpenAI-compatible API service	Service-oriented deployment of large models

The relationships and data flow among the components:

1. **Model conversion stage**: On the PC, the user uses RKNN3-Toolkit to convert an ONNX model into `.rknn` and `.weight` files. During this stage, quantization, performance evaluation, and accuracy analysis can be performed.

2. **On-device deployment stage**: Deploy the conversion outputs to the device, and load and run the model through RKNN3 Runtime (C API) or RKNN3-Toolkit Lite (Python). For LLM / MLLM, rkllm3-server can also provide an OpenAI-compatible service.
3. **Connected-board debugging stage**: RKNN3-Toolkit supports connected-board inference, where inference is initiated on the PC side and the model runs on the device-side NPU. This is used for model verification and accuracy analysis.

> **Note**: The versions of RKNN3-Toolkit and Runtime / Toolkit Lite must match. Otherwise, a model may convert successfully but fail to run on the device. For details, see 1.2.1 Version Compatibility Matrix.

1.1.5 Terms, Abbreviations, and Document Conventions

This section introduces the terms and abbreviations commonly used in this document.

Hardware and Deployment

- **Host SoC**: The main processor that runs user applications (such as RK3588 / RK3576), involved only in coprocessor mode.
- **Coprocessor**: A dedicated acceleration unit (such as RK1820 / RK1828) scheduled by the host SoC and sharing the computation workload via PCIe / USB / Ethernet interfaces, involved only in coprocessor mode.
- **RK182X**: The collective term for the RK1820 and RK1828 coprocessors. The two share the same architecture, and RK182X is used throughout this document to refer to both.
- **Connected-board inference**: The PC connects to the development board through the host SoC or via USB / network (adb connect). Inference is initiated by RKNN3-Toolkit on the PC side, and the model runs on the NPU. This is used for model verification and accuracy analysis.

Models and Tensors

- **RKNN model**: The model files that run on the NPU, consisting of two files: `.rknn` (model structure) and `.weight` (weights data).
- **LLM (large language model)**: Large Language Model, a large-scale language model based on the Transformer architecture, such as Qwen2.5 / Qwen3, used for tasks such as text generation and dialogue.
- **MLLM (multimodal large language model)**: Multimodal Large Language Model, a large model that extends the LLM to support multiple input modalities such as image, audio, and video, such as Qwen2.5-VL / Qwen3-VL.
- **Tensor**: A data structure representing a multi-dimensional array, consisting of data memory and attribute information (shape / dtype / layout, etc.).
- **Data type (dtype)**: The data type of tensor elements, such as FP32 / FP16 / INT8 / UINT8.
- **Data layout (layout)**: The arrangement of a tensor in memory, such as NCHW / NHWC / NC1HWC2.
- **NC1HWC2**: An NPU-optimized memory layout format that stores the channel dimension in blocks (16 sub-channels for FP16, 32 sub-channels for INT8) to improve NPU access

efficiency.

Quantization

- **Quantized model**: A model that uses low-precision data types such as INT8 and requires scale and zero_point parameters for quantization/dequantization.
- **Non-quantized model**: A model that uses the FP16 data type and requires no quantization parameters.

API and Features

- **Basic API**: Inference interfaces for general vision models such as CNN / ViT.
- **Session API**: Dedicated interfaces for LLMs, providing session management, KVCache / LoRA, sampling control, and other features.
- **Core mask (core_mask)**: Specifies which NPU cores the model runs on. For example, 0x3 indicates that core 0 and core 1 are used.
- **Dynamic shape**: A single model supports multiple input size combinations, which can be switched at runtime via `rknn3_set_shape()`.
- **KVCache**: The cached key-value attention states in a large language model, used to accelerate autoregressive generation.
- **LoRA**: Low-Rank Adaptation, an efficient model fine-tuning method.

Document Conventions

- Lines starting with `#` in code blocks are comments.
- Content enclosed in `<>` is a parameter placeholder and must be replaced according to the actual situation.
- Interface function names, parameter names, and file names are enclosed in backticks (`).
- For detailed descriptions of the API parameters involved in this document, refer to the corresponding API reference manuals.

1.2 SDK Components and Environment Constraints

> **Note**: This section describes the version compatibility, functional responsibilities, and environment constraints of the SDK components. For specific procedures such as Toolkit installation, hardware connection, and on-device environment deployment, refer to *RKNN3_SDK_Quick_Start*.

1.2.1 Version Compatibility Matrix

The versions of the RKNN3 SDK components (RKNN3-Toolkit, Runtime, communication services, and rkllm3-server) must match. Otherwise, problems may occur, such as a model converting successfully but failing to run on the device, or abnormal inference results.

Component Overview and Version Consistency

Component	Running Location	Function Description	Version Consistency Requirement
RKNN3-Toolkit	PC	Model conversion, quantization, evaluation	Same version as Runtime and the communication services
RKNN3 Runtime (librknn3_api.so)	On-device	Model execution, memory management, multi-core scheduling	Same version as Toolkit
rknn3_transfer_proxy	On-device	Communication proxy between the host SoC and the RK1820 / RK1828 coprocessors (coprocessor mode)	Same version as Toolkit / Runtime
rknn3-server	On-device	Handles connected-board inference requests from the PC and manages RK3572 NPU resources (non-coprocessor mode)	Same version as Toolkit / Runtime
rkllm3-server	On-device	OpenAI-compatible API service	Same version as Toolkit / Runtime

Scenarios Requiring a Version Consistency Check

- When switching to a new development machine or Docker environment
- After updating or reinstalling RKNN3-Toolkit (the communication services and Runtime must be updated in sync)
- When pulling code or models from a project / SDK shared by others
- When runtime errors or model loading failures occur

> **Note**: The RKNN model file records the Toolkit version used for conversion. If it does not match the on-device Runtime version, there is a risk of runtime errors. It is recommended to reconvert the model with a Toolkit of the same version as the Runtime. For version query methods, see 1.2.4 Environment Verification.

1.2.2 Host System Requirements

RKNN3-Toolkit runs on the PC for model conversion, quantization, and evaluation. The host environment must meet the following requirements:

Item	Requirement
Operating system	Ubuntu 20.04 / Ubuntu 22.04 / macOS (Beta)
Python	3.10 / 3.12 (only cp310 / cp312 supported as of V1.1.0)
Architecture	x86_64 (Ubuntu) / ARM64 (macOS Apple Silicon)
Core dependencies	onnx>=1.16 / onnxruntime>=1.22.0 / torch>=2.2 / numpy>=2.0

> **Note**: The requirements file shipped with the Toolkit is the authoritative source for the complete dependency list; only the core dependencies that deserve special attention are listed above. As of V1.1.0, Python support has converged to 3.10 / 3.12, and the cp36/cp37/cp38/cp39/cp311 requirements files from older versions are no longer provided. If dependency version conflicts cause installation failures, try switching the pip index (for example, `https://pypi.tuna.tsinghua.edu.cn/simple/`) or run `pip install <package> --no-deps` to skip dependency checks, but make sure the runtime dependencies are complete on your own. It is recommended to run GRQ quantization (RKQuantizer) in a CUDA or MPS environment; running it on the CPU takes a very long time.

1.2.3 On-Device Environment Requirements

The on-device environment differs depending on the deployment mode:

Coprocessor mode (RK182X)

Item	Requirement
Host SoC	RK3588 / RK3576, etc.
Coprocessor	RK1820 / RK1828
Host system	Android / Linux / Windows
Communication service	rknn3_transfer_proxy
Runtime	librknn3_api.so + librknn3_api_rkcp.so
Communication interface	PCIe / USB / Ethernet
Number of NPU cores	8

Non-coprocessor mode (RK3572)

Item	Requirement
SoC	RK3572
System	Android / Linux
Communication service	rknn3-server
Runtime	librknn3_api.so + librknn3_api_native.so
Number of NPU cores	1 (single core)

> **Note**: RK3572 supports only Linux on the aarch64 architecture and does not support x86_64. In RK182X coprocessor mode, the Runtime library depends on the host SoC architecture; select the library that matches the host.

1.2.4 Environment Verification

After the environment is deployed, verify that each component is ready using the following methods:

1. Verify the Toolkit installation

```
# Enter Python interactive mode and import the RKNN class
python
>>> from rknn.api import RKNN
# No errors indicate a successful installation
```

2. Verify the on-device communication service

- Coprocessor mode: Run `rknn3_transfer_proxy devices` in the host SoC terminal. The service works normally if the coprocessor devices are listed.
- Non-coprocessor mode: Run `rknn3-server -v` in the RK3572 terminal. The service works normally if the version number is printed.

3. Verify the device connection

```
# Query adb-connected devices on the PC
adb devices
# Device IDs listed indicate a normal connection
```

4. Verify version consistency

Confirm that the versions of Toolkit, Runtime, and the communication services match. Query the versions as follows:

```
# Query the rknn3-toolkit version
pip list | grep rknn3-toolkit

# Query the rknn3-toolkit version used for model conversion
strings xxxx.rknn | grep rknn3-toolkit

# Query the Runtime library version
strings /usr/lib/librknn3_api.so | grep -i "librknn3_api version"

# Coprocessor mode (RK182X): query the rknn3_transfer_proxy version
strings /usr/bin/rknn3_transfer_proxy | grep "Transfer version"

# Non-coprocessor mode (RK3572): query the rknn3-server version
/usr/bin/rknn3-server -v
```

> **Android platform paths**: On the Android platform, the library and executable paths differ from those on Linux. The query methods are the same, but the paths must be replaced: - Runtime library: `/system/lib64/librknn3_api.so` or `/vendor/lib64/librknn3_api.so` - rknn3-server: `/system/bin/rknn3-server` or `/vendor/bin/rknn3-server` - rknn3_transfer_proxy: `/system/bin/rknn3_transfer_proxy` or `/vendor/bin/rknn3_transfer_proxy`

1.3 Development Workflow and Interface Selection

1.3.1 General Model Development Workflow

The typical development workflow for vision models such as CNN / ViT consists of three stages: model conversion, model evaluation, and on-device deployment and execution. The overall workflow is shown in the following figure:

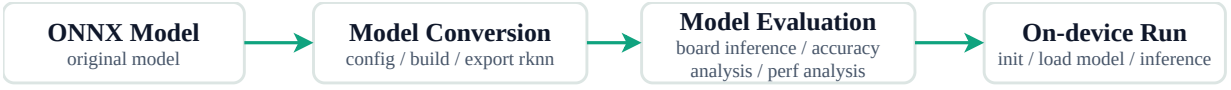


Figure 1-3 General model development workflow

1. **Model conversion**

Complete the following on the PC side with RKNN3-Toolkit:

a. Prepare the original model (in ONNX format). b. Configure parameters such as normalization, quantization, and target platform in RKNN3-Toolkit. c. Build the model via `rknn.build()`. Quantization can be enabled for higher performance and lower memory usage. d. Export the `.rknn` and `.weight` model files via `rknn.export_rknn()`.

1. **Model evaluation**

Connect the PC to the development board via USB or network (adb connect), and use the RKNN3-Toolkit Python API for connected-board evaluation:

a. Initialize the runtime environment for connected-board use via `rknn.init_runtime()`, and perform connected-board inference with `rknn.inference()` to verify functional correctness and refine pre-processing and post-processing logic. b. Accuracy analysis: Use `rknn.inference(accuracy_analysis=True)` on the simulator or a connected board to analyze the quantization error and the sources of on-device inference error. c. Performance analysis: Use `rknn.eval_perf()` to evaluate operator latency and optimize running-core scheduling.

1. **On-device deployment and execution**

Integrate inference into on-device applications using the RKNN3 Runtime C API:

a. Initialize the context via `rknn3_init()`, and load the `.rknn` and `.weight` files via `rknn3_load_model_from_path()`. b. Complete the model initialization configuration via `rknn3_model_init()` (including the multi-core mask `run_core_mask`), and query the input and output attributes via `rknn3_query()`. c. Allocate input and output memory via `rknn3_create_mem()`, synchronize the data via `rknn3_mem_sync()`, and invoke `rknn3_run()` to execute inference. d. Implement input pre-processing (data type conversion, layout

conversion) and output post-processing (dequantization, business logic), and integrate them into the actual business workflow.

> **Note**: For specific operations on model conversion, quantization, and evaluation, see Chapter 2 (Sections 2.1-2.4). For on-device deployment, see Chapter 3 (Sections 3.1-3.5).

1.3.2 LLM Development Workflow

The LLM development workflow differs from the CNN workflow mainly in the special handling of HuggingFace models, and in that on-device deployment uses the Session API, which differs from that of CNN models.

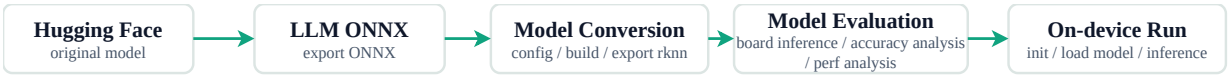


Figure 1-4 LLM model development workflow

The LLM model development workflow consists of three stages:

1. **Model conversion**

Complete the following on the PC side with RKNN3-Toolkit:

a. Export the HuggingFace model to ONNX. GRQ quantization with RKQuantizer is optional. b. Load the LLM model via `rknn.load_llm()`. c. Build the model via `rknn.build()`. Quantization can be enabled for higher performance and lower memory usage. d. Export deployment outputs such as `.rknn` / `.weight` / `.embed.bin` / `.tokenizer.gguf` via `rknn.export_rknn()`.

1. **Model evaluation**

Connect the PC to the development board via USB or network (adb connect), and use the RKNN3-Toolkit Python API for connected-board evaluation:

a. Initialize the runtime environment for connected-board use via `rknn.init_runtime()`, and perform connected-board inference with `rknn.inference()` to verify LLM model functionality and generation quality. b. Verify that features such as multi-turn dialogue and streaming generation work correctly.

1. **On-device deployment and execution**

Integrate inference on the device using the RKNN3 Runtime C API (Session interfaces) or rkllm3-server:

a. Load the model via `rknn3_init()` / `rknn3_load_model_from_path()`, and complete initialization via `rknn3_model_init()`. b. Create a session via `rknn3_session_init()`, and configure the Chat Template, callbacks, and sampling parameters. c. Perform session inference via `rknn3_session_run()`, with support for KVCache / LoRA, multimodal input, and more.

> **Note**: For detailed requirements of LLM model conversion (ONNX input conventions, KVCache, multiple shapes, etc.), see 2.2 LLM and MLLM Model Conversion. For Session deployment, see 3.3 LLM and MLLM Session Deployment.

1.3.3 MLLM Development Workflow

An MLLM (Multimodal Large Language Model) extends the LLM with non-text input capabilities such as vision and audio. The VLM (vision-language model) is the most common form.

An MLLM consists of a multimodal encoding part (such as Vision / Audio) and an LLM part. The overall development workflow consists of three stages: model conversion, model evaluation, and on-device deployment and execution. Each stage requires separate handling of the multimodal encoding part and the LLM part:

Multimodal encoding part development workflow

The multimodal encoding part is essentially a general model (such as a Vision model). Its development workflow follows 1.3.1 General Model Development Workflow: load it via `rknn.load_onnx()`, build it via `rknn.build()`, and export it via `rknn.export_rknn()`. On the device, run it through the general C API to obtain the multimodal embedding.

LLM part development workflow

The development workflow of the LLM part is identical to the 1.3.2 LLM Development Workflow. The difference is that during inference, the embedding output by the multimodal encoder must be fed as input together with the text prompt.

> **Note**: For conversion of the MLLM multimodal encoding part, see 2.2.2 Multimodal Encoding Part Export and 2.1 General Model Conversion. For multimodal input deployment, see 3.3.6 Multimodal Input.

1.3.4 Interface Selection

The RKNN3 SDK provides three on-device invocation methods. Users choose among them based on the scenario:

Method	Language	Applicable Models	Applicable Scenarios
RKNN3 Runtime (C API)	C / C++	General models, LLM / MLLM	Production deployment, performance-sensitive scenarios
RKNN3-Toolkit Lite	Python	General models, LLM / MLLM	Quick on-device verification, prototyping
rkllm3-server	HTTP API	LLM / MLLM	Service-oriented deployment, OpenAI ecosystem integration

Differences among the three:

- **Performance**: The C API delivers the best performance, followed by rkllm3-server (which incurs network and serialization overhead). Toolkit Lite is slightly lower due to the overhead of the Python wrapper.
- **Development convenience**: rkllm3-server is the most convenient (no programming required; start the service and invoke it via HTTP), followed by Toolkit Lite (Python programming, no cross-compilation required). The C API requires cross-compilation and integration.
- **Feature completeness**: The C API supports the full set of advanced features (custom sampling, KVCache management, dynamic LoRA switching, etc.); rkllm3-server supports common configurations; Toolkit Lite targets verification and covers fewer advanced features.

Selection recommendations:

- **General models**: Use the C API for production deployment and Toolkit Lite for on-device verification.
- **LLM / MLLM**: Use rkllm3-server to integrate with the OpenAI ecosystem or to quickly set up serving; use the Session interfaces of the C API when deep control over the inference flow is required.

1.3.5 Model Conversion and Deployment Outputs

The deployment outputs generated after model conversion are as follows:

General models (CNN / ViT)

File	Description
`.rknn`	RKNN model structure file
`.weight`	RKNN model weights file

LLM / MLLM models

File	Description
`xxx.rknn`	RKNN model structure file
`xxx.weight`	RKNN model weights file
`xxx.embed.bin`	Embedding layer weights; not required in Tied word embeddings mode
`xxx.tokenizer.gguf`	Tokenizer file, used to initialize the tokenizer during on-device deployment
`xxx.safetensors`	External ROPE cache file (required by some models, such as Gemma4)
`xxx_per_layer_inputs.embed.bin`	PLE (per_layer_inputs embed) file, the extra input for each layer (required by some models, such as Gemma4)

> **Note**: Transfer the above outputs to the device for deployment. C API deployment requires `.rknn` + `.weight` (for LLMs, `.embed.bin` + `.tokenizer.gguf` are also required). For Server deployment, specify the corresponding files using the parameters in 3.5 RKLLM3 Server Integration.

2. Model Conversion, Quantization, and Evaluation

2.1 General Model Conversion

This section describes the conversion flow for general models such as CNN / ViT, using RKNN3-Toolkit to convert ONNX models into RKNN models. The overall flow is: prepare the ONNX model → configure conversion parameters → load and build the model → export the RKNN model → verify the conversion result.



Figure 2-1 General model conversion flow

2.1.1 Model Preparation

General model conversion takes ONNX format as input. Users must first export the model to ONNX from the original training framework (PyTorch / TensorFlow, etc.):

- **ONNX export**: Complete the export on the original framework side. An opset version ≥ 11 is recommended. If the original model is in PyTorch, refer to the ONNX export scripts in the RKNN3 Model Zoo.
- **Input constraints**: When RKNN3-Toolkit loads an ONNX model, the model input name / shape / dtype must be specified. This information is used for subsequent `config` configuration and `load_onnx` loading.
- **Model inspection**: After export, use ONNX tools (such as `onnxruntime` or Netron) to check the model structure and operator support to ensure a correct export.

> **Note**: RKNN3-Toolkit currently supports only ONNX model loading. Models from other frameworks (TensorFlow / PyTorch, etc.) must be exported to ONNX first.

2.1.2 Conversion Configuration

During model conversion, the parameters of `rknn.config()` and `rknn.build()` affect the conversion result. `rknn.config()` configures parameters such as normalization, quantization, and target platform, while `rknn.build()` controls whether quantization is applied and specifies the quantization dataset.

Basic flow for determining conversion parameters

Refer to the following steps to determine the conversion parameters:

1. Determine the NPU platform (RK1820 / RK1828 / RK3572), and set `target_platform`.
2. Confirm whether the input is RGB or BGR, and set `quant_img_RGB2BGR`.
3. Confirm the normalization parameters, and set `mean_values` and `std_values`.
4. Confirm the model input size, which is used for `load_onnx()` loading.
5. Confirm the quantization bit width, and set `quantized_dtype`.

6. Confirm the quantization algorithm, and set `quantized_algorithm`.
7. Confirm whether to quantize, and set `do_quantization` of `rknn.build()`. When quantizing, specify the calibration dataset through `dataset`.

Parameter details

- **target_platform**: Target NPU platform. Supports `rk1820`, `rk1828`, and `rk3572`.
- **quant_img_RGB2BGR**: Whether quantization images undergo RGB2BGR conversion. Default is `False`. It takes effect only during the quantization stage. If the model is trained with RGB, set it to `False` or leave it unset, and input RGB images at inference; if the model is trained with BGR, set it to `True`, and input BGR images at inference. If the calibration data uses the npy format, do not use this parameter to avoid confusion. **Caution**: This parameter affects only the channel conversion when calibration set images are read during quantization; it does not affect input processing at inference. At inference, the input data is only mean-subtracted and divided by the standard deviation, and channel conversion must be completed by the user.
- **mean_values / std_values**: Mean and normalization values of the input, both in the same format. For a single input with N channels, the format is `[[channel_1, ..., channel_n]]`; for multiple inputs, `[[channels of input 1], [channels of input 2], ...]`. The normalization formula is `output = (input - mean) / std`, and the Toolkit normalizes the input according to this formula at inference. After configuration, no mean normalization is needed during the C API inference stage, reducing deployment time.
- **quantized_dtype**: Quantization type. Supports `w4a16` / `w6a16` / `w8a16` / `w16a16` / `w8a8`. Default is `w16a16`.
- **quantized_algorithm**: Quantization algorithm. Default is `normal`. `normal` is fast; a calibration set of 20-100 images is recommended. `mmse` achieves higher accuracy but is slower and memory-intensive; a calibration set of 20-50 images is recommended. `kl_divergence` takes time in between and suits uneven feature distributions; a calibration set of 20-100 images is recommended. Start with `normal`; try other algorithms only if the results are unsatisfactory.
- **quantized_method**: Quantization method. Supports `layer` / `channel` / `group{SIZE}`. Default is `None` (must be configured when `quantized_dtype` is not `w16a16`). `group{SIZE}` is valid only under `w4a16` / `w6a16` / `w8a16` and supports only MatMul / Linear type operators.
- **optimization_level**: Model optimization level. Default is `3` (all optimizations enabled). Values 2/1 disable some optimizations; 0 disables all optimizations.
- **input_attrs**: Input attributes, such as dtype / layout / shape / stride.
- **dynamic_input**: Dynamic input configuration. Specifies multiple groups of input shapes to simulate dynamic input. Default is `None`.

For the format and how to fill in the quantization calibration dataset, see section 2.3 Model Quantization. For more detailed parameter descriptions, refer to the *RKNN3_Toolkit_Python_API_Reference*.

Example:

```
rknn.config(
    mean_values=[[103.94, 116.78, 123.68]],
    std_values=[[58.82, 58.82, 58.82]],
    quant_img_RGB2BGR=False,
    input_attrs={'pixel': {'dtype': 'uint8', 'layout': 'NHWC'}},
    target_platform='rk1820')
```

2.1.3 Model Loading and Building

1. Initialize the RKNN object

The first step of the conversion flow is to initialize the RKNN object. Pass in `verbose` to control log output:

```
from rknn.api import RKNN
rknn = RKNN(verbose=True, verbose_file='./mobilenet_build.log')
```

2. Load the ONNX model

Invoke `rknn.load_onnx()` to load the ONNX model. Provide the model file path:

```
rknn.load_onnx(model='./arcface.onnx')
```

3. Build the RKNN model

Invoke `rknn.build()` to build the model. Choose whether to quantize:

- `do_quantization`: Whether to quantize. Default is `True`.
- `dataset`: Quantization calibration dataset in text file format, with one calibration sample path per line.

dataset.txt example:

```
./imgs/ILSVRC2012_val_00000665.JPEG
./imgs/ILSVRC2012_val_00001123.JPEG
./imgs/ILSVRC2012_val_00001129.JPEG
```

Build example:

```
rknn.build(do_quantization=True, dataset='./dataset.txt')
```

> **Note**: If `do_quantization=False`, quantization is skipped and the model is built in floating-point (w16a16) form. No dataset is required.

2.1.4 RKNN Model Export

After the build completes, invoke `rknn.export_rknn()` to export the RKNN model files for on-device deployment. The `export_path` parameter specifies the export file path. Based on this

path, the Toolkit generates two files: `.rknn` (model structure) and `.weight` (weights data):

```
rknn.export_rknn(export_path='./mobilenet_v1.rknn')
```

The example above generates `mobilenet_v1.rknn` and `mobilenet_v1.weight` in the current directory. After the conversion flow completes, invoke `rknn.release()` to release resources:

```
rknn.release()
```

2.1.5 Conversion Result Verification

After export, invoke `rknn.inference()` to run inference verification on the RKNN model. Compare the outputs with the original ONNX model to confirm that the conversion result is correct.

> **Note**: For more detailed evaluation methods such as accuracy analysis and performance evaluation, see section 2.4 Model Evaluation.

2.2 LLM and MLLM Model Conversion

This section describes the conversion flow for LLM and MLLM models.

LLM / MLLM models are usually stored in SafeTensors format (a safe tensor format developed by Hugging Face) and are loaded and managed through the HuggingFace ecosystem. Therefore, conversion takes HuggingFace-format models as input. Because RKNN3-Toolkit cannot convert SafeTensors / HuggingFace models directly, first export them to ONNX and accompanying files, and then convert them into RKNN models.

Differences from general models

- **Input format**: General models take ONNX as input; LLM / MLLM models take HuggingFace-format models (usually SafeTensors) as input and require an additional export step.
- **Export step**: An LLM must first export 4 types of files from the HuggingFace model (ONNX model, config, tokenizer, and embedding weights). The export depends on `transformers` (to load the model) and `torch` (`torch.onnx.export`); `export_llm.py` in rknn3-model-zoo wraps this export logic for direct use, and users can also refer to its implementation and export on their own.
- **Conversion tool**: Conversion still uses RKNN3-Toolkit, but it loads models with `rknn.load_llm()` and configures quantization, core, and attention parameters with `llm_config`, unlike the `rknn.load_onnx()` / `rknn.config()` used for general models.

MLLM specifics

An MLLM consists of a multimodal encoding part (such as Vision / Audio) and an LLM part. The two parts are converted separately and independently:

- **LLM part**: The export and conversion methods are the same as for LLM models (see 2.2.1~2.2.3).

- **Multimodal encoding part**: It is essentially a general model. Export it as an independent ONNX and convert it following the general model conversion method in 2.1 (see 2.2.2 Multimodal Encoding Part Export for details).
- **Joint usage**: The two parts are used together at on-device inference, where the LLM receives the embeddings output by the multimodal encoder.

The overall flow of LLM model conversion is shown below:



Figure 2-2 LLM model conversion flow

2.2.1 Hugging Face Model Preparation

LLM / MLLM model conversion takes HuggingFace-format models as input. Users must first prepare HuggingFace-format model files (including model weights, config.json, tokenizer.json, etc.), which can be loaded with `AutoModelForCausalLM` / `AutoConfig` from the `transformers` library:

```
from transformers import AutoModelForCausalLM, AutoConfig
config = AutoConfig.from_pretrained(args.model_path, **kwargs)
model = AutoModelForCausalLM.from_pretrained(args.model_path, **kwargs)
```

> **Note**: An MLLM model consists of a multimodal encoding part (such as Vision / Audio) and an LLM part. Prepare and convert them separately. For multimodal encoding part export, see 2.2.2; for its conversion, see 2.1.

2.2.2 ONNX, Config, Tokenizer, and Embedding Export

Before LLM model conversion, export 4 types of files from the HuggingFace model: ONNX model, config, tokenizer, and embedding weights. rknn3-model-zoo wraps these processes in `export_llm.py`, with the corresponding interfaces `causal_llm_to_onnx`, `export_llm_config`, `export_tokenizer`, and `export_embed_weight`.

1. Export the ONNX model

Convert the LLM Torch model into an ONNX model. Note that inputs must match the original torch definition; some models may not follow the `input_ids` / `attention_mask` / `position_ids` order:

```
dummy_input = torch.zeros((1, in_len), dtype=torch.long)
attention_mask = torch.ones((1, in_len), dtype=torch.float)
position_ids = torch.arange(0, in_len, dtype=torch.long).unsqueeze(0)
inputs = (dummy_input, attention_mask, position_ids)
input_names = ["input_ids", "attention_mask", "position_ids"]

torch.onnx.export(
    model, inputs, args.export_llm_path,
    export_params=True, opset_version=19,
    do_constant_folding=True,
    input_names=input_names, output_names=output_names,
    dynamic_axes=dynamic_axes)
```

> **Note**: Among the ONNX model inputs, `attention_mask` is required; the other inputs (`input_ids` / `position_ids` / `num_logits_to_keep` / `per_layer_inputs`, etc.) depend on the actual model. RKNN3-Toolkit classifies attention ops into N types based on the connection relationships of `attention_mask` and configures each type separately. The ONNX model must not have KVCache inputs/outputs (usually exported with `use_cache` set to False in the config).

For MLLM models, in addition to the LLM part ONNX export above, the multimodal encoding part (such as Vision / Audio) must also be exported separately as an independent ONNX model. rknn3-model-zoo wraps this process in scripts such as `export_vision.py`. The export method is similar to general model ONNX export and is completed with `torch.onnx.export`. The multimodal encoding part is converted the same way as a general model; see 2.1 General Model Conversion for details.

2. Export the config file

The config file is required by RKNN3-Toolkit for RKNN model conversion. It contains model specification parameters such as chat template, vocab size, and hidden size, as well as quantization-related configuration:

```
export_llm_config(args.model_path, os.path.splitext(args.export_llm_path)[0] +
'.config.pkl', chat_context, prompt)
```

3. Export the tokenizer file

The tokenizer converts between text and token ids. On-device inference requires the `tokenizer.gguf` file and the `libtokenizer.a` library:

- `tokenizer.gguf`: Exported with `export_tokenizer`. It contains tokenization and vocabulary information and corresponds to the `tokenizer.json` of the LLM model.
- `libtokenizer.a`: Parses the vocabulary information in `tokenizer.gguf` and provides interfaces such as text-to-token-id and token-id-to-text conversion. rknn3-model-zoo uses the llama.cpp tokenizer; the source code is at `rknn3_model_zoo/tokenizer`, and the build artifact `libtokenizer.a` is in `rknn3_model_zoo/3rdparty/tokenizer`. To adjust it, modify the code under `tokenizer/src` and rebuild.

```
export_tokenizer(args.model_path, os.path.splitext(args.export_llm_path)[0] +
'.tokenizer.gguf')
```

4. Export the embedding file

The embedding file contains the embedding data of the LLM model (Float16 type), i.e., the vocabulary vectors, stored in the original layout of the embedding weights in the Torch model, with shape (vocab_size, embedding_dim):

```
export_embed_weight(model.model.embed_tokens.weight,
os.path.splitext(args.export_llm_path)[0] + '.embed.bin')
```

> **Note**: Embedding layer operations run on the CPU side, which looks up the embedding data by token id and transfers it to the NPU. If the model's embedding layer operations are complex, embed.bin can be omitted and the user can manage the embedding in application logic. embed.bin is also not required in Tie Word mode.

2.2.3 LLM and MLLM Model Conversion

2.2.3.1 LLM MODEL CONVERSION

The LLM model conversion flow is: configure `llm_config` -> load with `rknn.load_llm()` -> build with `rknn.build()` -> export with `rknn.export_rknn()`.

1. Configure llm_config

For LLM models, `rknn.config()` adds the `llm_config` parameter (Python dictionary type). Users can first load the default configuration `DEFAULT_RKNN_LLM_CONFIG` and then modify it as needed:

```
from rknn.api import RKNN, DEFAULT_RKNN_LLM_CONFIG
rknn = RKNN(verbose=True)
my_config = DEFAULT_RKNN_LLM_CONFIG.copy()
rknn.config(target_platform='rk1820', llm_config=my_config)
```

`llm_config` mainly has 4 dictionary sub-items:

- **attention_config**: Configures the inference mechanism and KVCache of attention ops. The model must have a mask input with the attention structure. The Toolkit classifies attention ops into N types based on the mask connection relationships and configures each separately. Common fields:
- `mask_name`: Mask input name (required, e.g., `attention_mask`).
- `kvcache_buffer_len`: KVCache buffer length. Once exceeded, entries are overwritten cyclically in FIFO order. Must be 32-aligned.
- `tokens_per_page`: Number of tokens per page for Paged Attention; set to `None` to use the original contiguous KVCache (see 4.2.6). Default is `None`; a positive integer enables paged KVCache, and a multiple of 64 is recommended. Larger pages yield better inference

performance, but the performance gain will not be significant. Non-positive values or the SlidingAttention type fall back to `kvcache_buffer_len` automatically.

- `request_num`: Number of paged Attention requests. Only 1 is supported in the current version.
- `kvcache_dtype`: KVCache data type. Supports `Float16` / `Int8_to_F16` / `Int4_to_F16`, with memory usage decreasing in that order.
- `attention_type`: Attention type. Supports `FullAttention` / `SlidingAttention`.
- `position_name`: Position encoding input name, used to extract RoPE. Set to None when the model has no position encoding.
- `max_position_embeddings`: Maximum position encoding length. Must be 32-aligned.
- `position_embeddings_host_storage`: Storage location of position encodings (RoPE): `False` stores them in device memory, `True` in Host memory.
- `mrope_type`: mrope type. Supports `Qwen2.5-VL` / `Qwen3-VL` / `Qwen3.5`. When enabled, `position_name` must be configured.
- `mrope_section`: mrope section information. For Qwen2.5-VL / Qwen3-VL / Qwen3.5 models, the corresponding configuration values can be found in config.json.
- `mrope_new_id_name`: Name of the newly generated mrope position input. After mrope is enabled, a new input with [seq_len, 3] dimensions is generated.
- **llm_head**: Configures the optimization behavior of the LLM head layer (usually a large Linear layer). Common fields:
- `device`: Compute device. Supports `rk1820` / `rk3588` / `rk3576`.
- `quantized_dtype`: Quantization type. Supports `w16a16` / `w4a16` / `w6a16` / `w8a8`. Forced to `w8a8` on rk3576/rk3588.
- `quantized_method`: Quantization method. Supports `layer` / `channel` / `group{SIZE}`. Forced to `channel` on rk3576/rk3588.
- **vocab**: Configures vocabulary layer optimization. When `tie_embeddings` is True, vocab shares weights with llm_head (Tie Word mode) and embed.bin is not required.
- **keep_one_logit**: Configures taking a single value along the specified output axis (a gather op is inserted) to reduce lm_head computation in the Prefill stage. `idx_name` specifies the gather indices input name.

> **Note**: For the complete fields and default values of each `llm_config` sub-item, see `DEFAULT_RKNN_LLM_CONFIG`. For more detailed parameter descriptions, refer to the *RKNN3_Toolkit_Python_API_Reference*.

2. Load the LLM model

Invoke `rknn.load_llm()` to load the ONNX model. Parameters include `model` (ONNX path), `config` (config.pkl path), and `seq_lens` (list of input token counts supported per inference):

```
rknn.load_llm(model='./Qwen2.5-3B.onnx',
              config='./Qwen2.5-3B.config.pkl',
              seq_lens=[1, 128])
```

> **Note**: `seq_lens` configures multiple shape inputs (e.g., `[1, 64]` generates a model with prefill length 64 and decoder length 1). It applies only to models with standard inputs (input_ids / attention_mask / position_ids / num_logits_to_keep); for models with non-standard inputs, use the `dynamic_input` parameter of `rknn.config()`.

3. Build and export

Invoke `rknn.build()` to build the model (quantization can be enabled), then invoke `rknn.export_rknn()` to export the deployment artifacts:

```
rknn.build(do_quantization=True)
rknn.export_rknn(args.rknn_path)
```

> **Note**: The `dataset` (calibration dataset) is not required for LLM model conversion. The `dataset.txt` file is used only in general model conversion with quantization (see 2.1 / 2.3.3).

Exported artifacts include `.rknn`, `.weight`, `.embed.bin`, `.tokenizer.gguf`, etc. (see 1.3.5 Model Conversion and Deployment Artifacts). For loading LoRA weights, see 4.2.8 LoRA.

2.2.3.2 MLLM MODEL CONVERSION

An MLLM model consists of a multimodal encoding part (such as Vision / Audio) and an LLM part, and the two parts are converted separately and independently. The multimodal encoding part is essentially a general model: it is loaded with `rknn.load_onnx()`, built, and exported (see 2.1 for the conversion method). The LLM part is converted following the LLM model conversion flow above (configure `llm_config` -> `rknn.load_llm()` -> `rknn.build()` -> `rknn.export_rknn()`). The two parts are used together at on-device inference.

Taking the Qwen2.5-VL-3B model as an example, rknn3-model-zoo exports two files: `Qwen2.5-VL-3B-vision.onnx` (the vision encoder for image feature extraction) and `Qwen2.5-VL-3B-llm.onnx` (the LLM part for text generation). They are converted separately as follows:

```
# Build the vision model
rknn.config(target_platform='rk1820',
              quantized_dtype='w4a16', quantized_algorithm='normal',
              quantized_method='group32', core_num=8)
rknn.load_onnx(model='../../model/vision/Qwen2.5-VL-3B-vision.onnx')
rknn.build(do_quantization=True, dataset=args.dataset_path)
rknn.export_rknn('../../model/vision/Qwen2.5-VL-3B-vision.rknn')

# Build the LLM model
rknn.config(target_platform='rk1820',
              quantized_dtype='w4a16', quantized_algorithm='normal',
              quantized_method='group32')
rknn.load_llm(model='../../model/llm/Qwen2.5-VL-3B-llm.onnx',
            config='../../model/llm/Qwen2.5-VL-3B-llm.config.pkl')
rknn.build(do_quantization=True)
rknn.export_rknn('../../model/llm/Qwen2.5-VL-3B-llm.rknn')
```

> **Note**: At on-device inference, the vision model uses the general RKNN3 API (`rknn3_run()`, etc.) and obtains image embeddings (img_embeds) through inference. At LLM inference, the prompt and img_embeds are passed into the LLM model together for multimodal inference. For multimodal input deployment, see 3.3.6 Multimodal Input.

2.2.4 Model and Tokenizer Adaptation

This section describes how to use the tokenizer to convert between text and token ids when deploying LLM / MLLM models on device. The tokenizer converts text into token ids for model inference, and converts the output token ids back to text.

Tokenizer files and library

On-device inference requires the following two files:

- `tokenizer.gguf`: Tokenizer file containing tokenization rules and vocabulary information. It corresponds to the `tokenizer.json` of the LLM model and is exported with `export_tokenizer` during model conversion (see 2.2.2).
- `libtokenizer.a`: Tokenizer library that parses `tokenizer.gguf` and provides interfaces such as text-to-token-id and token-id-to-text conversion. rknn3-model-zoo uses the tokenizer

from llama.cpp, located at `rknn3_model_zoo/3rdparty/tokenizer`. To adjust it, modify the source code and rebuild.

Tokenizer class

rknn3-model-zoo wraps the `Tokenizer` class (header file `Tokenizer.h`). Its main interfaces are as follows:

- `Tokenizer(type, tokenizer_path)`: Constructor. Loads `tokenizer.gguf` and initializes the tokenizer. `type` takes `TOKENIZER_BACKEND_LLAMA`.
- `GetVocabInfo(&info)`: Gets vocabulary information (`vocab_size`, `special_bos_id`, `special_eos_id`, `linefeed_id`, etc.) for LLM session initialization.
- `Tokenize(text, text_len, tokens, n_tokens_max)`: Tokenizes text into a token id sequence and returns the number of tokens.
- `TokenToPiece(token)`: Converts a single token into a string piece for streaming output.
- `Decode(tokens, n_tokens)`: Restores a token sequence into a complete string for non-streaming output.

Using the tokenizer with the model

In on-device deployment, the tokenizer is registered to the LLM session via `tokenizer_callback` of `RKLLMCallback`, and the Runtime invokes it at inference to convert text into token ids. The usage flow is as follows:

1. Create a `Tokenizer` object and get the vocabulary information:

```
#include "Tokenizer.h"

// tokenizer_path is the path to the tokenizer.gguf file
Tokenizer* tokenizer = new Tokenizer(TOKENIZER_BACKEND_LLAMA, tokenizer_path);
VocabInfo vocab_info;
tokenizer->GetVocabInfo(&vocab_info);
```

1. Implement `tokenizer_callback`, and invoke `Tokenize` in the callback to convert text into token ids:

```
int tokenizer_callback(void *userdata, const char *text, int32_t text_len,
                       int32_t *tokens, int32_t n_tokens_max)
{
    Tokenizer *tokenizer = (Tokenizer *)userdata;
    int n_tokens = tokenizer->Tokenize(text, text_len, tokens, n_tokens_max);
    return n_tokens;
}
```

1. Register the tokenizer to the session callback. At inference, the Runtime automatically invokes `tokenizer_callback` for tokenization:

```
RKLLMCallback callback;
memset(&callback, 0, sizeof(RKLLMCallback));
callback.tokenizer_callback = tokenizer_callback;
callback.tokenizer_userdata  = tokenizer;
rknn3_session_set_callback(session, &callback);
```

1. In the result callback, use `TokenToPiece` or `Decode` to convert output tokens back to text:

```
// Streaming output: convert to string token by token
std::string piece = tokenizer->TokenToPiece(result->token_ids[0]);
// Non-streaming output: batch convert to string
std::string text = tokenizer->Decode(result->token_ids, result->num_tokens);
```

> **Note**: `tokenizer_callback` is an optional callback, used only when a custom tokenizer is needed. If not registered, the Runtime uses the default tokenization logic. For detailed descriptions of tokenizer-related interfaces, see `LLMTokenizerCallback` in the *RKNN3_Runtime_C_API_Reference*.

2.2.5 Conversion Result Verification

After LLM / MLLM model conversion, verify the results with rknn3-toolkit-lite (the on-device Python inference library). LLM models generate autoregressively. Set `llm_mode=True` at initialization, load the model, run inference verification, and check whether the model generates text (or image + text results) correctly:

```
from rknn3lite.api import RKNN3Lite

rknn_lite = RKNN3Lite(llm_mode=True)
rknn_lite.load_rknn(model_path='./Qwen2.5-3B-Instruct.rknn',
                    weight_path='./Qwen2.5-3B-Instruct.weight')
# Run inference verification
...
rknn_lite.release()
```

> **Note**: For detailed usage of rknn3-toolkit-lite, see 3.4 Toolkit Lite Python Deployment. For more detailed evaluation methods such as accuracy analysis and performance evaluation, see section 2.4 Model Evaluation.

2.2.6 Segmented Model Export

This section focuses on splitting an LLM into multiple consecutive Transformer-block segments and exporting them as separate ONNX / RKNN segments. Whether a model uses a multi-card pipeline and whether the model itself is an MoE model are independent choices; MoE conversion is described in Section 2.2.7.

Segmenting principle

Segment boundaries are placed between Transformer blocks without changing the computation order inside a block. The last segment always includes the final norm and `lm_head` in addition to its trailing blocks:

```
stage0: block 0 ... block k0
stage1: block k0+1 ... block k1
...
stageN-1: block kN-2 ... block L-1 + norm + lm_head
```

The number of segments is determined by the model size, target-device memory, and deployment mode. For a multi-card pipeline, one segment usually maps to one RK1820 / RK1828 card. Segmentation must account for the block weights, the `norm` / `lm_head` in the last segment, the KV cache, and internal scratch memory; simply dividing the number of blocks evenly is not sufficient.

Segmentation parameters

- `multi_segment`: Enables multi-segment ONNX / RKNN export.
- `num_segments N`: Specifies the number of segments. If omitted or set to 0, the script derives the number from the model depth and the memory planning strategy.
- `segment_layer_counts c0 c1 ...`: Explicitly specifies the number of consecutive blocks in each segment. The list length is the number of segments; every value must be positive and the sum must equal the total number of model blocks. This parameter takes precedence over `num_segments`.
- `max_context_len`: Sets the KV cache size and maximum context length for each segment. Select it together with the memory capacity of the target card.

When `segment_layer_counts` is not specified, the export script searches for a relatively balanced consecutive split based on weight size and target-card capacity, while reserving space for `norm` / `lm_head` in the last segment. The generated `segment_plan.json` records the actual split, the suggested context length, and the estimated memory per card. Final decisions should still be based on each segment's `model_report.html` and on-device results.

Segment export flow

For example, to split Qwen3.5-9B into two segments, export the ONNX segments first and then build RKNN for each segment:

```
cd rknn3-model-zoo/examples/multicard/python/qwen3_5
python export_llm_segment.py --model_path /path/to/Qwen3.5-9B --multi_segment --
num_segments 2
python export_rknn_segment.py --multi_segment --num_segments 2
```

To specify the split manually, for example 8, 8, and 16 blocks for a 32-block model:

```
python export_llm_segment.py --model_path /path/to/model --multi_segment --
segment_layer_counts 8 8 16
python export_rknn_segment.py --multi_segment --num_segments 3
```

The exported artifacts include:

Artifact	Purpose
`model_seg0.onnx` ... `model_segN.onnx`	ONNX graphs for the consecutive block segments
`model_seg0.rknn`, `model_seg0.weight` ... `model_segN.rknn`, `model_segN.weight`	RKNN graphs and external weights for each segment
`model.tokenizer.gguf`, `model.embed.bin`	Host-side tokenizer and the embedding file for models without tied embeddings
`model_segN.safetensors`	Cache file required by models using an external RoPE cache
`model.segment_plan.json`	Reference plan for the automatic split, context length, and memory per card
`tmp_segN/model_report.html`	Actual memory report after building each RKNN segment

If only the KV cache length is changed, an existing `tmp_segN` directory can be reused with `rebuild`. If the ONNX model, number of segments, or block split is changed, repeat the complete ONNX and RKNN export flow.

2.2.7 MoE Model Conversion

MoE model conversion and multi-card deployment are independent concerns. An MoE model can be converted and run as a single LLM model on one card; only when the model size, weights, or runtime memory exceed the capacity of one card is it necessary to use the segmentation method in Section 2.2.6 and deploy it across multiple cards.

MoE graph conversion

An MoE model contains a router, routed experts, and a shared expert. During export, a regular ONNX trace must not retain only the experts selected by the dummy input. The MoE export scripts in `examples/multicard` convert data-dependent expert dispatch into `rknn::exUnifiedMoE` graph nodes and retain the complete expert weights.

In the current version, `exUnifiedMoE` is directly supported by the matching RKNN3 Toolkit / Runtime, so MoE conversion and inference do not require an additional plugin. Export parameters and input/output conventions are model-specific and should follow the corresponding script.

MoE and model segmentation

If an MoE model needs to be segmented, split it consecutively between Transformer blocks and do not use expert parallelism. The router, all routed experts, and the shared expert of one MoE layer must remain in the same segment. The segment planner must account for the complete expert weights and MoE workspace rather than estimating memory only from the top-k experts activated by a particular input. If segmentation is not needed, do not enable `multi_segment` merely because the model is MoE.

Relevant model paths include:

Model	Export script
Qwen3-MoE (for example, Qwen3-30B-A3B)	`examples/multicard/python/qwen3/`
Qwen3.5-MoE	`examples/multicard/python/qwen3_5/`

Model	Export script
MiniCPM-V-4.7	`examples/multicard/python/minicpm4_7/` (contains `llm/` and `vision/`)
MiniCPM5-15B-A2.5B	`examples/multicard/python/minicpm5/`

2.3 Model Quantization

2.3.1 Quantization Basics

2.3.1.1 QUANTIZATION DEFINITION

Model quantization converts the floating-point parameters or activation values of a deep learning model into fixed-point representations, such as converting FP32 to INT8. Quantization reduces memory usage and enables model compression and inference acceleration, but usually introduces some accuracy loss.

Based on the form of the mapping function between floating-point parameters and fixed-point numbers, quantization falls into linear quantization and non-linear quantization. Common non-linear quantization includes exponential quantization, codebook quantization, and piecewise linear quantization. RKNN3-Toolkit supports only linear quantization. In linear quantization, the intervals between fixed-point numbers are uniform. For example, INT8 linear quantization divides the quantization range evenly into 256 values, so it is also called uniform quantization. Depending on whether the quantization zero point is 0, linear quantization falls into symmetric quantization and asymmetric quantization. Symmetric quantization is a simplified form of asymmetric quantization. Asymmetric quantization usually handles uneven data distributions better, so asymmetric quantization schemes are more commonly used in practice.

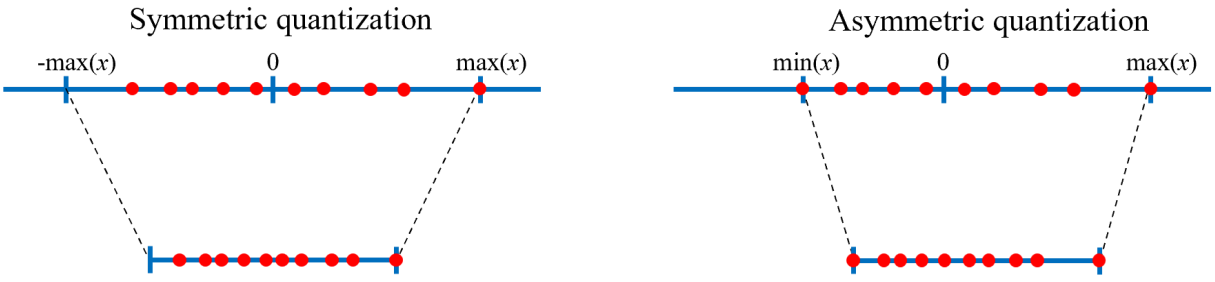


Figure 2-3 Linear symmetric quantization and linear asymmetric quantization

2.3.1.2 QUANTIZATION COMPUTATION

Taking linear asymmetric quantization as an example, the computation for quantizing a floating-point number into a signed fixed-point number is as follows:

$$x_{int} = clamp(\lfloor \frac{x}{s} \rceil + z; -2^{b-1},\ 2^{b-1} - 1) \tag{2-1}$$

where $x is the floating-point number, x_{int} is the quantized fixed-point number, \lfloor \cdot \rceil is the round-to-nearest operation, s is the quantization scale factor, z is the quantization zero point, and b is the quantization bit width (for the INT8 data type, b is 8); clamp$ is the clamping operation, defined as follows:

$$clamp(x;a,c) = \begin{cases} a, & x < a, \\ x, & a \le x \le c, \\ c, & x > c, \end{cases} \tag{2-2}$$

Converting a fixed-point number back into a floating-point number is the dequantization process, defined as follows:

$$x \approx \widehat{x} = s(x_{int} - z) \tag{2-3}$$

where $\widehat{x}$ is also called the fake-quantized floating-point number. Clearly, due to the rounding and clamping operations in quantization computation, $\widehat{x}$ does not equal x, and the difference between them is called the quantization error.

Given the quantization range $(q_{\min}, q_{\max}) and the clamping range (c_{\min}, c_{\max})$, $the quantization parameters s and z$ are computed as follows:

$$s = \frac{q_{\max} - q_{\min}}{c_{\max} - c_{\min}} = \frac{q_{\max} - q_{\min}}{2^b - 1} \tag{2-4}$$

$$z = c_{\max} - \lfloor \frac{q_{\max}}{s} \rceil \text{ or } z = c_{\min} - \lfloor \frac{q_{\min}}{s} \rceil \tag{2-5}$$

The clamping range is determined by the quantized data type; for example, the clamping range of INT8 is (-128, 127). The quantization range is determined by the specific quantization algorithm. Setting $z = 0$ in the definition formulas above yields the computation definition of linear symmetric quantization.

2.3.1.3 QUANTIZATION GRANULARITY

Quantization granularity specifies which parameters in a model's floating-point tensors are quantized as a whole. RKNN3-Toolkit supports three quantization granularities: Per-Layer / Per-Channel / Group. Per-Layer quantization quantizes all channels of a network layer's weights or activations as a whole, with all channels sharing the same quantization parameters. Per-Channel quantization quantizes each output channel of a network layer's weights independently, with each channel having its own quantization parameters. Group quantization applies only to linear layer weights. Based on Per-Channel quantization, it groups the elements within each output channel, and each group is quantized independently.

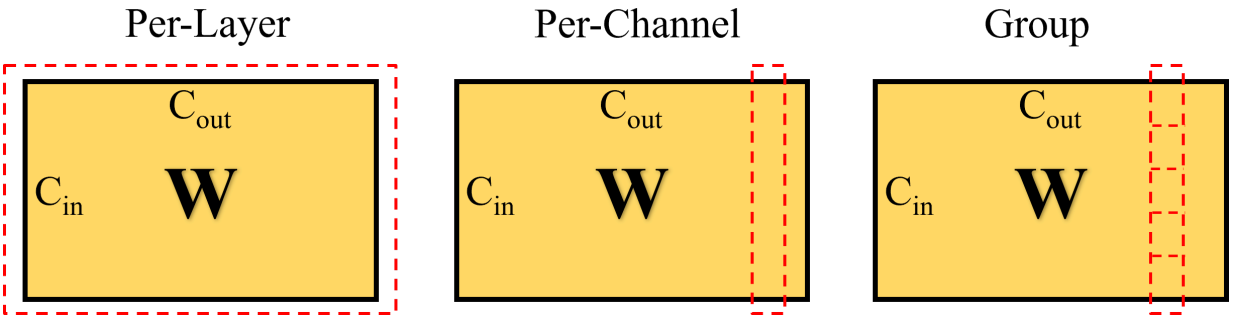


Figure 2-4 Per-Layer quantization, Per-Channel quantization, and Group quantization

> **Note**: In RKNN3-Toolkit, Group quantization and Per-Channel quantization apply only to weights; activations and intermediate values still use Per-Layer quantization; Group quantization is valid only for linear layers under a16 conditions.

2.3.1.4 QUANTIZATION ALGORITHMS

A quantization algorithm is a scheme that, under a given quantization form (such as linear quantization), solves for the low-bit fixed-point representation of a floating-point tensor and its quantization parameters. By implementation, quantization falls into two categories: quantization-aware training (QAT) and post-training quantization. QAT fine-tunes model parameters and quantization parameters during training to improve accuracy. It usually requires in-depth understanding of the training pipeline and large amounts of training data, so it is rarely used directly in model deployment toolchains. Post-training quantization usually needs only a small amount of calibration data, or even none, and obtains quantization parameters through low-cost computation. RKNN3-Toolkit supports four post-training quantization algorithms: Normal / KL-Divergence / MMSE, and GRQ.

2.3.2 Toolkit Quantization Flow Overview

RKNN3-Toolkit provides two flows: internal quantization and RKQuantizer quantization. The internal quantization flow first exports the floating-point model to a floating-point ONNX model, then exports the RKNN model through `rknn.config()` ⟶ `rknn.load_llm()/rknn.load_onnx()` ⟶ `rknn.build()` ⟶ `rknn.export_rknn()`. This flow offers strong compatibility, supports models such as CNN / ViT / LLM / MLLM, and can use the three quantization algorithms Normal / KL-Divergence / MMSE, but it does not support the GRQ quantization algorithm.

The RKQuantizer quantization flow mainly targets standard-architecture models such as ViT / LLM / MLLM. This flow uses RKQuantizer, the quantization component in the Toolkit, to run quantization algorithms such as GRQ, replacing the floating-point weights in the model with fake-quantized weights and exporting a fake-quantized ONNX model; the RKNN model is then exported through the same `rknn.config()` ⟶ `rknn.load_llm()/rknn.load_onnx()` ⟶ `rknn.build()` ⟶ `rknn.export_rknn()` chain. Importantly, when converting ONNX to RKNN, `rknn.config()` must set `quantized_algorithm='normal'` — `MMSE` or `KL-Divergence` must not be used — and must set the correct `quantized_dtype` (`w4a16`, `w6a16`, or `w8a16`) and `quantized_method='group32'`. This chapter gives a brief introduction to RKQuantizer usage. For more detailed interface and usage instructions (including mixed-precision quantization, calibration data preparation, and GPU memory usage), refer to the *RKNN3_Toolkit_Python_API_Reference*.

Internal quantization flow:

```
Floating-point PyTorch model ⟶ export floating-point ONNX ⟶ rknn.config() ⟶
rknn.load_llm()/rknn.load_onnx() ⟶ rknn.build() ⟶ rknn.export_rknn()
```

RKQuantizer quantization flow:

```
Floating-point PyTorch model ⟶ RKQuantizer ⟶ fake-quant PyTorch model ⟶ export
fake-quant ONNX ⟶ rknn.config() ⟶ rknn.load_llm()/rknn.load_onnx() ⟶
rknn.build() ⟶ rknn.export_rknn()
```

2.3.3 Internal Quantization Flow

2.3.3.1 DATA TYPES

Internal quantization supports the quantization data types `w4a16`, `w6a16`, `w8a16`, `w16a16`, and `w8a8`, with the default `w16a16`. Here, w4/w6/w8/w16 mean the weights are quantized to the corresponding bit width (int4/int6/int8/int16 or float16), and a8/a16 mean the activations are quantized to int8/float16.

2.3.3.2 INTERNAL QUANTIZATION ALGORITHMS

Internal quantization supports three quantization algorithms: Normal / KL-Divergence / MMSE:

- Normal: Determines the quantization range directly from the maximum and minimum floating-point values. This algorithm produces no clamping error but is sensitive to outliers. It runs fast and suits general scenarios.
- KL-Divergence: Determines the quantization range by comparing the distributions of floating-point and fixed-point values, adjusting thresholds, and minimizing the KL divergence. This algorithm adapts better to non-uniform data distributions, mitigates the impact of a few outliers, and runs slightly slower than Normal.
- MMSE: Determines the quantization range by minimizing the mean squared error between floating-point values and dequantized floating-point values. This algorithm usually achieves higher accuracy but requires more computing resources, runs slower, and has higher memory overhead. Compared with KL-Divergence, it often delivers better accuracy in outlier scenarios.

Users are advised to use the Normal algorithm first. When quantization accuracy issues arise, try KL-Divergence or MMSE.

2.3.3.3 CALIBRATION DATA PREPARATION

In the internal quantization flow, the calibration dataset is used to compute the quantization ranges of activations. Choose data that covers the different data distributions of the model's actual application scenarios; for example, the calibration data of a classification model should include images of different categories from real scenes. A calibration dataset of 20~200 images is generally recommended; adjust this as appropriate based on the running time of the quantization algorithm. Note that under a16 conditions, Normal quantization requires no calibration dataset. For the other quantization algorithms, increasing the number of calibration samples increases running time but does not necessarily improve accuracy.

> **Note**: The `dataset.txt` calibration dataset is used only for general model conversion with quantization; it is not required for LLM model conversion.

Internal quantization calibration data currently supports the text file format. Users can write the paths of the calibration images (jpg or png format) or npy files into a `.txt` file. Each line in the

text file is one path entry, for example:

```
a.jpg
b.jpg
```

For multiple inputs, separate the files of each input with spaces, for example:

```
a0.jpg a1.jpg
b0.jpg b1.jpg
```

2.3.3.4 SPECIFIC EXAMPLES

Example 1: ResNet50 internal quantization flow example:

```
from rknn.api import RKNN

model_path = "resnet50-v2-7.onnx" ## Floating-point ONNX model exported in advance
output_path = 'resnet50-v2-7.rknn'
DATASET_PATH =
'../../../datasets/imagenet/ILSVRC2012_img_val_samples/dataset_20.txt'

# Create RKNN object
rknn = RKNN(verbose=False)

# Pre-process config
print('--> Config model')
rknn.config(mean_values=[[255*0.485, 255*0.456, 255*0.406]],
            std_values=[[255*0.229, 255*0.224, 255*0.225]],
            target_platform="rk1828",
            input_attrs={'input': {'dtype': 'uint8', 'layout': 'NHWC'}},
            quantized_dtype='w8a8',           ## optional: "w8a16", "w4a16", etc.
            quantized_algorithm='normal',     ## optional: "MMSE", "KL-Divergence"
            quantized_method='channel',)
print('done')

# Load model
print('--> Loading model')
ret = rknn.load_onnx(model=model_path, inputs=['data'], input_size_list=[[1, 3, 224,
224]])

if ret != 0:
    print('Load model failed!')
    exit(ret)
print('done')

# Build model
print('--> Building model')
ret = rknn.build(do_quantization=True, dataset=DATASET_PATH)
if ret != 0:
    print('Build model failed!')
    exit(ret)
print('done')

# Export rknn model
print('--> Export rknn model')
ret = rknn.export_rknn(output_path)
if ret != 0:
    print('Export rknn model failed!')
    exit(ret)
print('done')
```

2.3.4 RKQuantizer Quantization Flow

2.3.4.1 DATA TYPES

RKQuantizer supports the quantization data types `w4a16`, `w6a16`, and `w8a16`. The weight quantization method (`quantized_method`) supports only group32.

2.3.4.2 RKQUANTIZER QUANTIZATION ALGORITHMS

RKQuantizer quantization supports two algorithms: GRQ and Normal. GRQ is based on backpropagation. It minimizes the mean squared error between the outputs of each block in the network (such as a DecodeLayer in an LLM) before and after quantization, and solves the quantization parameters of each linear layer within the block. Unlike algorithms such as KL-Divergence and MMSE that minimize tensor quantization errors, GRQ aims to make the outputs of the quantized model closer to those of the floating-point model, so it usually achieves higher quantization accuracy and is the preferred algorithm. Because GRQ uses backpropagation, run it in a GPU / CUDA environment.

2.3.4.3 CALIBRATION DATA PREPARATION

The Normal quantization algorithm requires no calibration dataset. The GRQ algorithm requires a calibration dataset to collect the activation distributions of each layer and fine-tune the weights. The calibration dataset is provided as a **JSON file** whose content is a list, with each element being one sample. The quantization component RKQuantizer reads the samples one by one and constructs the model inputs. Two sample formats are currently supported; users can choose based on the model type.

Method 1: Hook-captured inputs (general, recommended)

Capture the real inputs of the submodule to be quantized during forward passes of the floating-point model with PyTorch `register_forward_pre_hook`, save them with pickle serialization, and build the JSON index file. This applies to all models, including MLLM, speech, and other models with non-pure-text inputs.

Each sample in the JSON file is a dictionary containing only one `sample` field, whose value is the relative path of a pickle file (relative to the directory of the JSON file). The pickle file contains a dictionary with two keys, `args` (positional arguments) and `kwargs` (keyword arguments), corresponding to the positional and keyword inputs for the quantized model's forward pass. The format is as follows:

```
[
  {"sample": "model_inputs/sample_0"},
  {"sample": "model_inputs/sample_1"}
]
```

The standard way to obtain this data is: on the floating-point model to be quantized, register `register_forward_pre_hook` on the module to be quantized (enable `with_kwargs=True` to also capture keyword arguments); in the hook, store the inputs into a list and raise an exception to terminate the forward pass early (avoiding a full inference), then dump the captured inputs to disk with pickle. rknn3-model-zoo wraps this mechanism as the public utility `py_utils/cali_data.py`, providing the `capture_module_input(hook_module, run, *args, **kwargs)` function: it packs "register forward pre hook → invoke the model → capture `(args, kwargs)` → raise `StopForward` to short-circuit the forward → remove the hook" into one line, so callers need no awareness of hook details. The internal hook raises `StopForward` to terminate the forward of the captured submodule. Therefore, even when `model.generate` is passed in, only one forward pass runs before the interruption, and no tokens are actually generated. For example:

```python
import os
import json
import pickle
import torch
from transformers import AutoProcessor
# Import the common hook capture utility (adjust the path to your project structure)
from py_utils.cali_data import capture_module_input

model_path = 'Qwen/Qwen2.5-VL-3B-Instruct'
model = ...                       # Load the floating-point model; keep torch_dtype
consistent with quantization
processor = AutoProcessor.from_pretrained(model_path)

# Forward per sample; capture the inputs of the submodule to be quantized in one
call
info = []
datasets = json.load(open('./dataset.json', 'r'))   # Raw annotated data (images,
questions, etc.)
for idx, data in enumerate(datasets):
    inputs = processor(...)                # Construct the complete input as
required by the model
    inputs = inputs.to(model.device)
    with torch.inference_mode():
        # Pass the quantized model itself to hook_module (i.e., the submodule passed
to load_model; pass model.visual to quantize the vision part),
        # The captured inputs are those at the outermost entry of this submodule,
not the inputs of its first internal transformer block.
        # Pass the model call to trigger via run (wrapped in a lambda; may include
preceding steps such as prepare_inputs_embeds)
        temp = capture_module_input(
            model.visual,
            lambda: model(**inputs)
        )
        # temp is {"args": (...), "kwargs": {...}}; it is None if this input does
not flow through the submodule

    # Serialize the captured inputs with pickle and record them in the index JSON
    sample_name = 'sample_{}'.format(idx)
    os.makedirs('./model_inputs', exist_ok=True)
    with open(os.path.join('./model_inputs', sample_name), 'wb') as f:
        pickle.dump(temp, f)
    info.append({"sample": "model_inputs/{}".format(sample_name)})

with open('./cali_data.json', 'w', encoding='utf-8') as f:
    json.dump(info, f, indent=2, ensure_ascii=False)
```

Method 2: Pure-text JSON (for pure-text LLMs only)

For LLM models whose inputs are text only, a simpler method is available: each sample in the JSON file is a dictionary containing two keys, `input` and `target`, and RKQuantizer automatically encodes it into model inputs with `tokenizer(input + target)`. This method requires neither loading the floating-point model nor hooks, and has the lowest preparation cost. The format is as follows:

```
[
  {
    "input": "Cysteine and tyrosine can be derived from what in the human body,
respectively.\nA. Cystine and methionine\nB. Methionine and phenylalanine",
    "target": "\nAnswer:\n B"
  },
  {
    "input": "Natalia sold clips to 48 of her friends in April, and then she sold
half as many in May. How many altogether?",
    "target": ""
  }
]
```

Here, `input` is the prompt text and `target` is the expected output text (it can be an empty string). The two are concatenated and encoded as a whole by the tokenizer. `target` can include supplementary context such as answers or chains of thought, making the calibration distribution closer to real generation scenarios. For example:

```
import json

# Write samples directly or organize them from an existing dataset, then save as
JSON
samples = [
    {"input": "Briefly describe Newton's first law.", "target": "\nAnswer:\n
Newton's first law states..."},
    {"input": "What is 1+1?", "target": "\nAnswer:\n 2"},
]
with open('./cali_data.json', 'w', encoding='utf-8') as f:
    json.dump(samples, f, ensure_ascii=False, indent=2)
```

> **Note**: This method applies only to LLMs with pure-text inputs. For multimodal (MLLM), speech, and other models with non-text inputs, Method 1 must be used.

The quality of the calibration data greatly affects the accuracy of the GRQ-quantized model. Follow the core principle when selecting data: **use diverse data from real application scenarios**, so that the calibration data distribution stays as close as possible to the actual input distribution after the model is deployed. Specific recommendations:

- **Cover real scenarios**: Calibration samples should come from real inputs in the model's actual deployment scenarios, such as actual system prompts and function-tools. For agent tasks, such inputs are especially important and can be loaded through the `load_model` interface.
- **Maintain diversity**: Samples should be diverse and avoid highly homogeneous content, so that the quantization parameters adapt to different input distributions. Preparing 20~200 samples is generally recommended.

2.3.4.4 MIXED-PRECISION QUANTIZATION SETTINGS

RKQuantizer provides two mixed-precision quantization methods: manual per-layer specification (`layer_quant_config`) and automatic proportional selection (`auto_hybrid_rate`).

Method 1: Manual per-layer specification (layer_quant_config)

The user explicitly specifies which layers use w6a16 or w8a16. `layer_quant_config` accepts a dictionary or a JSON file path in the format `{layer_pattern: "w8a16"}`, where `layer_pattern` is a shell-style wildcard pattern for matching Linear layer names and supports the `*` wildcard (for matching same-named sublayers across all transformer blocks). Matched layers use the quantization type specified in the configuration; unmatched layers are quantized with `quantized_dtype`. Currently `layer_quant_config` supports only "w8a16" or "w6a16", and it must not be used together with `auto_hybrid_rate`.

Taking the Qwen3.5 model as an example, its `layer_quant_config.json` is as follows, specifying the linear-attention-related qkv/b/a/out_proj layers and the v_proj and down_proj layers as w8a16 (or w6a16):

```
{
  "model.layers.*.linear_attn.in_proj_qkv": "w8a16",
  "model.layers.*.linear_attn.in_proj_b": "w8a16",
  "model.layers.*.linear_attn.in_proj_a": "w8a16",
  "model.layers.*.linear_attn.out_proj": "w8a16",
  "model.layers.*.self_attn.v_proj": "w8a16",
  "model.layers.*.mlp.down_proj": "w8a16"
}
```

The corresponding quantization invocation is as follows:

```
from rknn.quantization.api import RKQuantizer

QuantTool = RKQuantizer(verbose=True)

QuantTool.load_model(...)

quant_model = QuantTool.quantize(quantized_dtype="w4a16",
                                 quantized_method="group32",
                                 quantized_algorithm='grq',
                                 dataset='./dataset.json',
                                 awq_config=None,
                                 layer_quant_config="layer_quant_config.json",
                                 auto_hybrid_rate=0.)
```

Method 2: Automatic proportional selection (auto_hybrid_rate)

No per-layer specification is needed. RKQuantizer automatically evaluates each layer's sensitivity to quantization from the calibration data and keeps the most sensitive Linear layers at w8a16 by the specified ratio. `auto_hybrid_rate` is a float in [0, 1) indicating the proportion of Linear layers set to w8a16. For example, 0.2 means about 20% of Linear layers are set to w8a16, and the rest are quantized with `quantized_dtype`. This parameter must not be used together with `layer_quant_config`. For example:

```python
from rknn.quantization.api import RKQuantizer

QuantTool = RKQuantizer(verbose=True)

QuantTool.load_model(...)

## auto_hybrid_rate=0.2 means 20% of Linear layers are set to w8a16
model = QuantTool.quantize(quantized_dtype="w4a16",
                           quantized_method="group32",
                           quantized_algorithm="grq",
                           dataset='./dataset.json',
                           auto_hybrid_rate=0.2,
                           awq_config=None)
```

Exporting per-layer quantization types (export_op_quantized_dtype)

With either mixed-precision method above, invoke `export_op_quantized_dtype` after quantization to export the quantization type actually used by each layer into a JSON file, which is used for alignment in the subsequent `rknn.build()` stage. This interface takes the path of the exported ONNX model to build the mapping between torch layer names and ONNX operators. The exported JSON is in `{onnx_op_name: dtype}` format, where the key is the name of an operator such as MatMul/Gemm in the ONNX model and the value is that operator's quantization type. Taking Qwen3.5 as an example, the exported `layer_bit.json` is as follows:

```
{
  "onnx::MatMul_8968": "w8a16",
  "onnx::MatMul_8973": "w8a16",
  "onnx::MatMul_9657": "w8a16",
  ...
}
```

For example:

```python
# Invoke after exporting the ONNX model, passing the ONNX model path
QuantTool.export_op_quantized_dtype(onnx_path='./Qwen3.5-0.8B-llm.onnx',
                                    op_dtype_path='layer_bit.json')
```

Corresponding changes when exporting the RKNN model

Because mixed-precision quantization is already done in the RKQuantizer stage (the ONNX weights are fake-quantized per layer type), when exporting the RKNN model via `rknn.config()` + `rknn.load_llm()` + `rknn.build()`, pass `layer_bit.json` through the `op_quantized_dtype` parameter of `rknn.config()`. This makes the RKNN conversion stage restore each layer at its specified quantization type, avoiding accuracy regression caused by re-quantizing sensitive layers to low bit widths during conversion. `op_quantized_dtype` accepts a dictionary or a JSON file path. It currently takes effect only for MatMul / Gemm / Linear operators, with supported values w6a16 or w8a16.

```
rknn.config(
    target_platform='rk1820',
    quantized_dtype='w4a16',
    quantized_algorithm='normal',
    quantized_method='group32',
    llm_config=llm_config,
    op_quantized_dtype='./layer_bit.json',   # Pass the per-layer quantization types
for mixed precision
)
```

> **Note**:
>
> 1. `layer_quant_config` and `auto_hybrid_rate` must not be used together.
> 2. `export_op_quantized_dtype` must be invoked after the ONNX export is complete, and the passed `onnx_path` must be the same ONNX that is later loaded by `rknn.load_llm()`. Otherwise, the mapping between torch layer names and ONNX operators fails.
> 3. When exporting RKNN, the JSON specified by `op_quantized_dtype` must correspond to the file exported by `export_op_quantized_dtype` in the quantization stage. Missing either one causes the mixed-precision configuration to not take effect or a conversion error.
> 4. If mixed-precision quantization is not enabled, there is no need to invoke `export_op_quantized_dtype` or configure `op_quantized_dtype` in `rknn.config()`.

2.3.4.5 SPECIFIC EXAMPLES

Example 1: qwen3 RKQuantizer quantization flow example. Fake-quantized ONNX model export:

```
from transformers import AutoModelForCausalLM, AutoTokenizer, AutoConfig
from rknn.quantization.api import RKQuantizer
import torch

model_path = "Qwen3-1.7B"

## Load the model
kwargs = {
    'trust_remote_code': True,
    'torch_dtype': torch.bfloat16
}
config = AutoConfig.from_pretrained(model_path, **kwargs)
update_config(config, ['use_cache'], False)
kwargs['config'] = config
model = AutoModelForCausalLM.from_pretrained(model_path, **kwargs)
tokenizer = AutoTokenizer.from_pretrained(model_path, trust_remote_code=True)

## Initialize the quantization tool
QuantTool = RKQuantizer(verbose=True)

## The quantization tool loads the model
ret = QuantTool.load_model(model=model, tokenizer=tokenizer, device='cuda',
                           system_prompt=None, tools=None, system_role='system',
user_role='user')
if ret != 0:
    print('Load model failed!')
    exit(ret)

## Run the quantization algorithm
dataset = args.cali_dataset
model = QuantTool.quantize(quantized_dtype="w4a16", quantized_method="group32",
quantized_algorithm='grq', dataset=dataset)
## Note: the weights in the model returned by QuantTool.quantize have been replaced
with fake-quantized weights
model = model.cpu()

'''
  Then export the ONNX model to obtain the fake-quantized ONNX model
'''
```

Converting the fake-quantized ONNX model to rknn:

```python
import numpy as np
from rknn.api import RKNN
from rknn.api import RKNN,DEFAULT_RKNN_LLM_CONFIG

ONNX_MODEL = 'Qwen3-1.7B.onnx'
LLM_CONFIG = 'Qwen3-1.7B.config.pkl'
RKNN_MODEL = 'Qwen3-1.7B.rknn'

# Create RKNN object
rknn = RKNN(verbose=True)

llm_config = DEFAULT_RKNN_LLM_CONFIG.copy()
llm_config['attention_config'][0]['kvcache_buffer_len'] = 4*1024
llm_config['attention_config'][0]['max_position_embeddings'] = 4*1024
# llm_config['attention_config'][0]['kvcache_dtype'] = 'Int4_to_F16'

# pre-process config
print('--> config model')
## Note: quantized_algorithm must be set to 'normal' here; quantized_dtype and
quantized_method must match those in QuantTool.quantize exactly.
rknn.config(target_platform="rk1820",
            quantized_dtype='w4a16', quantized_algorithm='normal',
quantized_method='group32', llm_config=llm_config
            )
print('done')

# Load model
print('--> Loading model')
ret = rknn.load_llm(model=ONNX_MODEL, config=LLM_CONFIG)
if ret != 0:
    print('Load model failed!')
    exit(ret)
print('done')

# Build model
print('--> Building model')
ret = rknn.build(do_quantization=True)
if ret != 0:
    print('Build model failed!')
    exit(ret)
print('done')

#Export rknn model
print('--> Export RKNN model')
ret = rknn.export_rknn(RKNN_MODEL)
if ret != 0:
    print('Export rknn failed!')
    exit(ret)
print('done')

rknn.release()
```

2.3.5 Other Quantized Model Conversion

For LLM / MLLM models, when users obtain a quantized model that meets scenario requirements with other quantization schemes (such as GPTQ / QAT), they can first save it as a Huggingface-format fake-quantized model and then export ONNX directly, without using RKQuantizer again. Note that with other quantization schemes, the supported quantization types are still `w4a16`,

`w6a16`, and `w8a16`, and `group_size=32` is required. Likewise, in the subsequent ONNX-to-RKNN conversion, `rknn.config()` must set `quantized_algorithm='normal'` and correctly set `quantized_dtype` (`w4a16`, `w6a16`, or `w8a16`) and `quantized_method='group32'`.

Other quantized model conversion flow:

```
Floating-point PyTorch model ⟶ AutoGPTQ/AutoAWQ/LLM-Compressor ⟶ fake-quant
PyTorch model ⟶ export fake-quant ONNX ⟶ rknn.config() ⟶
rknn.load_llm()/rknn.load_onnx() ⟶ rknn.build() ⟶ rknn.export_rknn()
```

2.4 Model Evaluation

After model conversion, a series of evaluations is required to verify the model's validity and performance on the target platform. This chapter describes evaluation methods based on the RKNN3-Toolkit Python API, covering the following four core dimensions:

- **Inference verification**: Verifies that the model's inference works correctly on the target device and that the output results match expectations.
- **Accuracy evaluation**: Compares the model's outputs between the RKNN backend and the original framework to quantify the accuracy loss.
- **Performance evaluation**: Measures key performance metrics such as inference latency and throughput on the NPU.
- **Memory evaluation**: Profiles memory usage during model runtime to support resource planning.

These evaluation tasks support the following two execution modes, and developers can choose flexibly based on the stage:

Execution Mode	Applicable Scenarios	Supported Capabilities
Simulator	No hardware connection required; suits quick verification in early development or CI pipelines	Inference verification, accuracy evaluation
Connected board	Connects real hardware; suits final acceptance and performance tuning before release	Inference verification, accuracy evaluation, performance evaluation, memory evaluation

> **Note**:
>
> 1. Performance evaluation and memory evaluation depend on a real NPU environment and are available only in connected-board mode.
> 2. Connected-board accuracy analysis and performance evaluation require building the model with `profile_mode=True` configured in `rknn.config()`; otherwise the corresponding data cannot be obtained.

2.4.1 Model Inference Verification

After model conversion, invoke `rknn.inference()` for inference verification to check the conversion result and functional correctness. Inference verification supports both simulator and connected-board modes. The two differ only in the parameter configuration of `init_runtime()`; the inference interface is the same:

- **Simulator inference**: Does not specify `target`. It simulates NPU execution on the PC and requires no connected development board.
- **Connected-board inference**: Specifies `target` (target platform), `device_id` (device ID), and `core_mask` (NPU core mask), and executes on a real NPU.

It is recommended to run simulator inference verification first (no hardware needed; quickly confirms the conversion result), and then connected-board inference verification (obtains actual inference results on a real NPU):

```
# 1. Simulator inference (no target specified; no development board required)
rknn.init_runtime()
outputs = rknn.inference(inputs=[img])

# 2. Connected-board inference (specify target and device_id; runs on the real NPU)
rknn.init_runtime(target=platform, device_id='515e9b401c060c0b', core_mask=0xff)
outputs = rknn.inference(inputs=[img])
```

> **Note**: Simulator inference requires no hardware and suits quick verification of conversion results in early development; connected-board inference executes on a real NPU and obtains actual inference results and performance. Simulator performance differs from actual on-device performance; performance evaluation must be done on a connected board (see 2.4.3).

2.4.2 Accuracy Evaluation

Accuracy evaluation is enabled with `rknn.inference(accuracy_analysis=True)`. It compares the differences between the inference results of the quantized model and the floating-point model, and analyzes the source of quantization errors. Like inference verification, accuracy analysis has two modes: simulator and connected-board:

Error Type	Meaning	Sub-items
`simulator_error`	Simulator error (compared against golden)	`entire` (accumulated error), `single` (single-layer independent error)
`runtime_error`	On-device Runtime error	`entire`, `single_sim` (single-layer comparison between Runtime and simulator)

Both cosine distance and Euclidean distance are reported; abnormally low cosine or high Euclidean values are highlighted in yellow or red.

Simulator accuracy evaluation:

```
# Simulator accuracy analysis (no target specified)
rknn.init_runtime()
outputs = rknn.inference(inputs=[img], accuracy_analysis=True)
```

The analysis result is as follows:

```
I GraphPreparing : 100%|████████████████████████████████████| 187/187
[00:00<00:00, 6481.46it/s]
I AccuracyAnalysing : 100%|████████████████████████████████████| 187/187
[00:05<00:00, 34.62it/s]

# simulator_error: calculate the output error of each layer of the simulator
(compared to the 'golden' value).
#              entire: output error of each layer between 'golden' and 'simulator',
these errors will accumulate layer by layer.
#              single: single-layer output error between 'golden' and 'simulator',
can better reflect the single-layer accuracy of the simulator.

layer_name                                                    simulator_error
                                                           entire              single
                                                        cos      euc        cos
euc
-------------------------------------------------------------------------------------
-------
[Input] images                                        1.00000 | 0.09873   1.00000 |
0.09873
[Conv] /model.0/conv/Conv_output_0                    0.99998 | 4.97621   0.99998 |
4.97621
[exSwish] /model.0/act/Mul_output_0                   1.00000 | 3.92585   1.00000 |
1.27413
[Conv] /model.1/conv/Conv_output_0                    1.00000 | 5.41606   1.00000 |
2.07959
...
```

Connected-board accuracy evaluation:

```
# Connected-board accuracy analysis (specify target / device_id / core_mask)
rknn.init_runtime(target=platform, device_id='515e9b401c060c0b', core_mask=0xff)
outputs = rknn.inference(inputs=[img], accuracy_analysis=True)
```

The analysis result is as follows:

```
I GraphPreparing : 100%|████████████████████████████████████| 187/187
[00:00<00:00, 5128.55it/s]
I AccuracyAnalysing : 100%|████████████████████████████████████| 187/187
[00:25<00:00,  7.33it/s]

# simulator_error: calculate the output error of each layer of the simulator
(compared to the 'golden' value).
#              entire: output error of each layer between 'golden' and 'simulator',
these errors will accumulate layer by layer.
#              single: single-layer output error between 'golden' and 'simulator',
can better reflect the single-layer accuracy of the simulator.
# runtime_error: calculate the output error of each layer of the runtime.
#              entire: output error of each layer between 'golden' and 'runtime',
these errors will accumulate layer by layer.
#              single_sim: single-layer output error between 'simulator' and
'runtime', can better reflect the single-layer accuracy of runtime.

layer_name                                                    simulator_error
runtime_error
                                                          entire              single
entire           single_sim
                                                        cos      euc        cos
euc        cos      euc        cos      euc
-------------------------------------------------------------------------------------
-----------------------------------------
[Input] images                                      1.00000 | 0.09873   1.00000 |
0.09873   1.00000 | 504.714   1.00000 | 0.70923
[Conv] /model.0/conv/Conv_output_0                  0.99998 | 4.97621   0.99998 |
4.97621   1.00000 | 5.45544   1.00000 | 1.46544
[exSwish] /model.0/act/Mul_output_0                 1.00000 | 3.92585   1.00000 |
1.27413   0.99998 | 7.01712   0.99998 | 5.80349
...
```

2.4.3 Performance Evaluation

Invoke `rknn.eval_perf()` to evaluate the model's performance on the NPU. It outputs the latency, cycles, and bandwidth of each operator.

```
rknn.init_runtime(target=platform, device_id='515e9b401c060c0b', core_mask=0xff)
rknn.eval_perf()
```

After execution, the performance evaluation result is output as follows:

```
-------------------------------------------------------------------------------
--
Op Time Ranking Table (Core 0)
-------------------------------------------------------------------------------
--
OpType             Calls   CPU(us)      NPU(us)      Total(us)    Ratio(%)
-------------------------------------------------------------------------------
--
ConvExSwish        56      0            20385        20385          47.49%
Reshape            3       0            9245         9245           21.54%
InputProc          1       0            4293         4293           10.00%
Split              6       0            2174         2174            5.06%
Concat             21      0            1878         1878            4.37%
ConvSigmoid        3       0            905          905             2.11%
exYoloPostProcess  1       784          0            784             1.83%
Mul                10      0            572          572             1.33%
Add                13      0            562          562             1.31%
MaxPool            3       0            465          465             1.08%
ConvAdd            1       0            426          426             0.99%
exSwish            1       0            416          416             0.97%
Resize             2       0            412          412             0.96%
Conv               1       0            252          252             0.59%
Sub                3       0            80           80              0.19%
Pow                3       0            75           75              0.17%
InputOperator      1       0            3            3               0.01%
OutputProc         1       0            0            0               0.00%
OutputOperator     1       0            0            0               0.00%
-------------------------------------------------------------------------------
--
Total                      784          42143        42927         100.00%
-------------------------------------------------------------------------------
--
```

The performance evaluation result outputs an Op Time Ranking Table of all operators. The fields are described as follows:

Field	Description
OpType	Operator type
Calls	Number of times the operator runs within a single frame
CPU(us)	Total time of the operator on the CPU within a single frame
NPU(us)	Total time of the operator on the NPU within a single frame
Total(us)	Total time of the operator within a single frame
Ratio(%)	Ratio of the operator's total time to the total time of a single frame

2.4.4 Memory Evaluation

Invoke `rknn.eval_memory()` to obtain the model's memory consumption evaluation, including Device Memory and Per-Core Memory Allocation:

```
rknn.init_runtime(target=platform, device_id='515e9b401c060c0b', core_mask=0xff)
memory_detail = rknn.eval_memory()
```

After execution, the memory evaluation result is output as follows:

```
========================== Memory Usage Information ==========================
Device Memory:
  System  :      18.98 MB total,      18.83 MB free,       0.16 MB used (  0.8%)
  Node 0  :     639.50 MB total,     590.57 MB free,      48.93 MB used (  7.7%)
  Node 1  :     639.50 MB total,     639.42 MB free,       0.08 MB used (  0.0%)
  Node 2  :     619.90 MB total,     619.81 MB free,       0.08 MB used (  0.0%)
  Node 3  :     639.50 MB total,     639.42 MB free,       0.08 MB used (  0.0%)
  Node 4  :     619.90 MB total,     619.81 MB free,       0.08 MB used (  0.0%)
  Node 5  :     619.90 MB total,     619.81 MB free,       0.08 MB used (  0.0%)
  Node 6  :     639.50 MB total,     639.42 MB free,       0.08 MB used (  0.0%)
  Node 7  :     639.50 MB total,     639.42 MB free,       0.08 MB used (  0.0%)

Per-Core Memory Allocation (MB):
  Core     Command      Weight       Internal     KVCache      Total
  -------- ------------ ------------ ------------ ------------ ------------
  0        0.22         15.42        33.20        0.00         48.84
  -------- ------------ ------------ ------------ ------------ ------------
  Total    0.22         15.42        33.20        0.00         48.84
================================================================================
```

The fields of the per-core memory allocation are described as follows:

Field	Description
Core	Core in use
Command	Memory usage of register configuration and related control information (MB)
Weight	Memory usage of model weight tensors (MB)
Internal	Memory usage of model intermediate tensors (MB)
KVCache	Memory usage of model KVCache (MB)
Total	Total memory usage of the model (MB)

3. On-device Deployment

3.1 Runtime Deployment Basics

This section describes the fundamental concepts of RKNN3 Runtime and is a prerequisite for on-device deployment (using the C API or Toolkit Lite). It covers model and context lifecycles, tensors and data layout formats, input and output memory management, synchronous and asynchronous inference mechanisms, and error handling and resource release conventions. These mechanisms apply to both general models and LLM / MLLM models.

3.1.1 Model and Context Lifecycle

`rknn3_context` is the core handle of RKNN3 Runtime and represents a runtime context instance. All model loading, inference configuration, and resource management operations are based on this handle. For on-device deployment, strictly follow the "create → use → destroy" lifecycle management convention to avoid resource leaks or runtime exceptions.

General models (CNN / ViT, etc.)

These models do not involve the session layer. Invoke `rknn3_run()` directly through the context handle to execute inference. After the context is initialized, `rknn3_run()` can be invoked multiple times, and the handle remains unchanged until the context is destroyed.

LLM / MLLM models

On top of general models, these models introduce a **Session** layer to manage the context state of text generation tasks. Create a session with `rknn3_session_init()` and configure LLM-related parameters (such as vocabulary, sampling strategy, and context window length), then invoke `rknn3_session_run()` to execute inference. One context can create multiple sessions. All sessions share model weights and internal memory, but their KVCache and LoRA enable states are independent of each other. Sessions cannot execute concurrently; only one session can run at a time. After the sessions are used, invoke `rknn3_session_destroy()` to destroy each session first, and finally invoke `rknn3_destroy()` to destroy the context.

> **Multi-card note**: A multi-card pipeline creates an independent context and session for each RK182X card. Stage sessions on different cards can run concurrently under host-side control, but each context/session still follows the constraint that only one session runs at a time. Stage synchronization, tensor forwarding, and sampling control in multi-card deployment are the application's responsibility; the Runtime does not automatically combine multiple contexts into one multi-card model.

3.1.2 Tensors and Data Layout Formats

The inputs and outputs of RKNN3 models are managed through **tensors**. This section describes the tensor structure hierarchy, data types and layouts, and the NPU-specific `NC1HWC2` data layout format.

Tensor Structure

General models and LLM / MLLM models use different tensor structure hierarchies:

General models (CNN / ViT, etc.) use `rknn3_tensor` as the input and output tensors for inference. It binds two sub-structures:

- **`rknn3_tensor`**: The tensor structure used for inference. It contains two core members: `mem` (pointing to `rknn3_tensor_mem`) and `attr` (pointing to `rknn3_tensor_attr`). Before inference, users must allocate memory for each input and output tensor and bind the attributes. The Runtime reads inputs and writes outputs through this structure.
- **`rknn3_tensor_attr`**: The tensor attribute structure, queried through `rknn3_query()`. It contains the tensor name, shape, data type (dtype), data layout (layout), quantization information (qnt_type/qnt_info), core ID, and more. For on-device deployment, query the attributes first, then allocate memory and organize data accordingly.
- **`rknn3_tensor_mem`**: The tensor data memory structure, allocated through interfaces such as `rknn3_create_mem()`. It represents the actual data storage of the tensor on the device. Users must synchronize data between the CPU and the device through `rknn3_mem_sync()`.

LLM / MLLM models use `rknn3_llm_input` as the input for Session inference. It distinguishes different input types through the `input_type` field and uses the corresponding sub-structure:

- **`rknn3_llm_input`**: The LLM inference input structure, containing `input_type` (input type) and the input data of the corresponding type. Main members:
- `role`: The message role, such as `"user"` (user input) or `"tool"` (function call result).
- `input_type`: The input type. Options include `RKNN3_LLM_INPUT_PROMPT` (text), `RKNN3_LLM_INPUT_TOKEN` (token id), `RKNN3_LLM_INPUT_EMBED` (embedding vector), `RKNN3_LLM_INPUT_MULTIMODAL` (multimodal), and `RKNN3_LLM_INPUT_AUX` (auxiliary input).
- `llm_input`: A `rknn3_llm_tensor` structure when the input type is PROMPT/TOKEN/EMBED.
- `multimodal_input`: A `rknn3_llm_multimodal_tensor` structure when the input type is MULTIMODAL (members described below).
- `aux_input`: An auxiliary input tensor when the input type is AUX (currently `rknn3_aux_tensor` is the same type as `rknn3_tensor`).
- **`rknn3_llm_tensor`**: A sub-structure of `rknn3_llm_input` that represents the LLM input content. Main members:
- `name`: The tensor name.
- `prompt`: The text prompt input, used when the input type is `RKNN3_LLM_INPUT_PROMPT`.
- `embed`: The embedding vector pointer, used when the input type is `RKNN3_LLM_INPUT_EMBED` (size is `n_tokens * hidden_size`).
- `tokens`: The token id array, used when the input type is `RKNN3_LLM_INPUT_TOKEN`.
- `n_tokens`: The number of tokens.
- `enable_thinking`: Whether to enable thinking mode.
- **`rknn3_llm_multimodal_tensor`**: A sub-structure of `rknn3_llm_input`, used when the input type is `RKNN3_LLM_INPUT_MULTIMODAL`. It represents multimodal input (text, image, audio, video). Top-level members include `name`, `prompt` / `tokens` (text input; provide one of the two — if both are provided, prompt is used for inference by default), `n_tokens`, `enable_thinking`, and three modality sub-structures:

- **`image` sub-structure**: Image input. `image_embed` (FP16 embedding pointer, size is `n_image × n_image_tokens × embedding_dim`), `n_image_tokens` (number of tokens per image), `n_image` (number of images), `image_start` / `image_end` / `image_content` (start/end/placeholder tags of the image in the text), `image_width` / `image_height` (image width and height).
- **`audio` sub-structure**: Audio input. `audio_embed` (FP16 embedding pointer, size is `n_audio × n_audio_tokens × embedding_dim`), `n_audio_tokens` (number of tokens per audio segment), `n_audio` (number of audio segments), `audio_start` / `audio_end` / `audio_content` (audio tags).
- **`video` sub-structure**: Video input. `video_embed` (FP16 embedding pointer, size is `n_video × n_frame_per_video × n_frame_tokens × embedding_dim`), `n_frame_tokens` (number of tokens per frame), `n_frame_per_video` (number of frames per video), `n_video` (number of videos), `video_start` / `video_end` / `video_content` (video tags).

The `*_embed` of each modality is the inference output of the corresponding vision/audio/video encoder model and is passed into the LLM through this structure. The `*_start` / `*_end` / `*_content` tags must match the special tokens of the model tokenizer. For details on how to populate and use the structure, see 3.3.6 Multimodal Input.

Data Types and Layouts

The data type (dtype) and data layout (layout) of a tensor can be queried through the `dtype` and `layout` fields of `rknn3_tensor_attr`.

- **Data type (dtype)**: Includes FP32 / FP16 / INT8 / UINT8, etc., determined by the quantization configuration during model conversion. For non-quantized models, both weights and activations are FP16. For quantized models, weights can be INT8, etc.
- **Data layout (layout)**: There are three main layouts: `NCHW`, `NHWC`, and `NC1HWC2`. `NCHW` and `NHWC` are common data layouts in deep learning. User inputs typically use these two formats, and the Runtime handles the conversion internally. `NC1HWC2` is a memory layout format optimized for NPU hardware. To directly access NPU memory (for example, in zero-copy scenarios), organize the data in the `NC1HWC2` layout.

NC1HWC2 Data Layout Format

`NC1HWC2` stores the channel dimension in blocks, where C1 is the number of channel blocks (ceil(C/C2)) and C2 is the number of sub-channels, determined by the platform and data type:

	RK1820/RK3572
INT8	32
FP16	16

The data layout and storage of `NC1HWC2` are shown in the figure below. The number 0 represents one data element; C2 data elements are stored at a time, and the storage order follows the increasing order of the values in the figure (0-15 are stored first, then 16-31). The figure below uses C2 = 8 as an example (to illustrate the layout principle; on current platforms C2 = 16 for FP16 and C2 = 32 for INT8, with the same layout). When the feature is (1, 13, 4, 4), the corresponding `NC1HWC2` is (1, 2, 4, 4, 8). Only the first 5 data elements in each C2 block are valid; the rest are alignment padding.

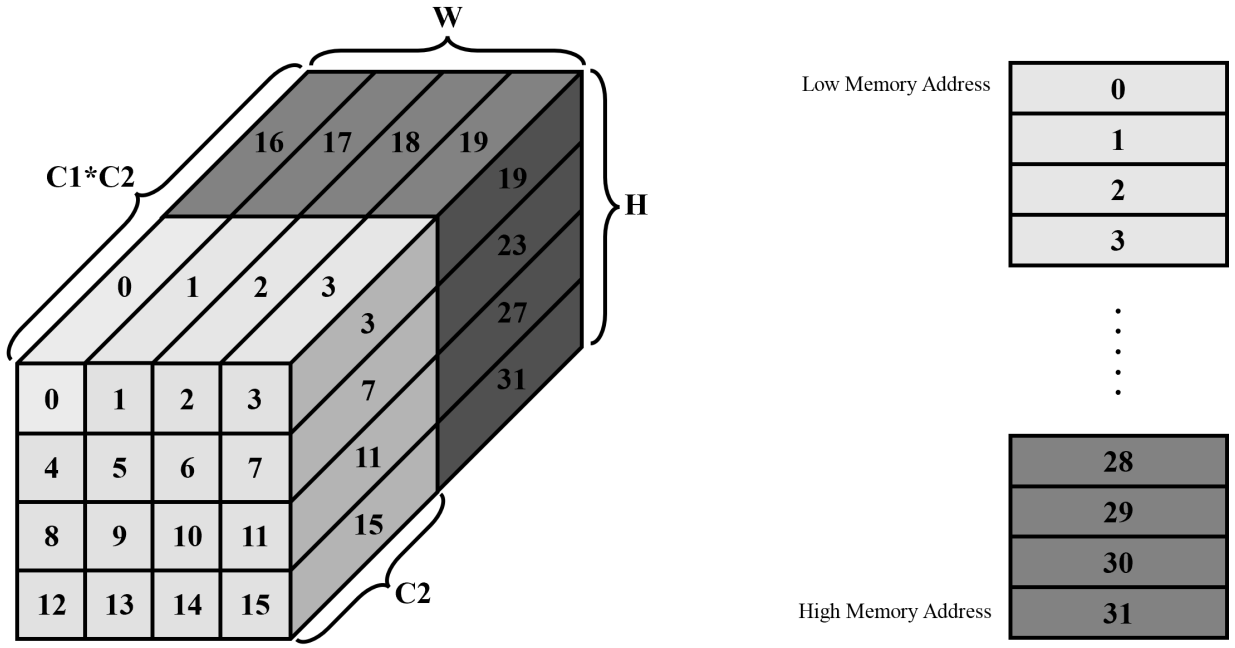


Figure 3-1 RKNPU NC1HWC2 data layout and storage

Taking feature (1, 13, 2, 2) with C2 = 8 as an example, the `NC1HWC2` is (1, 2, 2, 2, 8). Its storage is expanded below; the red parts are invalid data used for alignment padding:

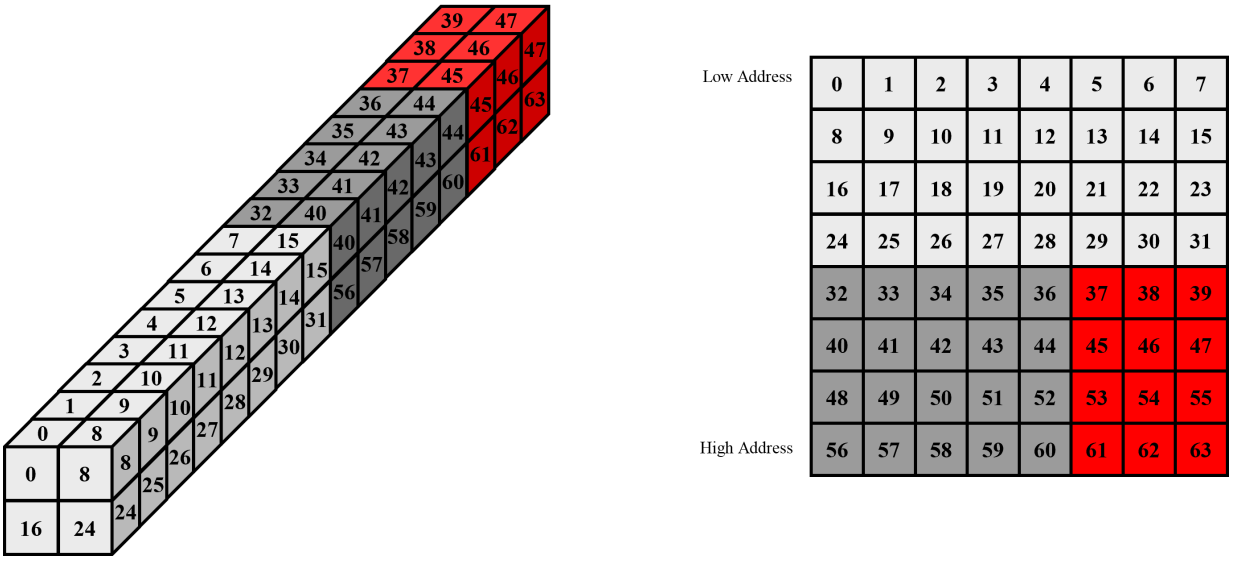


Figure 3-2 NC1HWC2 data layout expanded

After removing the invalid data, it becomes `NCHW` (1, 13, 2, 2):

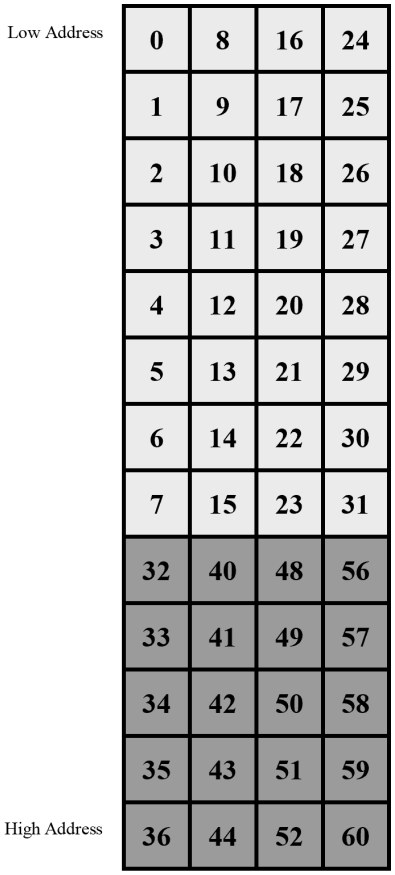


Figure 3-3 NCHW data layout

After removing the invalid data, it becomes `NHWC` (1, 2, 2, 13):

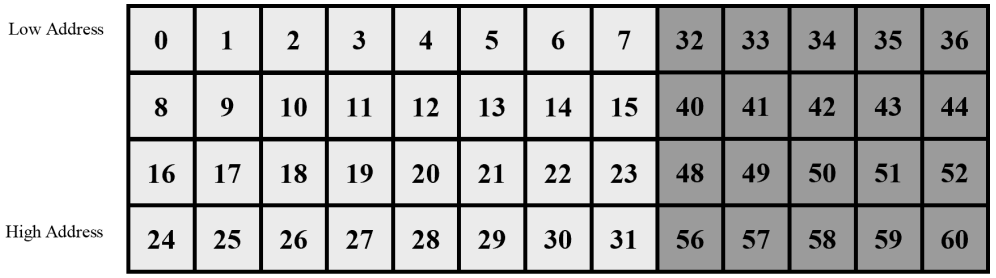


Figure 3-4 NHWC data layout

Layout Conversion

When you need to directly access NPU memory (for example, in zero-copy scenarios), convert the data between `NC1HWC2` and `NCHW` / `NHWC`. The following are common conversion examples:

`NC1HWC2` to `NCHW` (INT8):

```
#define ALIGNED_SIZE(x, bytes) (((x) + ((bytes) - 1)) & (~((bytes) - 1)))
int NC1HWC2_to_NCHW(const int8_t* src, int8_t* dst, int* dims, int channel)
{
  int batch        = dims[0];
  int height       = dims[2];
  int width        = dims[3];
  int cn           = dims[4];
  int align_surf   = width * height == 1 ? 1 : ALIGNED_SIZE(width * height, 4);

  int src_batch_elems = dims[1] * align_surf * dims[4];
  int dst_batch_elems = channel * width * height;

  for (int i = 0; i < batch; ++i) {
    int idx = 0;
    for (int c = 0; c < channel; ++c) {
      int           plane  = c / cn;
      const int8_t* src_c = plane * align_surf * cn + src;
      int           offset = c % cn;

      for (int cur_h = 0; cur_h < height; ++cur_h) {
        for (int cur_w = 0; cur_w < width; ++cur_w) {
          int cur_hw = cur_h * width + cur_w;
          dst[idx++] = src_c[cn * cur_hw + offset];
        }
      }
    }

    src += src_batch_elems;
    dst += dst_batch_elems;
  }
  return 0;
}
```

`NCHW` to `NC1HWC2` (INT8):

```
#define ALIGNED_SIZE(x, bytes) (((x) + ((bytes) - 1)) & (~((bytes) - 1)))
int NCHW_to_NC1HWC2(const int8_t* src, int8_t* dst, int* dims, int channel)
{
  int batch        = dims[0];
  int height       = dims[2];
  int width        = dims[3];
  int cn           = dims[4];
  int align_surf   = width * height == 1 ? 1 : ALIGNED_SIZE(width * height, 4);
  int hw = height * width;

  int src_batch_elems = channel * width * height;
  int dst_batch_elems = dims[1] * align_surf * cn;

  for (int i = 0; i < batch; ++i) {
    for (int c = 0; c < channel; ++c) {
      int plane     = c / cn;
      int offset    = c % cn;
      int8_t* dst_c = dst + (size_t)plane * align_surf * cn;

      for (int cur_h = 0; cur_h < height; ++cur_h) {
        for (int cur_w = 0; cur_w < width; ++cur_w) {
          int cur_hw                  = cur_h * width + cur_w;
          dst_c[cn * cur_hw + offset] = src[c * hw + cur_h * width + cur_w];
        }
      }
    }

    src += src_batch_elems;
    dst += dst_batch_elems;
  }

  return 0;
}
```

> **Note**: For the complete conversion code (including `NC1HWC2` to `NHWC`), refer to the utility functions in rknn3-model-zoo.

3.1.3 Input and Output Memory

In RKNN3 Runtime, the input and output data of a model are managed through `rknn3_tensor_mem` memory objects. For on-device deployment, allocate memory for each input and output tensor, bind it to the tensor attributes, synchronize the data between the CPU and the NPU, and finally release the memory. This section describes memory allocation methods, the synchronization mechanism, and release conventions.

Memory Allocation Methods

RKNN3 provides multiple memory allocation methods for different scenarios:

- **`rknn3_create_mem()`**: The Runtime allocates device memory of the specified size. This is the most commonly used allocation method. When allocating, specify the memory size, core ID (the core the memory belongs to in multi-core scenarios), and memory allocation flags (`rknn3_mem_alloc_flags`). The flags control the cache attributes of the memory: `RKNN3_FLAG_MEMORY_CACHEABLE` (default; creates cacheable memory with better

performance but requires synchronization with `rknn3_mem_sync()`) and `RKNN3_FLAG_MEMORY_NON_CACHEABLE` (creates non-cacheable memory; no synchronization required but lower access performance). The allocated memory object is bound to the tensor through the `rknn3_tensor.mem` field, and the Runtime reads inputs and writes outputs through this binding during inference. This method suits most regular deployment scenarios.

- **`rknn3_create_mem_from_fd()`**: Creates a memory object from an existing file descriptor (fd) and reuses externally allocated memory (such as DMA-BUF). It suits zero-copy scenarios where memory must be shared with other hardware modules (such as cameras or displays). It avoids data copy overhead but requires a valid fd and a matching size for the external memory.
- **`rknn3_create_mem_from_phys()`**: Creates a memory object from a physical address (not implemented in the current version). It is designed for advanced scenarios that directly access physical memory.

Selection advice: For regular deployment, use `rknn3_create_mem()` first; it is simple and reliable. For zero-copy integration with modules such as cameras or RGA, use `rknn3_create_mem_from_fd()`.

Allocation example:

```
// Allocate cacheable memory for the input tensor
inputs[i].mem = rknn3_create_mem(ctx, input_attrs[i].aligned_size,
                                 input_attrs[i].core_id,
                                 RKNN3_FLAG_MEMORY_CACHEABLE);
```

Memory Synchronization

When memory is allocated with the `RKNN3_FLAG_MEMORY_CACHEABLE` flag (or the default `RKNN3_FLAG_MEMORY_FLAGS_DEFAULT`; both behave the same), the CPU and the device maintain separate caches. Data written directly to CPU-side memory may not be immediately visible to the device, and vice versa. Therefore, explicitly synchronize the memory data between the CPU and the device through `rknn3_mem_sync()` before and after inference to ensure data consistency. If memory is allocated with the `RKNN3_FLAG_MEMORY_NON_CACHEABLE` flag, no synchronization is required, but access performance is lower.

The synchronization direction is specified by `rknn3_mem_sync_mode`:

- **`RKNN3_MEMORY_SYNC_TO_DEVICE`**: Synchronizes CPU-side data to the device. Use it after the input data is ready and before inference executes, to ensure the device reads the latest input data.
- **`RKNN3_MEMORY_SYNC_FROM_DEVICE`**: Synchronizes device-side data to the CPU. Use it after inference completes and before reading the output results, to ensure the CPU reads the final results written by the device.
- **`RKNN3_MEMORY_SYNC_BIDIRECTIONAL`**: Synchronizes in both directions. It suits scenarios where both the CPU and the device read and write.

Synchronization example:

```
// Before inference: synchronize input data to the device
for (int i = 0; i < n_inputs; i++) {
    rknn3_mem_sync(ctx, inputs[i].mem, RKNN3_MEMORY_SYNC_TO_DEVICE);
}

// After inference: synchronize output data to the CPU
for (int i = 0; i < n_outputs; i++) {
    rknn3_mem_sync(ctx, outputs[i].mem, RKNN3_MEMORY_SYNC_FROM_DEVICE);
}
```

Precautions:

1. Synchronization is a blocking operation; it waits for the data transfer to complete. Perform input synchronization after all input data is written, and perform output synchronization after inference completes.
2. Each input tensor must be synchronized to the device individually, and each output tensor must be synchronized back to the CPU individually.
3. If you skip synchronization and directly read the output of cacheable memory, you may read stale cache data and get incorrect results.

Memory Release

After the memory is used, release it through `rknn3_destroy_mem()`. Release must follow these lifecycle conventions:

1. Memory release must complete before the context is destroyed (`rknn3_destroy()`); otherwise, resource leaks may occur.
2. Release the input and output memory first, then destroy the context.
3. Do not use released memory objects again; doing so causes undefined behavior.

Release example:

```
// Release input and output memory first
for (int i = 0; i < n_inputs; i++) {
    rknn3_destroy_mem(ctx, inputs[i].mem);
}
for (int i = 0; i < n_outputs; i++) {
    rknn3_destroy_mem(ctx, outputs[i].mem);
}
// Then destroy the context
rknn3_destroy(ctx);
```

3.1.4 Synchronous and Asynchronous Inference

RKNN3 Runtime provides two inference execution modes: synchronous and asynchronous.

- **Synchronous inference** (`rknn3_run()`): Blocks the current thread after invocation and returns after inference completes. It is simple to use and suits most scenarios. Before invocation, ensure the input data has been synchronized to the device (`RKNN3_MEMORY_SYNC_TO_DEVICE`); after inference completes, synchronize the output data back to the CPU (`RKNN3_MEMORY_SYNC_FROM_DEVICE`).

```
// 1. Synchronize input data to the device
for (int i = 0; i < n_inputs; i++) {
    rknn3_mem_sync(ctx, inputs[i].mem, RKNN3_MEMORY_SYNC_TO_DEVICE);
}

// 2. Run synchronous inference (blocks until completion)
int ret = rknn3_run(ctx, inputs, n_inputs, outputs, n_outputs);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_run failed! ret=%d\n", ret);
    return -1;
}

// 3. Synchronize output data to the CPU
for (int i = 0; i < n_outputs; i++) {
    rknn3_mem_sync(ctx, outputs[i].mem, RKNN3_MEMORY_SYNC_FROM_DEVICE);
}
```

- **Asynchronous inference** (`rknn3_run_async()` + `rknn3_wait()`): First initiates non-blocking inference through `rknn3_run_async()`. The main thread can continue with other tasks, then waits for inference to complete through `rknn3_wait()`. It suits scenarios that process other work in parallel during inference (such as data pre-processing) and can improve overall throughput. **Note: `rknn3_run_async()` and `rknn3_wait()` are not implemented in the current version.**

Selection advice: The current version supports only synchronous inference (`rknn3_run()`); the asynchronous inference interfaces are not implemented yet. Once supported in later versions, use the asynchronous mode if parallel tasks are needed during inference.

3.1.5 Error Handling and Resource Release

Error Handling

Interfaces of the RKNN3 C API return an `int` status code. `RKNN3_SUCCESS` (0) indicates success, and negative values indicate errors. For on-device deployment, check the return value of every interface invocation. On error, do not continue with subsequent operations; jump to the resource cleanup routine and exit, to avoid undefined behavior caused by executing in an abnormal state.

Common status codes and their meanings:

Status code	Value	Description
`RKNN3_SUCCESS`	0	Execution succeeded
`RKNN3_ERR_FAIL`	-1	Execution failed
`RKNN3_ERR_ARGUMENT_INVALID`	-2	Invalid argument
`RKNN3_ERR_MODEL_INVALID`	-3	Invalid model
`RKNN3_ERR_CTX_INVALID`	-4	Invalid context
`RKNN3_ERR_OUT_OF_MEMORY`	-6	Out of memory
`RKNN3_ERR_TIMEOUT`	-7	Execution timed out
`RKNN3_ERR_DEVICE_UNAVAILABLE`	-10	Device unavailable
`RKNN3_ERR_COMMUNICATION`	-13	Communication error
`RKNN3_ERR_INCOMPATIBLE_VERSION`	-18	Incompatible component versions

For the complete error code list, see the *RKNN3_Runtime_C_API_Reference*.

Error handling example for the initialization stage (jumps to the cleanup routine on error):

```
int ret = rknn3_init(&ctx, NULL);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_init failed! ret=%d\n", ret);
    goto out;
}
```

Resource Release

In on-device deployment, release all types of resources in the correct order to avoid resource leaks. It is recommended to use `goto` to jump to a unified cleanup routine, ensuring correct resource release on both the normal and error paths.

- **General models**: Release order: release input and output memory -> destroy the context.

```
out:
    // Release input and output memory
    for (int i = 0; i < n_inputs; i++) {
        if (inputs[i].mem) rknn3_destroy_mem(ctx, inputs[i].mem);
    }
    for (int i = 0; i < n_outputs; i++) {
        if (outputs[i].mem) rknn3_destroy_mem(ctx, outputs[i].mem);
    }
    // Destroy the context
    if (ctx) rknn3_destroy(ctx);
```

- **LLM / MLLM models**: Release order: destroy the session -> release input and output memory -> destroy the context.

```
out:
    // Destroy the session
    if (session) rknn3_session_destroy(session);
    // Release input and output memory
    for (int i = 0; i < n_inputs; i++) {
        if (inputs[i].mem) rknn3_destroy_mem(ctx, inputs[i].mem);
    }
    for (int i = 0; i < n_outputs; i++) {
        if (outputs[i].mem) rknn3_destroy_mem(ctx, outputs[i].mem);
    }
    // Destroy the context
    if (ctx) rknn3_destroy(ctx);
```

> **Note**: After the context is destroyed, all resources created from it (memory, sessions, etc.) are no longer accessible. Ensure all resources are released before the context is destroyed.

3.2 General Model C API Deployment

This section describes how to deploy general models (CNN / ViT, etc.) with the RKNN3 Runtime C API, covering the complete flow from model initialization through inference execution to resource release. It suits production deployment and performance-sensitive scenarios.

3.2.1 Complete Invocation Flow

The typical invocation sequence for general model C API deployment is: initialize context -> load model -> initialize model -> query attributes -> create memory -> prepare input -> execute inference -> process output -> release resources. The overall flow is shown in the figure below:

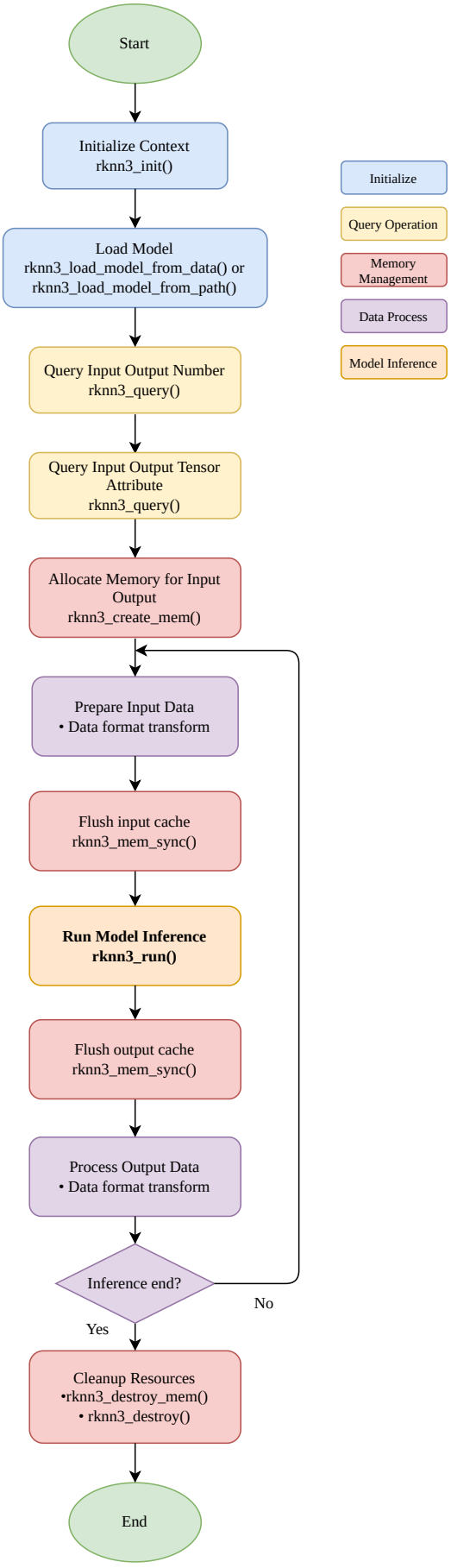


Figure 3-5 RKNN3 C API inference flow

The following subsections describe the operations and precautions for each step in this flow.

3.2.2 Model Initialization

Model initialization consists of three steps: create the context, load the model, and configure runtime parameters.

1. Create the context

Create the runtime context through `rknn3_init()`. This is the prerequisite for all subsequent operations. In multi-device scenarios, specify the device ID through `init_extend`; in single-device scenarios, pass `NULL`:

```
rknn3_context ctx;
int ret = rknn3_init(&ctx, NULL);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_init failed! ret=%d\n", ret);
    goto out;
}
```

2. Load the model

Load the `.rknn` model and `.weight` weights from a file through `rknn3_load_model_from_path()`, or load from memory through `rknn3_load_model_from_data()` (suits scenarios where the model is already in memory, such as loading an encrypted model after decryption):

```
// Load from file
ret = rknn3_load_model_from_path(ctx, model_path, weight_path);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_load_model_from_path failed! ret=%d\n", ret);
    goto out;
}
```

> **Note**: Passing `NULL` for `weight_path` enables streaming weight loading mode. After `rknn3_model_init()`, upload the weights in chunks through `rknn3_load_weight_chunk()`. This suits memory-constrained scenarios.

3. Configure runtime parameters

Configure the model runtime parameters and complete initialization through `rknn3_model_init()`. The `rknn3_config` structure contains the core mask (`run_core_mask`), run priority (`priority`), timeout (`run_timeout`), and other configurations. The core mask controls which NPU cores the model runs on. It must match the `core_mask` configured during model conversion; otherwise, an error is reported:

```
rknn3_config config;
memset(&config, 0, sizeof(rknn3_config));
config.run_core_mask = 0xff;  // Use all NPU cores
ret = rknn3_model_init(ctx, &config);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_model_init failed! ret=%d\n", ret);
    goto out;
}
```

3.2.3 Input and Output Preparation

After model initialization, query the model input and output attributes, then allocate memory and prepare the input data accordingly.

1. Query input and output attributes

Query the number and attributes of the model input and output tensors through `rknn3_query()`. First query the numbers (`RKNN3_QUERY_IN_OUT_NUM`), then query the attributes of each tensor (`RKNN3_QUERY_INPUT_ATTR` / `RKNN3_QUERY_OUTPUT_ATTR`) to obtain information such as shape / dtype / layout, quantization parameters, `aligned_size` (aligned memory size), and `core_id` (the core the tensor belongs to):

```
// Query the number of inputs and outputs
rknn3_input_output_num io_num;
ret = rknn3_query(ctx, RKNN3_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));
if (ret != RKNN3_SUCCESS) {
    printf("query io_num failed! ret=%d\n", ret);
    goto out;
}

// Query the attributes of each input tensor
rknn3_tensor_attr input_attrs[io_num.n_input];
for (uint32_t i = 0; i < io_num.n_input; i++) {
    input_attrs[i].index = i;
    ret = rknn3_query(ctx, RKNN3_QUERY_INPUT_ATTR, &input_attrs[i],
sizeof(rknn3_tensor_attr));
    if (ret != RKNN3_SUCCESS) {
        printf("query input_attr %d failed! ret=%d\n", i, ret);
        goto out;
    }
}

// Query the attributes of each output tensor
rknn3_tensor_attr output_attrs[io_num.n_output];
for (uint32_t i = 0; i < io_num.n_output; i++) {
    output_attrs[i].index = i;
    ret = rknn3_query(ctx, RKNN3_QUERY_OUTPUT_ATTR, &output_attrs[i],
sizeof(rknn3_tensor_attr));
    if (ret != RKNN3_SUCCESS) {
        printf("query output_attr %d failed! ret=%d\n", i, ret);
        goto out;
    }
}
```

2. Allocate input and output memory

Based on the queried `aligned_size` and `core_id`, allocate memory for each input and output tensor through `rknn3_create_mem()` and bind it to the `mem` field of `rknn3_tensor`:

```
rknn3_tensor inputs[io_num.n_input];

// Create memory for the input tensor
for (uint32_t i = 0; i < io_num.n_input; i++) {
    inputs[i].mem = rknn3_create_mem(ctx, input_attrs[i].aligned_size,
                                     input_attrs[i].core_id,
                                     RKNN3_FLAG_MEMORY_CACHEABLE);
    if (inputs[i].mem == NULL) {
        printf("create input mem %d failed!\n", i);
        goto out;
    }
}

// Create memory for the output tensor
rknn3_tensor outputs[io_num.n_output];
for (uint32_t i = 0; i < io_num.n_output; i++) {
    outputs[i].mem = rknn3_create_mem(ctx, output_attrs[i].aligned_size,
                                      output_attrs[i].core_id,
                                      RKNN3_FLAG_MEMORY_CACHEABLE);
    if (outputs[i].mem == NULL) {
        printf("create output mem %d failed!\n", i);
        goto out;
    }
}
```

3. Prepare input data

Before inference, write the preprocessed input data (after resize, color space conversion, etc.) into the corresponding input tensor memory (`inputs[i].mem`). After writing, invoke `rknn3_mem_sync()` to synchronize the data to the NPU device side.

> **Note**:
>
> - **Data attribute consistency**: The `dtype` (data type) and `shape` of the input data must strictly match the queried tensor attributes; otherwise, inference errors occur.
> - **Mean and normalization processing**: If `mean_values` and `std_values` were configured in `rknn.config()` during model conversion, the C API inference stage **does not require manual mean normalization**, which effectively reduces latency in the deployment stage.
> - **Data layout requirements**: The input data layout must match the `layout` field in the tensor attributes. RKNN3 Runtime internally supports the common `NHWC` and `NCHW` layouts and automatically handles the necessary layout conversions during inference.

```
// Write pre-processed data into the input memory
// memcpy(inputs[0].mem->virt_addr, input_data, input_attrs[0].aligned_size);

// Synchronize input data to the device
for (uint32_t i = 0; i < io_num.n_input; i++) {
    ret = rknn3_mem_sync(ctx, inputs[i].mem, RKNN3_MEMORY_SYNC_TO_DEVICE);
    if (ret != RKNN3_SUCCESS) {
        printf("mem_sync input %d failed! ret=%d\n", i, ret);
        goto out;
    }
}
```

3.2.4 Inference Execution

After the input data is synchronized to the device, execute synchronous inference through `rknn3_run()`, passing the context, the input tensor array and count, and the output tensor array and count. This interface blocks until inference completes:

```
ret = rknn3_run(ctx, inputs, io_num.n_input, outputs, io_num.n_output);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_run failed! ret=%d\n", ret);
    goto out;
}
```

After inference completes, the output data has been written to the output tensor memory but remains on the device side. Synchronize it back to the CPU through `rknn3_mem_sync()` before reading it (see 3.2.5 Output Processing).

3.2.5 Output Processing

After inference completes, the output data is on the device side. Synchronize it back to the CPU first, then read and process the results according to the output tensor attributes.

1. Synchronize output data

Synchronize the memory of each output tensor from the device back to the CPU:

```
for (uint32_t i = 0; i < io_num.n_output; i++) {
    ret = rknn3_mem_sync(ctx, outputs[i].mem, RKNN3_MEMORY_SYNC_FROM_DEVICE);
    if (ret != RKNN3_SUCCESS) {
        printf("mem_sync output %d failed! ret=%d\n", i, ret);
        goto out;
    }
}
```

2. Read output data

After synchronization, read the output data through the virtual address of `outputs[i].mem`. Note the following when reading:

- **Data type**: Determine the data type to read based on `output_attrs[i].dtype` (such as FP16 / INT8).

- **Data layout**: The output layout may be `NC1HWC2`. Convert it to `NCHW` or `NHWC` before post-processing (see 3.1.2 for the conversion method).
- **Quantization parameters**: If the output is quantized (INT8, etc.), dequantize it with the `scale` and `zero_point` in `output_attrs[i].qnt_info` to restore floating-point values.

```
// Read output data (FP16 in this example)
float16* output_data = (float16*)outputs[0].mem->virt_addr;
// Post-process according to output_attrs[0].shape and layout
// ...
```

3. Post-processing

Post-processing depends on the specific model and application scenario, for example NMS for object detection models or argmax for classification models. Refer to the example code for each model in rknn3-model-zoo.

3.2.6 Complete Example

The following is a complete example of general model C API inference, integrating model initialization, input and output preparation, inference execution, output processing, and resource release:

```c
#include "rknn3_api.h"

int main() {
    int ret;
    rknn3_context ctx = NULL;

    // 1. Initialize the context
    ret = rknn3_init(&ctx, NULL);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_init failed! ret=%d\n", ret);
        goto out;
    }

    // 2. Load the model
    ret = rknn3_load_model_from_path(ctx, "model.rknn", "model.weight");
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_load_model_from_path failed! ret=%d\n", ret);
        goto out;
    }

    // 3. Initialize the model
    rknn3_config config = {0};
    config.run_core_mask = 0xff;
    ret = rknn3_model_init(ctx, &config);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_model_init failed! ret=%d\n", ret);
        goto out;
    }

    // 4. Query the number of inputs and outputs
    rknn3_input_output_num io_num;
    ret = rknn3_query(ctx, RKNN3_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));
    if (ret != RKNN3_SUCCESS) {
        printf("query io_num failed! ret=%d\n", ret);
        goto out;
    }

    // 5. Query attributes and allocate memory
    rknn3_tensor_attr input_attrs[io_num.n_input];
    rknn3_tensor_attr output_attrs[io_num.n_output];
    rknn3_tensor inputs[io_num.n_input];
    rknn3_tensor outputs[io_num.n_output];

    for (uint32_t i = 0; i < io_num.n_input; i++) {
        input_attrs[i].index = i;
        ret = rknn3_query(ctx, RKNN3_QUERY_INPUT_ATTR, &input_attrs[i],
sizeof(rknn3_tensor_attr));
        if (ret != RKNN3_SUCCESS) {
            printf("query input_attr %d failed! ret=%d\n", i, ret);
            goto out;
        }
        inputs[i].mem = rknn3_create_mem(ctx, input_attrs[i].aligned_size,
                                         input_attrs[i].core_id,
RKNN3_FLAG_MEMORY_CACHEABLE);
        if (inputs[i].mem == NULL) {
            printf("create input mem %d failed!\n", i);
            goto out;
        }
    }
    for (uint32_t i = 0; i < io_num.n_output; i++) {
```

```
        output_attrs[i].index = i;
        ret = rknn3_query(ctx, RKNN3_QUERY_OUTPUT_ATTR, &output_attrs[i],
sizeof(rknn3_tensor_attr));
        if (ret != RKNN3_SUCCESS) {
            printf("query output_attr %d failed! ret=%d\n", i, ret);
            goto out;
        }
        outputs[i].mem = rknn3_create_mem(ctx, output_attrs[i].aligned_size,
                                          output_attrs[i].core_id,
RKNN3_FLAG_MEMORY_CACHEABLE);
        if (outputs[i].mem == NULL) {
            printf("create output mem %d failed!\n", i);
            goto out;
        }
    }

    // 6. Prepare input data and synchronize it to the device
    // ... Fill inputs[0].mem->virt_addr ...
    for (uint32_t i = 0; i < io_num.n_input; i++) {
        ret = rknn3_mem_sync(ctx, inputs[i].mem, RKNN3_MEMORY_SYNC_TO_DEVICE);
        if (ret != RKNN3_SUCCESS) {
            printf("mem_sync input %d failed! ret=%d\n", i, ret);
            goto out;
        }
    }

    // 7. Run inference
    ret = rknn3_run(ctx, inputs, io_num.n_input, outputs, io_num.n_output);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_run failed! ret=%d\n", ret);
        goto out;
    }

    // 8. Synchronize outputs and process them
    for (uint32_t i = 0; i < io_num.n_output; i++) {
        ret = rknn3_mem_sync(ctx, outputs[i].mem, RKNN3_MEMORY_SYNC_FROM_DEVICE);
        if (ret != RKNN3_SUCCESS) {
            printf("mem_sync output %d failed! ret=%d\n", i, ret);
            goto out;
        }
    }
    // ... Read outputs[i].mem->virt_addr and post-process ...

out:
    // 9. Release resources
    for (uint32_t i = 0; i < io_num.n_input; i++) {
        if (inputs[i].mem) rknn3_destroy_mem(ctx, inputs[i].mem);
    }
    for (uint32_t i = 0; i < io_num.n_output; i++) {
        if (outputs[i].mem) rknn3_destroy_mem(ctx, outputs[i].mem);
    }
    if (ctx) rknn3_destroy(ctx);
    return 0;
}
```

> **Note**: For complete example code of more models, refer to rknn3-model-zoo (such as `examples/yolov6/cpp/`).

3.2.7 Debugging Methods

If accuracy or performance issues arise during on-device deployment, use the following debugging tools for troubleshooting:

Exporting intermediate features

`rknn3_dump_features()` exports the model intermediate features layer by layer to a specified directory as `.npy` files, making it easy to compare with a floating-point model layer by layer and analyze accuracy issues. When exporting, specify the core mask (`core_mask`) and export range (`dump_selector`; for example, `"*"` or `"all"` to export all operators):

```
ret = rknn3_dump_features(ctx, inputs, n_inputs, NULL, 0,
                          "./dump/", 0, "all");
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_dump_features failed! ret=%d\n", ret);
}
```

Per-layer operator performance profiling

`rknn3_profile_ops()` prints the per-layer operator performance information, including statistics such as latency, cycles, and bandwidth, and outputs a summary table. `log_level` controls the output verbosity: 0 prints only the latency summary, 1 adds the per-operator latency table, and 2 further adds cycle and bandwidth information:

```
ret = rknn3_profile_ops(ctx, inputs, n_inputs, NULL, 0, 2);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_profile_ops failed! ret=%d\n", ret);
}
```

Troubleshooting common issues

1. **Memory alignment**: Use `aligned_size` instead of `n_elems * sizeof(dtype)` when allocating memory; otherwise, out-of-bounds memory access may occur.
2. **Core ID**: Use `attr->core_id` when creating memory to ensure the memory is allocated on the correct core.
3. **Cache synchronization**: When using cacheable memory, invoke `rknn3_mem_sync()` correctly; otherwise, data inconsistency may occur.
4. **Layout conversion**: The `NC1HWC2` layout requires special handling. Note that FP16 and INT8 have different sub-channel counts (see 3.1.2).
5. **Quantization parameters**: INT8 models require `qnt_info.scale` and `zero_point` for quantization/dequantization.
6. **core_mask consistency**: The runtime `core_mask` must match the configuration used during model conversion; otherwise, an error is reported.

3.3 LLM and MLLM Session Deployment

This section describes how to deploy LLM / MLLM models with the RKNN3 Runtime Session API, covering the session lifecycle, input and callbacks, inference execution, chat templates,

multimodal input, and more. The Session API suits LLM / MLLM deployment scenarios that require advanced features such as session management, KVCache, and sampling control.

3.3.1 Complete Invocation Flow

The typical invocation sequence for LLM / MLLM Session deployment is: initialize context -> load model -> initialize model -> initialize Session -> set callbacks -> prepare input -> execute inference -> process results -> destroy Session -> destroy context. The overall flow is shown in the figure below:

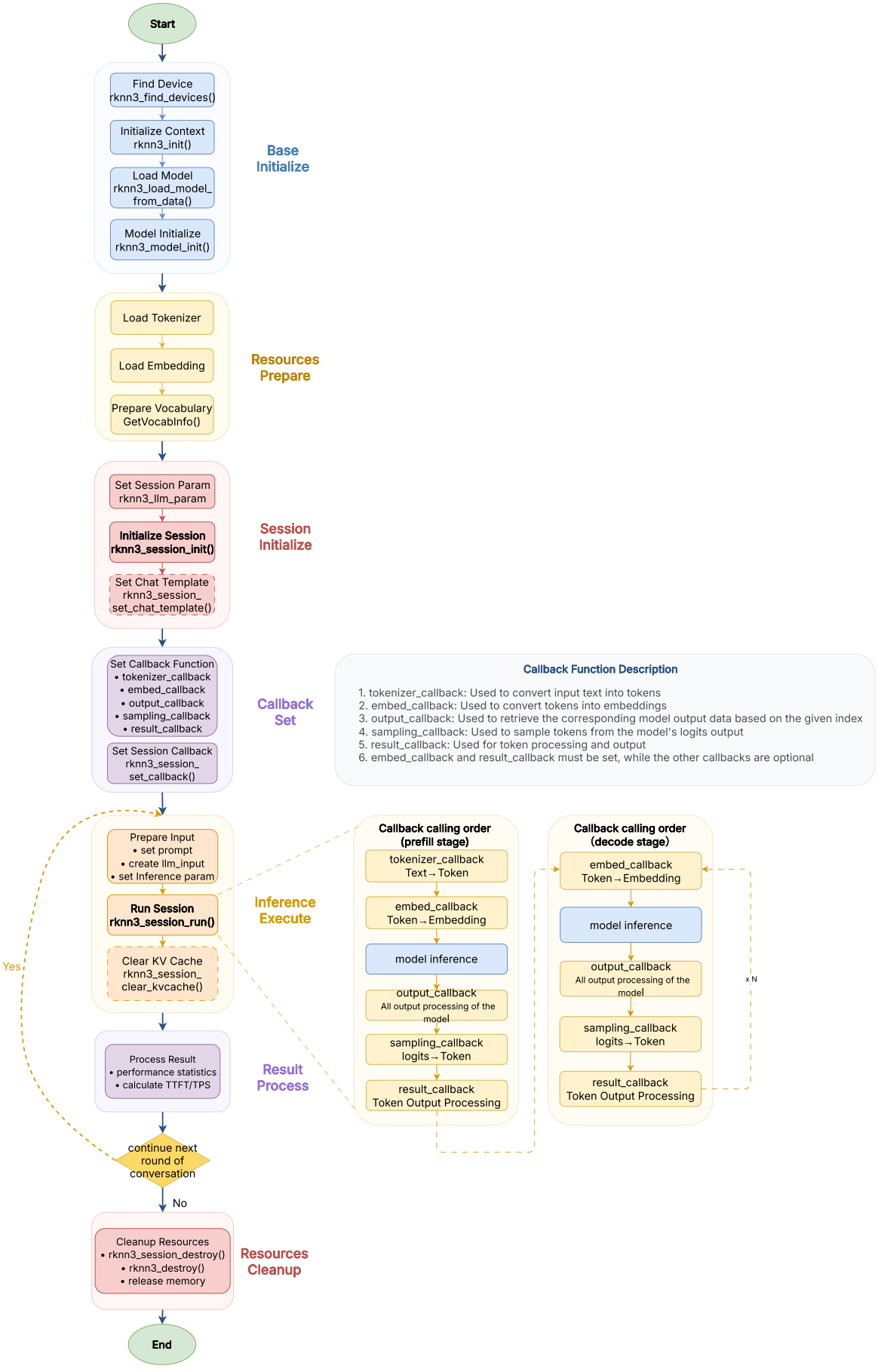


Figure 3-6 LLM model deployment flow based on Session management

The following subsections describe the operations and precautions for each step in this flow.

3.3.2 Session Lifecycle

After the model is loaded and initialized (`rknn3_init()` -> `rknn3_load_model_from_path()` -> `rknn3_model_init()`), create a session through `rknn3_session_init()` . When creating the session, configure the `rknn3_llm_param` structure, which contains the following key parameters:

- `logits_name` : The name of the inference output node.
- `max_context_len` : The maximum context length (in tokens).
- `sampling_param` : The sampling parameters, which control the randomness and quality of generation, including:
- `temperature` : The sampling temperature. A larger value produces more random generation (higher diversity); a smaller value produces more deterministic generation (favoring high-probability tokens).
- `top_k` : Top-K sampling. Samples only from the K highest-probability tokens. A smaller value produces more conservative generation.
- `top_p` : Top-P sampling. Samples from the smallest set of tokens whose cumulative probability reaches P, dynamically adjusting the candidate range.
- `repeat_penalty` : The repeat penalty. Penalizes already generated tokens to reduce the probability of generating repeated content.
- `frequency_penalty` : The frequency penalty. Applies an additional penalty to high-frequency tokens.
- `presence_penalty` : The presence penalty. Penalizes tokens that have already appeared, encouraging the generation of new content.
- `vocab_info` : The vocabulary information, usually obtained from the tokenizer's `GetVocabInfo()` , including:
- `vocab_size` : The vocabulary size.
- `special_bos_id` : The list of special beginning-of-sequence (BOS) token IDs; `n_special_bos_id` is the count.
- `special_eos_id` : The list of special end-of-sequence (EOS) token IDs; `n_special_eos_id` is the count. It controls when inference ends.
- `linefeed_id` : The token ID of the line feed.
- `ignore_eos_token` : Whether to ignore the EOS token during generation. When set to True, generation does not stop even if an end token is encountered.

```
rknn3_llm_param params;
memset(&params, 0, sizeof(rknn3_llm_param));

params.logits_name = "logits";
params.max_context_len = 1024;
params.sampling_param.temperature = 1.0f;
params.sampling_param.top_k = 1;
params.sampling_param.top_p = 0.9;
params.sampling_param.repeat_penalty = 1.1f;
params.vocab_info.vocab_size = vocab_info.vocab_size;
params.vocab_info.n_special_eos_id = vocab_info.n_special_eos_id;
memcpy(params.vocab_info.special_eos_id, vocab_info.special_eos_id,
sizeof(vocab_info.special_eos_id));

rknn3_session *session = rknn3_session_init(ctx, &params, 1);  /* The third
parameter n_params is the number of parameter sets */
if (session == NULL) {
    printf("rknn3_session_init failed!\n");
    goto out;
}
```

After the session is used, destroy it through `rknn3_session_destroy()` to release session-related resources (KVCache, etc.). Do not use the session pointer after destruction:

```
ret = rknn3_session_destroy(session);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_session_destroy failed! ret=%d\n", ret);
}
```

> **Note**: One context can create multiple sessions. The sessions share model weights and internal memory, but their KVCache and LoRA enable states are independent of each other. Sessions cannot execute concurrently; only one session can run at a time.

3.3.3 Input and Callbacks

LLM inference handles tokenization, embedding, sampling, result output, and other steps through the callback mechanism. After creating a session, register the callback functions through `rknn3_session_set_callback()`. The `RKLLMCallback` structure contains the following callbacks:

- **`result_callback`**: The result callback. Triggered each time a token is generated, and used to process the model output (for example, converting tokens to text output). Typically used with the tokenizer's `TokenToPiece()` or `Decode()`.
- **`tokenizer_callback`** (optional): The tokenizer callback. Converts input text into a token id sequence. Typically implemented by invoking the tokenizer's `Tokenize()`.
- **`embed_callback`** (optional): The embedding callback. Converts token ids into embedding vectors. Typically looked up by token id from the `embed.bin` file.
- **`sampling_callback`** (optional): The sampling callback. Samples token ids from the logits output by the model. If not set, the Runtime uses the default sampling strategy (configured through `sampling_param`).

- **`output_callback`** (optional): The output callback. Obtains the model output tensors. Use it with the `output_tensors` and `n_output_tensors` fields to specify the output tensor array. It suits scenarios that directly process output tensors (for example, using the output as input to other models).
- **`input_callback`** (optional): The input callback. Provides custom input tensors. Use it with the `input_tensors_index` field to specify the input tensor indexes. It suits scenarios that directly pass in preprocessed tensors.

```
RKLLMCallback callback;
memset(&callback, 0, sizeof(RKLLMCallback));

callback.result_callback    = result_callback;
callback.result_userdata    = tokenizer;
callback.tokenizer_callback = tokenizer_callback;
callback.tokenizer_userdata = tokenizer;
callback.embed_callback     = embed_callback;
callback.embed_userdata     = &embedding_info;
callback.sampling_callback  = sampling_callback;
callback.sampling_userdata  = &embedding_info;

ret = rknn3_session_set_callback(session, &callback);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_session_set_callback failed! ret=%d\n", ret);
    goto out;
}
```

> **Note**: For implementations of `tokenizer_callback` and `embed_callback`, refer to 2.2.4 Model and Tokenizer Adaptation. If Tie Word mode is used (see 4.2.9 Tie Word Embedding), `embed_callback` does not need to be set.

3.3.4 Inference Execution and Interruption

Executing inference

After the session is created and the callbacks are set, execute inference through `rknn3_session_run()`. Prepare the `rknn3_llm_input` input structure and the `rknn3_llm_infer_param` inference parameters. Specify the input type through `input_type`, commonly `RKNN3_LLM_INPUT_PROMPT` (text input). The inference parameters include `keep_history` (whether to keep the history context) and `max_new_tokens` (the maximum number of tokens to generate):

```
char *prompt = "Please explain the basic concept of relativity";
rknn3_llm_infer_param param = {.keep_history = 0, .max_new_tokens = 128};
rknn3_llm_tensor input_tensor = {.name = NULL, .prompt = prompt,
                                 .embed = NULL, .tokens = NULL,
                                 .n_tokens = 0, .enable_thinking = false};
rknn3_llm_input input;
input.input_type = RKNN3_LLM_INPUT_PROMPT;
input.llm_input = input_tensor;

rknn3_llm_input inputs[1] = {input};
ret = rknn3_session_run(session, inputs, 1, &param);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_session_run failed! ret=%d\n", ret);
    goto out;
}
```

During inference, each generated token is returned through the `result_callback` callback, and users process the output in the callback (for example, printing text). Inference continues until `max_new_tokens` is reached or an EOS token is encountered.

The Runtime also provides the asynchronous inference interface `rknn3_session_run_async()`. Its usage is the same as the synchronous interface, but it does not block the current thread.

> **Note**: For general models, `rknn3_run_async()` / `rknn3_wait()` are not implemented in the current version (see 3.1.4); for LLM Sessions, `rknn3_session_run_async()` is available.

Interrupting inference

During inference, interrupt the current session inference through `rknn3_session_stop()`. The Runtime stops generation and returns:

```
ret = rknn3_session_stop(session);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_session_stop failed! ret=%d\n", ret);
}
```

3.3.5 Chat Template

Applicable Scenarios

Different LLMs use different conversation formats (such as ChatML / Llama-3 / Qwen). Models learn specific system prompts, role markers, and delimiters during training. Feeding raw user text directly into the model degrades generation quality or produces irrelevant answers due to missing format markers. The Chat Template wraps the user input with the conversation format required by the model so that the input meets the model's expectations.

Usage

Set the chat template through `rknn3_session_set_chat_template()`. It consists of three parts:

- `system_prompt`: The system prompt. It defines the model's role and behavioral context and is always prepended at the beginning of each conversation round.

- `prompt_prefix`: The user input prefix. It marks the beginning of a user message (such as `<|im_start|>user\n`).
- `prompt_postfix`: The user input postfix. It marks the end of a user message and prompts the model to generate (such as `<|im_end|>\n<|im_start|>assistant\n`).

Once set, whenever a PROMPT input is passed to `rknn3_session_run()`, the Runtime automatically concatenates `system_prompt + prompt_prefix + user input + prompt_postfix` into the complete conversation and feeds it to the model. Invoke this interface after `rknn3_session_init()` and before `rknn3_session_run()`. Usually it only needs to be set once.

```
const char* system_prompt  = "<|im_start|>system\nYou are a helpful assistant.
<|im_end|>\n";
const char* prompt_prefix  = "<|im_start|>user\n";
const char* prompt_postfix = "<|im_end|>\n<|im_start|>assistant\n";
ret = rknn3_session_set_chat_template(session, system_prompt, prompt_prefix,
prompt_postfix);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_session_set_chat_template failed! ret=%d\n", ret);
    goto out;
}
```

> **Note**: The template strings must strictly match the conversation format used during model training. They can be derived from the `chat_template` field in the model's `tokenizer_config.json`. An incorrectly set template causes chaotic or repetitive model output. If a chat template was already configured through the config file during model conversion (see 2.2.2), this setting overrides it at runtime; if not set, the conversion-time configuration is used.

3.3.6 Multimodal Input

Applicable Scenarios

In addition to text, an MLLM (multimodal large language model) also accepts inputs such as images, audio, and video. The typical flow is: first use the corresponding encoder model (vision/audio/video) to encode the raw modality data into embedding vectors (embeds), then feed the text prompt and the modality embeddings together into the LLM, which fuses the multimodal information to generate an answer. RKNN3 supports this flow through the `RKNN3_LLM_INPUT_MULTIMODAL` input type.

Input Structure

Multimodal input uses the `rknn3_llm_multimodal_tensor` structure (`rknn3_llm_input.multimodal_input`). Its members fall into two parts:

- **Text part**: Provide either `prompt` (text prompt) or `tokens` (token id array); if both are provided, `prompt` is used for inference by default. `enable_thinking` controls whether thinking mode is enabled for this input. When used together with Grammar, see 4.2.11 for the complete output constraint rules.

- **Modality sub-structures**: The three sub-structures `image`, `audio`, and `video` are structurally symmetric. Each contains the `*_embed` embedding pointer from the corresponding encoder model output, the token count (`n_*_tokens`), the modality count (`n_image` / `n_audio` / `n_video`), and the placeholder tags in the text (`*_start` / `*_end` / `*_content`, which must match the special tokens of the model tokenizer). Populate one or more of them based on the actual input modalities. For the complete member definitions, see the *RKNN3_Runtime_C_API_Reference*.

Usage (VLM example)

In VLM (vision-language model) deployment, the vision model and the LLM model are two separate `.rknn` files that must be initialized and run separately. The vision model uses the general RKNN3 C API (see 3.2) to infer the image embeddings. On the LLM side, set the input type to `RKNN3_LLM_INPUT_MULTIMODAL`, populate the `image` sub-structure and the text prompt, and then invoke `rknn3_session_run()`:

```
/* img_embeds is the output of vision model inference */
rknn3_llm_multimodal_tensor tensor;
memset(&tensor, 0, sizeof(rknn3_llm_multimodal_tensor));
tensor.name   = "input_embeds";
tensor.prompt = prompt;                          // Text prompt

tensor.image.image_embed = img_embeds;           // Image embeddings output by the
vision model
if (rknn_app_ctx.vision.embeds_ndims == 2) {
    tensor.image.n_image_tokens = rknn_app_ctx.vision.embeds_shape[0];
    tensor.image.n_image        = 1;
} else {
    tensor.image.n_image_tokens = rknn_app_ctx.vision.embeds_shape[1];
    tensor.image.n_image        = rknn_app_ctx.vision.embeds_shape[0];
}
tensor.image.image_width   = rknn_app_ctx.vision.model_width;
tensor.image.image_height  = rknn_app_ctx.vision.model_height;
tensor.image.image_start   = "<|vision_start|>";   // Must match the model tokenizer
tensor.image.image_end     = "<|vision_end|>";
tensor.image.image_content = "<|image_pad|>";
tensor.enable_thinking = false;

rknn3_llm_input inputs[1];
memset(inputs, 0, sizeof(inputs));
inputs[0].input_type       = RKNN3_LLM_INPUT_MULTIMODAL;
inputs[0].multimodal_input = tensor;

ret = rknn3_session_run(session, inputs, 1, &param);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_session_run failed! ret=%d\n", ret);
    goto out;
}
```

> **Note**: The audio / video modalities are populated exactly like the image modality; simply replace the `image_*` members with the corresponding `audio_*` / `video_*` members. For complete multimodal deployment examples (vision + LLM integration), refer to the Qwen2.5-VL / FastVLM example projects in `rknn3-model-zoo`.

3.3.7 Complete Example

This section provides a complete invocation example of LLM Session deployment, chaining together the lifecycle, callback, and inference steps described above. For the complete project code, refer to the SDK example directory `rknn3-runtime/examples/rknn3_session_test_demo/`.

The complete example covers the following steps:

1. **Initialize the context and load the model**: `rknn3_init()` -> `rknn3_load_model_from_path()` -> `rknn3_model_init()`.
2. **Create the session**: Configure `rknn3_llm_param` (logits_name / max_context_len / sampling_param / vocab_info), and invoke `rknn3_session_init()`.
3. **Set callbacks**: Populate `RKLLMCallback` (result_callback / tokenizer_callback / embed_callback / sampling_callback), and invoke `rknn3_session_set_callback()`.
4. **Set the chat template** (optional): Invoke `rknn3_session_set_chat_template()`.
5. **Execute inference**: Populate `rknn3_llm_input` (input_type=PROMPT) and `rknn3_llm_infer_param`, and invoke `rknn3_session_run()`. Results are returned in streaming mode through `result_callback`.
6. **Destroy resources**: `rknn3_session_destroy()` -> `rknn3_destroy()`.

```cpp
#include "rknn3_api.h"
#include "Tokenizer.h"

/* Result callback: invoked after each token is generated; handle different states
by LLMCallState */
static int result_callback(void* userdata, RKLLMResult* result, LLMCallState state)
{
    Tokenizer* tokenizer = (Tokenizer*)userdata;

    if (state == RKLLM_RUN_NORMAL) {
        std::string piece;
        if (result->num_tokens == 1) {
            piece = tokenizer->TokenToPiece(result->token_ids[0]);
        } else {
            piece = tokenizer->Decode(result->token_ids, result->num_tokens);
        }
        printf("%s", piece.c_str());
    } else if (state == RKLLM_RUN_FINISH) {
        printf("\n[generation finished]\n");
    } else if (state == RKLLM_RUN_ERROR) {
        printf("\n[inference error]\n");
    }
    fflush(stdout);
    return 0;
}

/* Tokenizer callback: converts text into a token id sequence */
int tokenizer_callback(void *userdata, const char *text, int32_t text_len, int32_t
*tokens, int32_t n_tokens_max)
{
    int n_tokens = 0;
    Tokenizer *tokenizer = (Tokenizer *)userdata;
    n_tokens = tokenizer->Tokenize(text, text_len, tokens, n_tokens_max);

    if (n_tokens <= 0)
    {
        printf("tokenizer failed for %s\n", text);
        return n_tokens;
    }
    return n_tokens;
}

/* Embedding callback: looks up embeddings in embed.bin by token id */
int embed_callback(void* userdata, int32_t* tokens, uint64_t num_tokens, void*
embed, uint64_t len)
{
    struct embedding_info* embed_info = (struct embedding_info*)userdata;

    if (len != num_tokens * embed_info->embedding_dim * sizeof(float16)) {
        printf("invalid embed buffer\n");
        return -1;
    }

    for (int n = 0; n < num_tokens; n++) {
        memcpy((unsigned char*)embed +
               n * embed_info->embedding_dim * sizeof(float16),
               embed_info->embedding_data + tokens[n] * embed_info->embedding_dim,
               embed_info->embedding_dim * sizeof(float16));
    }
    return 0;
```

```
}

int main(int argc, char** argv)
{
    int ret = 0;
    rknn3_context ctx = NULL;
    rknn3_session* session = NULL;

    /* Creation/loading of tokenizer and embedding_info is omitted; see the SDK
example for the complete implementation */
    Tokenizer* tokenizer = NULL;
    struct embedding_info embedding_info;

    /* 1. Initialize the context and load the model */
    ret = rknn3_init(&ctx, NULL);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_init failed! ret=%d\n", ret);
        goto out;
    }
    ret = rknn3_load_model_from_path(ctx, "model.rknn", NULL);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_load_model_from_path failed! ret=%d\n", ret);
        goto out;
    }
    rknn3_config config = {0};
    config.run_core_mask = 0xff;  /* Configure according to the actual core count */
    ret = rknn3_model_init(ctx, &config);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_model_init failed! ret=%d\n", ret);
        goto out;
    }

    /* 2. Create the session */
    rknn3_llm_param params;
    memset(&params, 0, sizeof(rknn3_llm_param));
    params.logits_name    = "logits";
    params.max_context_len = 1024;
    params.sampling_param.temperature   = 1.0f;
    params.sampling_param.top_k         = 1;
    params.sampling_param.top_p         = 0.9;
    params.sampling_param.repeat_penalty = 1.1f;
    /* vocab_info is populated by tokenizer->GetVocabInfo(); omitted here */

    session = rknn3_session_init(ctx, &params, 1);
    if (session == NULL) {
        printf("rknn3_session_init failed!\n");
        goto out;
    }

    /* 3. Set callbacks */
    RKLLMCallback callback;
    memset(&callback, 0, sizeof(RKLLMCallback));
    callback.result_callback    = result_callback;
    callback.result_userdata    = tokenizer;
    callback.tokenizer_callback = tokenizer_callback;
    callback.tokenizer_userdata = tokenizer;
    callback.embed_callback     = embed_callback;
    callback.embed_userdata     = &embedding_info;
    ret = rknn3_session_set_callback(session, &callback);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_session_set_callback failed! ret=%d\n", ret);
```

```
        goto out;
    }

    /* 4. Set the chat template */
    const char* system_prompt  = "<|im_start|>system\nYou are a helpful assistant.
<|im_end|>\n";
    const char* prompt_prefix  = "<|im_start|>user\n";
    const char* prompt_postfix = "<|im_end|>\n<|im_start|>assistant\n";
    ret = rknn3_session_set_chat_template(session, system_prompt, prompt_prefix,
prompt_postfix);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_session_set_chat_template failed! ret=%d\n", ret);
        goto out;
    }

    /* 5. Run inference */
    char* prompt = "Please explain the basic concept of relativity";
    rknn3_llm_infer_param param = {.keep_history = 0, .max_new_tokens = 128};
    rknn3_llm_tensor input_tensor = {.name = NULL, .prompt = prompt,
                                     .embed = NULL, .tokens = NULL,
                                     .n_tokens = 0, .enable_thinking = false};
    rknn3_llm_input inputs[1];
    inputs[0].input_type = RKNN3_LLM_INPUT_PROMPT;
    inputs[0].llm_input  = input_tensor;

    ret = rknn3_session_run(session, inputs, 1, &param);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_session_run failed! ret=%d\n", ret);
        goto out;
    }

out:
    /* 6. Release resources */
    if (session) {
        rknn3_session_destroy(session);
    }
    rknn3_destroy(ctx);
    return ret;
}
```

> **Note**: The example above omits preparation steps such as tokenizer initialization, the `GetVocabInfo()` invocation, and `embed.bin` loading. For the complete implementation, refer to the SDK example `rknn3-runtime/examples/rknn3_session_test_demo/`. The `state` parameter of `result_callback` is an `LLMCallState` enum. Common values include `RKLLM_RUN_NORMAL` (normal generation), `RKLLM_RUN_FINISH` (inference finished), `RKLLM_RUN_ERROR` (error), and `RKLLM_RUN_MAX_NEW_TOKEN_REACHED` (maximum generation count reached). For the complete state definitions, see the *RKNN3_Runtime_C_API_Reference*.

3.4 Toolkit Lite Python Deployment

Prerequisites: Before using Toolkit Lite, convert the model to `.rknn` + `.weight` with RKNN3-Toolkit on the PC side.

Positioning and Applicable Scenarios

RKNN3-Toolkit Lite is the on-device Python inference interface. It wraps the RKNN3 Runtime (C API) in Python and provides a high-level inference experience consistent with the traditional

`RKNN-Toolkit Lite`, while supporting LLM / MLLM capabilities such as Session, Callback, and KVCache. Compared with the C API:

Item	Toolkit Lite (Python)	Runtime (C API)
Development efficiency	High; rapid Python iteration	Low; requires compilation and deployment
Runtime performance	Slightly lower (wrapping overhead)	Optimal
Typical scenarios	Prototype validation, algorithm debugging, application development	Production deployment, maximum performance
Supported platforms	RK1820 / RK1828 / RK3572	Same as left

Toolkit Lite suits rapid on-device Python validation and upper-layer application development. When performance and resources are critical and production is planned, migrate to the C API.

3.4.1 Python Runtime Environment

Toolkit Lite runs on the device side. The currently supported platforms are as follows:

Item	Supported range
Hardware platform	RK1820 / RK1828 / RK3572
Operating system	Debian 10 / 11 / 12 (aarch64)
Python	3.9 / 3.10 / 3.11 / 3.12
Main Python dependencies	`numpy`, `transformers`, `jinja2`

For the detailed version matrix, installation steps, and import methods, see Chapter 1 of the *RKNN3_Toolkit_Lite_Python_API_Reference*.

3.4.2 RKNN3Lite Object Initialization and Release

`RKNN3Lite` is the main Python entry point of Toolkit Lite, and all operations revolve around this object. Construct the object before use and release it after use. The entire lifecycle follows the order: create object -> load model -> initialize runtime -> inference -> release.

Constructing the object

Constructing `RKNN3Lite()` only builds the Python-side wrapper. It does not immediately create a Runtime or Session; the underlying resources are created by the subsequent `init_runtime()` / `init_llm_session()`. Main parameters:

- `llm_mode`: Whether to use LLM / MLLM mode. Defaults to `False`. Set it to `True` for LLM / MLLM to initialize the Session-related code; leave it unset for general models.
- `verbose`: Whether to print detailed logs to the terminal. Defaults to `False`.
- `verbose_file`: The log file path. When specified, logs are also written to this file. Defaults to `None`.

```
from rknn3lite.api import RKNN3Lite

# General model
rknn_lite = RKNN3Lite(verbose=True)
# LLM / MLLM model
rknn_lite = RKNN3Lite(llm_mode=True, verbose=True, verbose_file='./inference.log')
```

Releasing the object

`release()` releases resources. In LLM / MLLM multi-Session scenarios, specify `session_index` to release only one Session while keeping the shared Runtime; if unspecified, all Sessions and the Runtime are released:

```
# Release all resources
rknn_lite.release()
# Release only the specified LLM session (keep the Runtime for other sessions)
rknn_lite.release(session_index=0)
```

> **Note**: Invoke `release()` after all inference is complete, and invoke it exactly once for each `RKNN3Lite` object to avoid resource leaks.

3.4.3 Loading and Initializing the Model

Loading the model

`load_rknn()` loads the converted `.rknn` and `.weight` files. The two paths must be paired; otherwise, loading fails:

```
ret = rknn_lite.load_rknn(model_path='mobilenetv2-12.rknn',
                          weight_path='mobilenetv2-12.weight')
if ret != 0:
    raise RuntimeError(f"load_rknn failed, ret={ret}")
```

Initializing the runtime

`init_runtime()` initializes the runtime environment. The parameters differ between general models and LLM / MLLM models:

- **General models**: Specify `target`, `core_mask`, and `device_id` (obtained through `get_devices_id()` for multiple devices).
- **LLM / MLLM**: Additionally pass sampling and other parameters through `llm_args`, and pass the callback object through `llm_callback`.

```
# General model
device_id = rknn_lite.get_devices_id()
ret = rknn_lite.init_runtime(target='rk1820', core_mask=0x01,
device_id=device_id[0])

# LLM / MLLM
ret = rknn_lite.init_runtime(target='rk1820', core_mask=0xff,
                             llm_args=ARGS, llm_callback=callback)
```

Initialization key points

- `core_mask` must match the number of cores configured in `rknn.config(core_num=...)` during model conversion; otherwise, an error is reported. RK1820 / RK1828 support 0x1~0xff (1-8 cores), and RK3572 supports only 0x1 (single core).
- For LLM / MLLM, the recommended step-by-step flow is: invoke `init_runtime` first, then query the model configuration with `rknn3_query(RKNN3_QUERY_LLM_CONFIG)`, and finally invoke `init_llm_session()`. This reads the model's actual `vocab_size`, `embedding_dim`, `max_ctx_len`, KVCache configuration, and more before creating the Session, avoiding hardcoding. If the application has already determined `llm_args` and the Callback, they can also be passed directly at `init_runtime()` time, and the initialization flow creates the first Session.
- One LLM object can create multiple Sessions on the same Runtime (managed by `session_index`). Each Session independently maintains its conversation state, KVCache, and per-Session configuration, but only one Session can run at a time.
- The sampling parameters of `llm_args` (`top_k` / `top_p` / `temperature` / `repeat_penalty` / `max_new_tokens`, etc.) correspond in meaning to the C API's `rknn3_sampling_params` / `rknn3_llm_param`. See 3.3.2.

3.4.4 Tensor and Memory Management

Toolkit Lite wraps tensor and memory management at the Python layer. By ownership, there are three categories:

- **High-level inputs and outputs**: numpy ndarray inputs and outputs for traditional models. The high-level interfaces automatically handle data preparation and layout conversion, so users do not need to care about the underlying memory.
- **Tensor Memory created by the Runtime**: Device memory allocated through interfaces such as `create_mem()`. The Runtime manages its lifecycle, and users synchronize the data between the CPU and the device through `mem_sync()`.
- **User-provided model runtime memory**: Pre-allocated user memory bound to the model through `set_internal_mem()` / `set_weight_mem()`. It suits advanced scenarios such as memory reuse and cross-process sharing.

Organizing multimodal embeddings

In MLLM deployment, the outputs of each modality encoder model (vision/audio/video) must be converted to the FP16 format expected by Toolkit Lite and then passed into the LLM through the corresponding dedicated type. The three modality types are structurally symmetric. Each

contains the embedding pointer from the encoder model output, the token count, the modality count, and the text placeholder tags:

Modality	Type	Embedding pointer	Token count	Modality count	Placeholder tags
Image	`RKNN3Image`	`image_embed`	`n_image_tokens`	`n_image`	`image_start` / `image_end` / `image_content`
Audio	`RKNN3Audio`	`audio_embed`	`n_audio_tokens`	`n_audio`	`audio_start` / `audio_end` / `audio_content`
Video	`RKNN3Video`	`video_embed`	`n_frame_tokens` / `n_frame_per_video`	`n_video`	`video_start` / `video_end` / `video_content`

Taking an image as an example, convert the vision model output to FP16, populate `RKNN3Image`, and pass it into the LLM:

```
import numpy as np
import ctypes
from rknn3lite.api import RKNN3Image, Float16

# Convert the vision model output to FP16
outputs = np.float16(np.expand_dims(rknn_vision.inference(inputs=[feature])[0], 0))

llm_input = RKNN3Image()
llm_input.image_embed   = outputs.ctypes.data_as(ctypes.POINTER(Float16))
llm_input.n_image_tokens = outputs.shape[1]
llm_input.n_image        = outputs.shape[0]
llm_input.image_width    = 392
llm_input.image_height   = 392
llm_input.image_start    = "<|vision_start|>".encode('utf-8')
llm_input.image_end      = "<|vision_end|>".encode('utf-8')
llm_input.image_content  = "<|image_pad|>".encode('utf-8')

ret, perf = rknn_llm.session_run(inputs=[llm_input], prompt="<image>Describe the
content of this image")
```

Audio and video are populated exactly like images: convert the encoder model output to FP16 and populate the members of the corresponding type (`audio_embed` / `video_embed` , etc.). Multiple modalities can be passed in at the same time; construct the corresponding `RKNN3Image` / `RKNN3Audio` / `RKNN3Video` lists based on the actual inputs and pass them to `session_run(inputs=...)` .

> **Note**: The placeholder tags of each modality must match the special tokens of the model tokenizer. The underlying structure corresponds to the C API's `rknn3_llm_multimodal_tensor` (see 3.3.6). For the complete member definitions, see section 5.5.25 of the *RKNN3_Runtime_C_API_Reference*.

3.4.5 Executing Inference

General models

Use `inference()` for synchronous inference. Both inputs and outputs are numpy ndarrays:

```
outputs = rknn_lite.inference(inputs=[img])
if outputs is None:
    raise RuntimeError("inference failed")
```

Asynchronous `inference_async()` -> `wait()` -> `get_outputs()` is also supported. It suits scenarios that execute other tasks while waiting for the results.

LLM / MLLM

Use `session_run()` for inference. It supports streaming output, and the results are returned asynchronously through `result_callback`:

```
prompt = "Describe how LLM models work."
ret, [n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time] = \
    rknn_llm.session_run(prompt=prompt)
if ret != 0:
    raise RuntimeError(f"session_run failed, ret={ret}")
```

- `prompt` (text) and `embeds` (embedding vectors, 3D: batch × seq × hidden) are mutually exclusive; the latter skips tokenization.
- Multimodal input passes dedicated types such as `RKNN3Image` / `RKNN3Audio` / `RKNN3Video` / `RKNN3AuxTensor` through `inputs`, populated in correspondence with the C API's `rknn3_llm_multimodal_tensor`.
- The return value contains performance statistics `[n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time]`, which can be used for performance analysis.
- Asynchronous inference uses `session_run_async()`, combined with `session_stop()` / `session_pause()` / `session_resume()` to interrupt, pause, and resume.

3.4.6 Complete Example

This section provides two minimal runnable examples, one for the general model flow and one for the LLM flow. For the complete code, see the `rknn3-toolkit-lite/examples` directory in the SDK.

General model example:

Using MobileNetV2 image classification as an example, this demonstrates the complete flow: create object -> load model -> initialize runtime -> query tensor attributes -> inference -> post-processing -> release:

```python
from rknn3lite.api import RKNN3Lite

rknn_lite = RKNN3Lite()

# 1. Load the model
ret = rknn_lite.load_rknn(model_path='mobilenetv2-12.rknn',
                          weight_path='mobilenetv2-12.weight')

# 2. Initialize the runtime (core_mask must match the one used for conversion)
device_id = rknn_lite.get_devices_id()
ret = rknn_lite.init_runtime(target='rk1820', core_mask=0x01,
device_id=device_id[0])

# 3. Query input and output tensor attributes (shape/dtype/layout, etc.)
for attr in rknn_lite.get_inputs_tensor_attr():
    print("input:", attr)
for attr in rknn_lite.get_outputs_tensor_attr():
    print("output:", attr)

# 4. Run inference (input is a pre-processed numpy ndarray)
outputs = rknn_lite.inference(inputs=[img])

# 5. Post-process (Top5 classification) and release
show_top5(outputs[0])
rknn_lite.release()
```

LLM example:

Using Qwen2.5-0.5B Instruct as an example, this demonstrates the complete flow in LLM mode: create object -> load model -> construct callbacks -> initialize runtime -> set chat template -> multi-round inference -> release:

```
import ctypes, numpy as np
from transformers import AutoTokenizer
from rknn3lite.api import (RKNN3Lite, RKLLMCallback,
                           LLMResultCallback, LLMGetEmbedCallback,
LLMTokenizerCallback)

# Preparation: load the tokenizer and embed.bin (vocabulary vectors, shape =
vocab_size × embedding_dim)
tokenizer = AutoTokenizer.from_pretrained(TOKENIZER_PATH)
embeds_data = np.fromfile(EMBED_PATH, dtype=np.float16).reshape(VOCAB_SIZE, -1)

# 1. Create the LLM-mode object and load the model
rknn = RKNN3Lite(llm_mode=True, verbose=True)
rknn.load_rknn(RKNN_MODEL, WEIGHT_MODEL)

# 2. Build callbacks: result (streaming output), tokenizer (text to token), embed
(vocabulary lookup by token)
callback = RKLLMCallback()
callback.result_callback = LLMResultCallback(result_callback)
callback.tokenizer_callback = LLMTokenizerCallback(tokenizer_callback)
callback.embed_callback = LLMGetEmbedCallback(embed_callback)
# userdata passes Python object pointers via ctypes.py_object + ctypes.cast
userdata = ctypes.py_object(tokenizer)
callback.tokenizer_userdata = ctypes.cast(ctypes.pointer(userdata), ctypes.c_void_p)
userdata = ctypes.py_object(embeds_data)
callback.embed_userdata = ctypes.cast(ctypes.pointer(userdata), ctypes.c_void_p)

# 3. Initialize the runtime (llm_args contains sampling parameters; llm_callback
passes the callback object)
ARGS = [{"top_k": 1,
         "top_p": 0.8,
         "temperature": 0.8,
         "repeat_penalty": 1.1,
         "vocab_size": VOCAB_SIZE,
         "special_eos_id": 151645,
         "max_context_len": 1024,
         "keep_history": 0,
         "max_new_tokens": 1024}]
rknn.init_runtime(target='rk1820', core_mask=0xff, llm_args=ARGS,
llm_callback=callback)

# 4. Set the chat template
rknn.set_chat_template(system_prompt, prompt_prefix, prompt_postfix)

# 5. Multi-round inference; the return value contains performance statistics
for prompt in prompts:
    ret, [n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time] = \
        rknn.session_run(prompt=prompt)
    printf_perf(first_token, n_decode_tokens, n_prefill_tokens, llm_start_time,
llm_end_time)

rknn.release()
```

Key points of the examples

- **Callback implementation**: `result_callback` handles branches by `LLMCallState` (`RKLLM_RUN_NORMAL` for normal generation, `RKLLM_RUN_FINISH` for completion, `RKLLM_RUN_ERROR` for errors, `RKLLM_RUN_WAITING` for waiting for UTF-8 characters,

`RKLLM_RUN_STOP` for stop, `RKLLM_RUN_MAX_NEW_TOKEN_REACHED` for reaching the maximum token count, `RKLLM_RUN_PAUSE` / `RKLLM_RUN_RESUME` for pause/resume). In the normal state, it accumulates tokens and prints them with incremental decoding, solving the issue of UTF-8 characters being truncated across tokens. `tokenizer_callback` invokes the HuggingFace tokenizer for tokenization; `embed_callback` looks up the vocabulary vectors from `embeds_data` by token id and copies them to the buffer.

- **userdata passing**: Python objects are wrapped with `ctypes.py_object`, cast to `c_void_p`, and passed into the callback. The callback restores and uses them, preventing them from being collected by the Python GC.
- **Performance statistics**: Combined with the first-token time, `[n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time]` returned by `session_run` can be used to calculate the per-token latency and throughput (tokens/s) of the prefill and generate stages separately.
- **Multi-round conversation**: With `keep_history=1`, the Session automatically keeps the history KVCache for multi-round context; with `keep_history=0`, each round is independent.

> **Note**: The specified RKNN model path must match the paired weight file path. Due to differences in model version, input image, or runtime environment, actual outputs may differ slightly from the examples. For the complete example code, see the `rknn3-toolkit-lite/examples` directory in the SDK.

3.5 RKLLM3 Server Integration

rkllm3-server is a lightweight LLM HTTP Server built on RKNPU3. It provides inference services compatible with the OpenAI API, with the underlying inference acceleration implemented by RKLLM3 Runtime (which wraps RKNN3 Runtime). This section describes its applicable scenarios, deployment methods, and client invocation.

3.5.1 Server Applicable Scenarios

rkllm3-server suits the following scenarios:

- **Service-oriented deployment**: Wraps LLM / MLLM models as an HTTP service that multiple clients invoke through the standard OpenAI API, integrating with the existing application ecosystem.
- **Multimodal inference**: Supports text, image, and audio inputs (video input is not supported yet), covering various model types such as LLMs, VLMs, speech models, and multimodal fusion models.
- **Rapid validation and integration**: No C/Python inference code development required. Invoke it through `curl`, the OpenAI SDK, or any HTTP client. It suits model quality validation and prototype integration.
- **Multiple models and multiple sessions**: Supports loading multiple models in a single process with concurrent sessions (up to 16). It suits scenarios such as multi-model routing and multi-user services.

Positioning difference from native interfaces: rkllm3-server trades some peak performance and flexibility (limited by HTTP and the OpenAI protocol) for ease of use and ecosystem

compatibility. Use the C API or Toolkit Lite when peak performance, fine-grained control (such as KVCache/LoRA/Callback), or embedded integration is required.

3.5.2 Service Deployment Overview

rkllm3-server is deployed as an on-device executable, with the model, vocabulary, sampling configuration, and more specified through command-line parameters. The typical deployment flow is: prepare the model artifacts (`.rknn` / `.weight` / `tokenizer.gguf` / `embed.bin` , etc.) -> start the service process -> client access.

Startup examples

```
# LLM model (standard mode; embed file required)
./rkllm3-server -m qwen2.5-3b.rknn --vocab qwen2.5-3b.tokenizer.gguf \
  --embed qwen2.5-3b.embed.bin --host 0.0.0.0 --port 8080 \
  -c 768 --n_predict 512 --top-k 1 --top-p 0.8 --temp 0.8

# Multimodal model (LLM + Vision)
./rkllm3-server -m Qwen2.5-VL-3B-llm.rknn --model2 Qwen2.5-VL-3B-vision.rknn \
  --vocab Qwen2.5-VL-3B-llm.tokenizer.gguf --embed Qwen2.5-VL-3B-llm.embed.bin \
  --host 0.0.0.0 --port 8080 -c 768 --n_predict 512 \
  --img-start "<|vision_start|>" --img-end "<|vision_end|>" --img-content "
<|image_pad|>" \
  --img-width 392 --img-height 392

# Multimodal model (LLM + Vision + Audio)
./rkllm3-server -m Qwen2.5-Omni-3B-llm.rknn \
  --model2 Qwen2.5-Omni-3B-vision.rknn --model3 Qwen2.5-Omni-3B-audio.rknn \
  --vocab Qwen2.5-Omni-3B-llm.tokenizer.gguf --embed Qwen2.5-Omni-3B-llm.embed.bin \
  --mel-filter mel_128_filters.txt --host 0.0.0.0 --port 8080 \
  --img-start "<|vision_bos|>" --img-end "<|vision_eos|>" --img-content "<|IMAGE|>"
\
  --audio-start "<|audio_bos|>" --audio-end "<|audio_eos|>" --audio-content "
<|AUDIO|>"
```

Key parameter categories

Category	Common parameters	Description
Model paths	`-m` / `--weight` / `--model2` / `--model3`	LLM/vision/audio model and weight paths; for multimodal, use `--model2` / `--model3` to specify the vision/audio models
Vocabulary and embedding	`--vocab` / `--embed` / `--mel-filter`	tokenizer (`.gguf`), vocabulary embedding (`.bin`), audio mel filter
Context and generation length	`-c` / `--n_predict`	Context length (0 = loaded from model), number of tokens to generate (-1 = context size)
Sampling parameters	`--temp` / `--top-k` / `--top-p` / `--repeat-penalty` / `--presence-penalty` / `--frequency-penalty`	Same meaning as the C API sampling parameters; see 3.3.2

Category	Common parameters	Description
Multimodal placeholder tags	`--img-start` / `--img-end` / `--img-content` (and audio/video counterparts)	Must match the special tokens of the model tokenizer
Service configuration	`--host` / `--port` / `--timeout` / `--n-session`	Listen address and port, timeout, concurrent session count (up to 16)
NPU cores	`--core-mask` / `--core-mask2` / `--core-mask3`	NPU core masks for the LLM/vision/audio models; RK182X supports 0x1~0xff, and RK3572 supports only 0x1
Advanced features	`--lora-weight` / `--tie-word-embedding` / `--checkpoint-*` / `--embedding`	LoRA, Tie Word, KVCache Checkpoint, embedding model (`--embedding` identifies an embedding model, which differs from `--embed` specifying the embed.bin path)

> **Note**: The multimodal placeholder tags (such as `--img-start`) must strictly match the special tokens of the model tokenizer; otherwise, the model cannot correctly identify the positions of the modality inputs. `--core-mask` must match the `core_mask` used during model conversion. For the complete parameter list and the multi-model `--params-file` JSON configuration format, see the *RKLLM3_Server_User_Guide_and_API_Reference*.

Library-based integration

In addition to the executable, rkllm3-server also provides `librkllm3-server.so`, which integrates the service capabilities into your own application as a library. The configuration is passed in as a JSON string. It suits embedded integration scenarios that do not want to deploy a standalone service process:

```
#include "rkllm3-server.h"
// Start the server with JSON configuration; parameters are identical to the command
line / params.json
rkllm3_server_start(json_config_str);
```

3.5.3 Client Invocation Methods

rkllm3-server is compatible with the OpenAI API. Clients can invoke it directly with the OpenAI SDK, `curl`, or any HTTP client.

Invoking with the OpenAI SDK

Point `base_url` to the service address to reuse existing OpenAI application code:

```python
from openai import OpenAI

client = OpenAI(base_url="http://<board_ip>:8080/v1", api_key="rkllm3")
response = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[{"role": "user", "content": "Please explain the basic concept of
relativity"}],
    stream=True,   # Streaming output supported
)
for chunk in response:
    print(chunk.choices[0].delta.content or "", end="", flush=True)
```

Multimodal invocation

Image input is passed through `image_url` in `messages`, and audio input is passed through the corresponding field. The format follows the OpenAI multimodal specification:

```python
response = client.chat.completions.create(
    model="Qwen2.5-VL-3B",
    messages=[{"role": "user", "content": [
        {"type": "text", "text": "Describe the content of this image"},
        {"type": "image_url", "image_url": {"url": "file:///path/to/image.jpg"}},
    ]}],
)
```

Invoking with curl

```bash
curl http://<board_ip>:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"Qwen2.5-0.5B","messages":[{"role":"user","content":"Who are
you?"}],"stream":false}'
```

> **Note**: The `model` field must match the `-a/--alias` or model name used at startup. Both streaming (`stream:true`) and non-streaming modes are supported. For supported endpoints and the complete request/response fields, see the *RKLLM3_Server_User_Guide_and_API_Reference*.

3.5.4 Choosing Between Native C/Python Interfaces

The three LLM / MLLM deployment methods are compared below. Choose based on the scenario:

Item	rkllm3-server	C API (Runtime)	Toolkit Lite (Python)
Invocation method	HTTP/OpenAI API	C inference interface	Python inference interface
Ease of use	Highest; standard protocol	Low; requires compilation	High; rapid iteration
Performance	Slightly lower (HTTP overhead)	Optimal	Slightly lower (wrapping overhead)

Item	rkllm3-server	C API (Runtime)	Toolkit Lite (Python)
Feature completeness	Within OpenAI protocol coverage	Most complete	Slightly lower
Typical scenarios	Service-oriented, multi-client, ecosystem integration	Production deployment, peak performance, fine-grained control	Prototype validation, algorithm debugging

Selection advice:

- **Service-oriented / multi-client / rapid integration**: Choose rkllm3-server.
- **Production / peak performance / fine-grained control**: Choose the C API.
- **On-device Python validation / upper-layer applications**: Choose Toolkit Lite.

3.5.5 Server Documentation Entry

For the complete rkllm3-server content (all command-line parameters, multi-model JSON configuration, OpenAI-compatible endpoints and request/response fields, error codes, `librkllm3-server.so` integration API, etc.), see the standalone document *RKLLM3_Server_User_Guide_and_API_Reference*. This guide only covers the core deployment and invocation flow.

3.6 RK182X Multi-Card LLM Deployment

This section describes how to deploy an LLM split into consecutive Transformer blocks across multiple RK1820 / RK1828 accelerator cards, following `rknn3-model-zoo/examples/multicard`. The solution creates multiple RKNN3 contexts and sessions on the host and connects them into a pipeline with host-side threads and queues. The Runtime does not automatically combine multiple contexts into one multi-card model.

Multi-card deployment and NPU multi-core configuration are different levels:

Configuration	Scope	Description
`rknn3_config.run_core_mask` / `core_mask`	Within one card	Selects the NPU cores used by a stage on one RK182X card; it must match the core configuration used during conversion
`stage_count`	Number of model segments	One stage maps to one RK182X card and must match the number of exported segments
`device_id`	Physical device selection	Binds a stage to a specific RK182X device
`bucket_size`	Host pipeline granularity	Number of tokens forwarded from one stage to the next per prefill step; 128 is a reasonable initial value

RK3572 is a single-SoC NPU and does not use the multi-device `device_id` flow described in this section. Use the normal single-model Session deployment on RK3572.

3.6.1 Multi-Card Topology and Runtime Boundaries

Segment boundaries, export parameters, and artifact naming are defined in Section 2.2.6. This section only describes how the exported `model_segN.rknn` files are mapped to multiple RK182X cards and used for inference: `stageN` uses the corresponding `segN` model and `device_id[N]`, while host-side threads, queues, and callbacks form the pipeline. The Runtime does not automatically combine multiple contexts into one multi-card model.

The logical data flow is shown below. The arrows represent host-side hidden-state forwarding:

```
prompt/token
    |
    v
stage0: seg0
    |  output_callback -> queue
    v
stage1: seg1
    |  output_callback -> queue
    v
...
    |
    v
stageN-1: segN-1
    |
    v  result_callback
logits -> sampling -> next token
```

The multi-card pipeline is model parallelism, not data parallelism: it does not replicate the complete model on every card or automatically broadcast one batch of requests to all cards. Intermediate stages only forward hidden states; the last stage handles logits, sampling, and generation state. The current demo uses prefill bucketing to improve pipeline utilization.

3.6.2 Runtime Parameters and Resource Constraints

Follow Section 2.2.6 for offline segmented export, RKNN building, file naming, and memory planning. At runtime, the multi-card application only loads these segments onto the specified devices. The application must preserve the following correspondences:

- `stage_count`, the actual number of segments, and the number of RK182X cards used for inference must be consistent.
- `device_id[i]`, the model path, and `stage[i]` must use the same index order; device order must not be mixed with the `segN` order.
- `run_core_mask` must match the core configuration used to build the segment. It controls only the NPU cores within one card and does not add more cards.
- All stages must use consistent `max_context_len`, vocabulary information, and session parameters. `bucket_size` only controls the host-side prefill forwarding granularity.

The Runtime does not re-segment the model at startup or automatically migrate a stage according to memory or compute load. If a segment runs out of memory or the load is unbalanced, return to Section 2.2.6 and adjust the number of segments, the consecutive block split, or the context length before rebuilding. `segment_plan.json` and `model_report.html` are used for offline planning and locating the resource bottleneck of a particular stage.

3.6.3 Runtime Initialization for Each Stage

Each card must independently initialize its context, model, output tensors, session, and callbacks. The recommended order is as follows:

Order	Call or action	Key requirement
1	`rknn3_find_devices()` or read external `device_id` values	The number of devices must be at least `stage_count`; external IDs are ordered as stage0, stage1, ...
2	`rknn3_init()`	Bind the context to the target card through `rknn3_init_extend.device_id`
3	`rknn3_load_model_from_path()`	Load the `.rknn` and `.weight` files for that stage
4	`rknn3_model_init()`	Use the `run_core_mask` matching the conversion configuration for the stage
5	Register an optional custom-op plugin	Register it after `model_init` on the corresponding context; in multi-card mode, register it separately for every stage context
6	Query LLM configuration and output attributes; allocate output tensors	Intermediate stages need output buffers for `output_callback`
7	`rknn3_session_init()`	Use consistent `max_context_len`, vocabulary information, and `logits_name` across stages
8	`rknn3_session_set_callback()`	stage0 needs tokenizer/embed; non-last stages need `output_callback`; every stage needs `input_callback` when external caches are used

The core initialization structure is shown below. Add error handling and output-tensor allocation as required by the actual application:

```
for (size_t i = 0; i < stage_count; ++i) {
    rknn3_init_extend ext;
    memset(&ext, 0, sizeof(ext));
    ext.device_id = device_ids[i];
    rknn3_init(&ctx[i], &ext);

    rknn3_load_model_from_path(ctx[i], model_paths[i], weight_paths[i]);

    rknn3_config cfg;
    memset(&cfg, 0, sizeof(cfg));
    cfg.run_core_mask = run_core_mask;
    rknn3_model_init(ctx[i], &cfg);

    if (custom_op_path != NULL) {
        rknn3_register_custom_ops_plugins(
            ctx[i], custom_op_path, strlen(custom_op_path));
    }

    session[i] = rknn3_session_init(ctx[i], &llm_param, 1);
    rknn3_session_set_callback(session[i], &callback[i]);
}
```

The application may accept only the stage0 model path and derive the remaining paths from the `_seg0` suffix, but it must verify that all segment files are available before initialization. Do not continue initialization when `stage_count`, the number of device IDs, and the actual segment count are inconsistent.

3.6.4 Host-Side Data Forwarding and Inference Control

Intermediate data between cards is forwarded through host-side callbacks. A typical implementation includes the following steps:

1. stage0 runs with `PROMPT` or `TOKEN` input; `tokenizer_callback` converts text to tokens and `embed_callback` looks up FP16 embeddings from `embed.bin`.
2. The `output_callback` of every non-last stage copies hidden states from the Runtime output buffer into a queue item containing tensor attributes and the token count. Do not pass a pointer to a Runtime-owned output buffer directly to an asynchronous worker and then reuse that buffer.
3. Stages from stage1 through stageN-1 each have a worker that consumes a batch from the upstream queue and invokes `rknn3_session_run()` with `RKNN3_LLM_INPUT_EMBED` as the current-stage input.
4. Each batch carries its actual token count. Downstream stages calculate the embedding byte count from that token count instead of assuming that every callback uses the fixed `bucket_size`.
5. Queues need `producer_done`, `failed`, and `close` states. When any stage fails, wake up and close downstream queues; join all workers in the main thread before releasing resources.

A pipeline step can use the following LLM inference parameters:

```
rknn3_llm_infer_param infer_param;
memset(&infer_param, 0, sizeof(infer_param));
infer_param.keep_history = 1;
infer_param.max_new_tokens = 1;
infer_param.prefill_only = true;
infer_param.disable_sampling = true;
```

During prefill, stage0 may split the prompt into multiple buckets. Intermediate stages only propagate hidden states and do not sample. The last stage enables sampling only for the last bucket of the current round, preventing repeated token generation for every bucket. During decode, the application sends the previous output token back to stage0; each pipeline step generates one token. In single-card mode, stage0 is also the last stage and should set `disable_sampling` to false.

Each stage has its own KV cache, so prefill and decode must traverse all stages in the same logical order. When clearing the context, call `rknn3_session_clear_kvcache()` for every session. Release resources in the order: destroy sessions, release tensors and memory mappings, then destroy contexts.

3.6.5 External Inputs and Callbacks

External RoPE cache

The multicard demos for Qwen3.5 and Gemma4 can provide an external RoPE cache through safetensors. The application generally only needs to `mmap` one copy on the host and, for every stage:

1. Query the stage input attributes and locate the `rope_cos_cache` / `rope_sin_cache` indices. If the model has other external inputs such as `per_layer_inputs`, handle them according to that model's export convention.
2. Register `input_callback` and pass these indices to the callback.
3. In the callback, copy the cache slice for the current position according to `LLMInputCallbackParam.pos`.

Tensor names and dimension layouts must follow the exported input attributes. The demo currently recognizes Qwen3.5 `rope_cos_cache` / `rope_sin_cache` and Gemma4 numbered RoPE caches. Do not register this callback for models that do not use an external RoPE cache.

Callback responsibilities

Callback	Responsibility in the multicard demo
`tokenizer_callback`	Converts the prompt to token ids when text input is required
`embed_callback`	Maps token ids to embeddings in stage0 and records the token count for the current round
`output_callback`	Copies hidden states and enqueues them for non-last stages; the last stage normally does not forward output
`result_callback`	Outputs tokens and updates `next_token` and the generation state in the last stage
`input_callback`	Fills external RoPE or other per-layer external inputs

3.6.6 Command-Line Deployment Example

The multicard demo only requires the model and weight paths for seg0; it can derive the remaining segments from the `_seg0` suffix. The following example uses a four-segment LLM model:

```
adb shell
cd /data/rknn_multicard_demo
export LD_LIBRARY_PATH=./lib
./rknn_multicard_demo --model /data/models/multicard/model_seg0.rknn --weight
/data/models/multicard/model_seg0.weight --vocab
/data/models/multicard/model.tokenizer.gguf --embed
/data/models/multicard/model.embed.bin --ctx-size 4096 --core-mask 0xff --stage-
count 4 --bucket-size 128 --predict 128 --device-id id0#id1#id2#id3
```

Key constraints are as follows:

- `stage_count` must equal the number of exported segments and the number of RK182X cards used. This example requires seg0 through seg3 and four available devices.

- `device-id` values are separated with `#` and ordered from stage0 through stage3. If omitted, the demo calls `rknn3_find_devices()` and automatically assigns the first `stage_count` devices.
- `core-mask` controls only the NPU cores within each card; it does not indicate the number of cards.
- If the model uses an external RoPE cache, add the `rope-tensor` parameter. If the model or other graph nodes require a runtime extension, add `custom-op-path`; the extension is registered separately for each stage context.
- In performance mode, the input and output lengths must satisfy `input_tokens + output_tokens - 1 <= max_context_len` for every stage.

3.7 MatMul C API Usage

> **Note: RK182X is not supported yet.**

The MatMul C API executes matrix multiplication directly from an application: A is an M×K matrix, B is a K×N matrix, and C is an M×N matrix. It is intended for applications that prepare their own inputs and weights and invoke NPU matrix multiplication independently. No RKNN model generation or loading, or Session creation, is required. Create a parent `rknn3_context` and a separate `rknn3_matmul_context`, then execute with `rknn3_matmul_run()`.

3.7.1 Interfaces and Runtime Environment

Include `rknn3_api.h` and `rknn3_matmul_api.h`, and use matching SDK headers and libraries that provide the MatMul interfaces. RK3572 uses the native library. See 3.1 for library deployment; check actual API return values for type, shape, and resource requirements.

Zero-initialize input structures and reserved fields. A MatMul handle uses one NPU core; select a single core with `core_mask` rather than combining multiple core masks.

`desc.type`	A	B	C	Quantization parameters
`RKNN3_MATMUL_W16A16`	FP16	FP16	FP16	None
`RKNN3_MATMUL_W4A16`	FP16	INT4	FP16	B only, grouped along K
`RKNN3_MATMUL_W6A16`	FP16	INT6	FP16	B only, grouped along K
`RKNN3_MATMUL_W8A16`	FP16	INT8	FP16	B only, grouped along K
`RKNN3_MATMUL_W8A8_PER_CHANNEL`	INT8	INT8	INT8	Per-tensor A/C; per-output-channel B
`RKNN3_MATMUL_W8A8_PER_LAYER`	INT8	INT8	INT8	Per-tensor A/C; per-layer B for each batch
`RKNN3_MATMUL_W8A8O32`	INT8	INT8	INT32	None; K must not exceed 4096

W16A16 supports both ordinary weight B and two feature inputs; see 3.7.3 for the latter.

Logical B for W4A16/W6A16 uses unpacked storage: each weight occupies one `int8_t`, with values in `[-8,7]` / `[-32,31]`, respectively. Do not pass bit-packed data as logical B. Set `group_size` to 32, 64, or 128 for grouped quantization types, and to 0 for other types.

3.7.2 Data Layout, Memory, and Call Flow

Mathematical dimensions and contiguous logical storage are listed below. For a single batch, the indices are `k*M+m` for A, `k*N+n` for B, and `n*M+m` for C. Ordinary row-major A stored at `m*K+k` cannot be passed directly as logical input.

Tensor	Canonical logical shape	Contiguous NCHW index
A	`[A_batch,K,1,M]`	`(batch*K+k)*M+m`
B	`[B_batch,K,1,N]`	`(batch*K+k)*N+n`
C	`[C_batch,N,1,M]`	`(batch*N+n)*M+m`

`A_batch` / `B_batch` must both be positive. `C_batch=max(A_batch,B_batch)` must be divisible by both. The example below uses a single batch.

`rknn3_matmul_query_infos()` returns A/B/C attributes for each shape, including layout, physical strides, size in bytes, alignment, and core ID. Use the queried NPU attributes unchanged. Zero-initialize logical attributes and fill in `n_dims=4`, shape, dtype, NCHW layout, and contiguous data size in bytes. NPU A/C use NC1HWC2. Ordinary B specified with `desc.B_layout=NCHW` is queried as the packed weight layout O1I1HWI2O2.

The following diagram shows the typical MatMul C API call flow. See the table below for details.

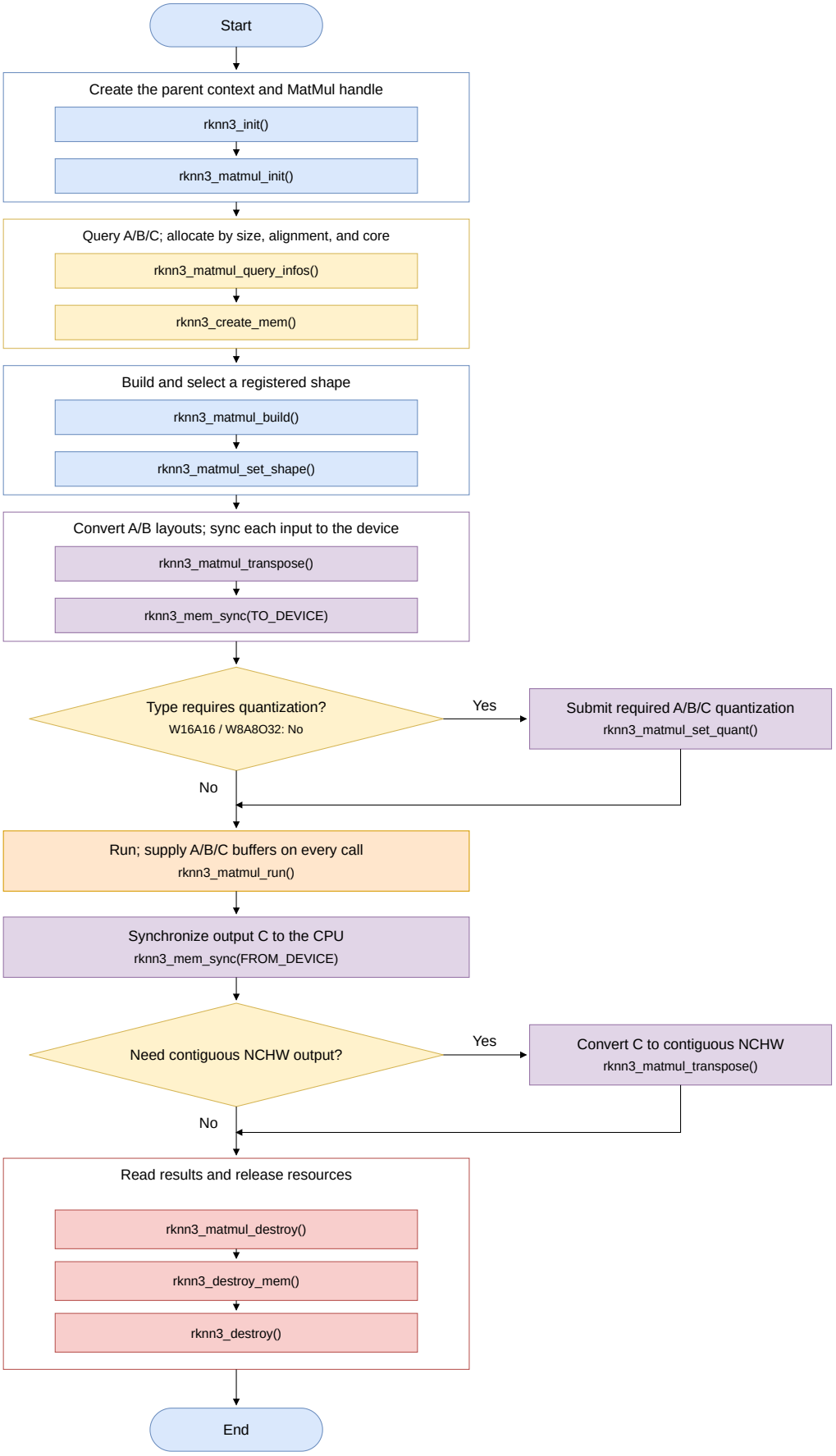


Figure 3-7 MatMul C API call flow

Step	Interfaces and operations
1. Initialize	Create the parent context with `rknn3_init()`; register one or more shapes with `rknn3_matmul_init()`.
2. Query and allocate	Call `rknn3_matmul_query_infos()`; allocate A/B/C with `rknn3_create_mem()` using each attribute's size, alignment, and core_id.
3. Build and select	Call `rknn3_matmul_build()`, then `rknn3_matmul_set_shape()` with an index into the registered shape array.
4. Prepare inputs	Convert A/B to queried layouts with `rknn3_matmul_transpose()`; synchronize each with `rknn3_mem_sync(..., TO_DEVICE)`.
5. Submit quantization	Call `rknn3_matmul_set_quant()` for types requiring quantization; see 3.7.4.
6. Execute	Call `rknn3_matmul_run(matmul, a_mem, b_mem, c_mem)`, supplying A/B/C on every invocation.
7. Read results	After success, synchronize C to the CPU; use `rknn3_matmul_transpose()` if contiguous NCHW output is required.
8. Release	Release A/B/C buffers and then destroy the parent context only after `rknn3_matmul_destroy()` succeeds. Retain these resources if destruction fails; see 3.7.6.

Do not allocate solely from M×K, K×N, or M×N: NPU layouts include padding. Align `phys_addr + offset` to the queried alignment and ensure remaining capacity from offset is at least the attribute size. CPU and NPU access must use the same offset. `transpose()` converts layouts and does not replace explicit memory synchronization. For cacheable buffers, synchronize CPU writes to the device and device outputs to the CPU before reading. C is an output buffer; its data does not need to be filled before execution.

3.7.3 W16A16 Core Call Example

The following fragment shows the successful call flow for M=16, K=128, and N=64. Memory allocation, logical attributes and data preparation, return-value checks, and failure cleanup are omitted; it is not a standalone program. An application must check every return value and proceed only after success. `am/bm/cm` are NPU buffers allocated and aligned according to the query. `aa/ba/ca` and `al/bl/cl` are contiguous NCHW logical attributes and memory descriptors, respectively; see 3.7.2 for preparation rules. W16A16 requires no quantization parameters.

```
rknn3_context ctx = 0;
rknn3_matmul_context mm = 0;
rknn3_matmul_desc desc = {0};
rknn3_matmul_info info = {0};

desc.M = 16;
desc.K = 128;
desc.N = 64;
desc.A_batch = 1;
desc.B_batch = 1;
desc.type = RKNN3_MATMUL_W16A16;
desc.A_layout = RKNN3_TENSOR_NCHW;
desc.B_layout = RKNN3_TENSOR_NCHW;
desc.C_layout = RKNN3_TENSOR_NCHW;
desc.core_mask = RKNN3_NPU_CORE_0;

rknn3_init(&ctx, NULL);
rknn3_matmul_init(ctx, &desc, 1, &mm);
rknn3_matmul_query_infos(mm, &info, 1);

/* Allocate aligned am/bm/cm and prepare logical aa/ba/ca and al/bl/cl. */
rknn3_matmul_build(mm, NULL);
rknn3_matmul_set_shape(mm, 0);

rknn3_matmul_transpose(mm, &aa, &al, &info.A, am);
rknn3_mem_sync(ctx, am, RKNN3_MEMORY_SYNC_TO_DEVICE);

rknn3_matmul_transpose(mm, &ba, &bl, &info.B, bm);
rknn3_mem_sync(ctx, bm, RKNN3_MEMORY_SYNC_TO_DEVICE);

/* W16A16 requires no set_quant call. */
rknn3_matmul_run(mm, am, bm, cm);

rknn3_mem_sync(ctx, cm, RKNN3_MEMORY_SYNC_FROM_DEVICE);
rknn3_matmul_transpose(mm, &info.C, cm, &ca, &cl);

/* Read logical C; destroy the MatMul handle before releasing its buffers. */
if (rknn3_matmul_destroy(mm) == RKNN3_SUCCESS) {
    rknn3_destroy_mem(ctx, am);
    rknn3_destroy_mem(ctx, bm);
    rknn3_destroy_mem(ctx, cm);
    rknn3_destroy(ctx);
}
```

Compile applications with `rknn3_api.h` and `rknn3_matmul_api.h` from the public SDK. Link `librknn3_api.so` and `librknn3_api_native.so` for RK3572. See 3.1 for program and library deployment and library search path configuration.

W16A16 with two feature inputs (A/B)

This mode still uses `RKNN3_MATMUL_W16A16`; A/B/C are FP16 and require no `set_quant()` call. Set `desc.B_layout` to `RKNN3_TENSOR_NC1HWC2` before initialization. Keep A_layout/C_layout as NCHW to describe contiguous logical input/output. The queried NPU layouts of A, B, and C are all NC1HWC2, while ordinary weight B is queried as O1I1HWI2O2.

Use this mode when B is also a runtime feature input. Keep the dimensions, type, and core settings from the preceding example, and use the configuration and input preparation below.

Return-value checks and memory preparation are again omitted. Output synchronization, conversion, and resource release follow the preceding example.

```
/* Set these fields before init; keep the other W16A16 fields unchanged. */
desc.A_layout = RKNN3_TENSOR_NCHW;
desc.C_layout = RKNN3_TENSOR_NCHW;
desc.B_layout = RKNN3_TENSOR_NC1HWC2;

rknn3_matmul_init(ctx, &desc, 1, &mm);
rknn3_matmul_query_infos(mm, &info, 1);

/* Allocate A/B/C using this query; build and select shape 0. */
rknn3_matmul_build(mm, NULL);
rknn3_matmul_set_shape(mm, 0);

/* Prepare logical aa/ba/al/bl; convert both inputs to feature layouts. */
rknn3_matmul_transpose(mm, &aa, &al, &info.A, am);
rknn3_mem_sync(ctx, am, RKNN3_MEMORY_SYNC_TO_DEVICE);

rknn3_matmul_transpose(mm, &ba, &bl, &info.B, bm);
rknn3_mem_sync(ctx, bm, RKNN3_MEMORY_SYNC_TO_DEVICE);

rknn3_matmul_run(mm, am, bm, cm);
```

Prepare logical B as contiguous NCHW with shape `[B_batch,K,1,N]`, set `ba.layout` to NCHW, and use the new query's `info.B` as the conversion destination. Do not reuse packed weight B attributes or allocation sizes. If A/B are already in NC1HWC2 and their shape, dtype, strides, capacity, alignment, and core satisfy the queried requirements, pass the corresponding NPU buffers directly to omit logical layout conversion. Cacheable inputs written by the CPU still require synchronization to the device.

Two feature inputs are currently supported only for single-shape W16A16. Supply the current A/B/C on every run; after updating B, complete the required layout conversion and synchronization before execution. This mode cannot use the fixed-B multiple-shape flow in 3.7.5.

3.7.4 Preparing Quantization Parameters

Copy the requirements from `info.A/B/C.quant_info`, then fill in scale/zp arrays and their byte sizes. Queries return formats and counts, not actual quantization values. Store the two arrays separately and contiguously, without interleaving or NPU layout padding; `scales[i]` and `zero_points[i]` describe the same quantization unit.

Let `G=ceil(K/group_size)`. Array organization is as follows:

Type / Tensor	Logical shape of each array	Parameter index	Scale element
W4A16/W6A16/W8A16, B	`[B_batch,G,N]`	`(batch*G+k/group_size)*N+n`	FP16 bit pattern, 2 bytes
W8A8 per-channel, B	`[B_batch,N]`	`batch*N+n`	FP32, 4 bytes
W8A8 per-layer, B	`[B_batch]`	`batch`	FP32, 4 bytes
W8A8, A/C separately	`[1]`	`0`, covering all elements and batches	FP32, 4 bytes

All zero points use unpacked `int32_t` storage, 4 bytes each, even when the queried `zero_point_dtype` is INT4/INT8. `scales_size` / `zero_points_size` are byte counts: `num_quant_params * scale element size` and `num_quant_params * sizeof(int32_t)`, respectively. Scales must be finite and positive. Supply an all-zero zp array even when every zero point is 0.

For example, grouped B with `B_batch=1,K=64,N=64,group_size=32` requires 128 elements in each array: 256 bytes for FP16 scales and 512 bytes for int32 zero points. Indices 0..63 represent the 64 output channels of the first group, and 64..127 represent the second group.

- W4A16/W6A16/W8A16 call `set_quant(matmul, NULL, &qb, NULL)` to submit B only.
- W8A8 per-channel / per-layer call `set_quant(matmul, &qa, &qb, &qc)` to submit A/B/C.
- W16A16/W8A8O32 do not call `set_quant()`.

Submit after build and set_shape, and before run. B layout conversion and quantization submission can occur in either order. If K is not divisible by group_size, the final partial group still needs N parameter pairs. Use the queried `num_quant_params` as the required count. See the *RKNN3_Runtime_C_API_Reference* for detailed value constraints.

3.7.5 Multiple Shapes Sharing Fixed B

Register multiple shapes in one handle and select them with `set_shape(matmul, shape_id)`. Only M and A_batch may differ. The type, core_mask, B_batch, K, N, group_size, B_layout, and queried B size and layout must match. Use ordinary weight B (`desc.B_layout=NCHW`). Feature B (NC1HWC2) is available only for single-shape W16A16.

For example, fix K=128 and N=64, and register M=1 and M=16:

1. Pass both descriptions to one init call, query `infos[2]`, and build once.
2. Convert and synchronize B once. Preallocate A/C using the maximum size across shapes while satisfying each shape's alignment and core requirements.
3. After each set_shape, prepare and synchronize current A using `infos[shape_id].A`, then run with the shared fixed B.
4. Synchronize and process C using the current shape. Complete output conversion or saving before selecting another shape.

For grouped A16 quantization, submit B successfully once for the entire handle. For W8A8, first select each shape and submit its A/B/C parameters separately. B quantization values must match across shapes; A/C values may differ. Once prepared, switching shapes requires no resubmission. W16A16/W8A8O32 require no quantization setup.

3.7.6 Return Values, Resource Lifetime, and Further Documentation

Check every return value. Do not build or execute a type/shape rejected during initialization. `run()` is synchronous; process C only after success and do not treat C as valid on failure. Serialize shape selection, conversion, quantization, execution, and output conversion on the same handle.

If `RKNN3_ERR_MATMUL_COMPLETION_UNKNOWN` is returned, retain the MatMul handle, parent context, and A/B/C. Do not modify, reuse, or release these buffers. Release related resources only

after `rknn3_matmul_destroy()` succeeds. If destroy fails, do not release A/B/C or the parent context. An integrated application must retain these resources until handle destruction succeeds.

See the MatMul sections of *RKNN3_Runtime_C_API_Reference* for details.

4. Advanced Features and Extensions

This chapter describes the advanced features and extension capabilities of RKNN3, including multi-core NPU, dynamic shape, and memory management at the Runtime layer; multiple input types, callbacks, Function Calling, KVCache / LoRA, and more at the LLM / MLLM layer; and extension and security capabilities such as post-process plugins, custom operators, and model encryption. These features target advanced deployment and optimization scenarios; you can skip this chapter for basic deployment.

4.1 Runtime Advanced Features

4.1.1 NPU Multi-Core

Applicable Scenarios

The RK1820 / RK1828 coprocessor NPU contains 8 cores (RK3572 has a single core). Multi-core operation splits computation across multiple cores for parallel execution. It suits models with computationally heavy layers (such as large-parameter LLMs and high-resolution vision models). For small networks, operator switching between cores requires CPU involvement and may degrade performance; single-core operation is recommended.

Usage

Multi-core operation requires coordinated configuration on both the model conversion side and the on-device inference side:

- **Conversion side**: `rknn.config(core_num=N)` specifies the number of cores the model occupies. `core_num=0` is automatic mode (models loaded with `load_llm` use 8 cores; all others use 1 core).
- **Inference side**: The core mask must match the `core_num` used at conversion time. Set it through `rknn3_config.run_core_mask` (C) or `init_runtime(core_mask=)` (Python). The range is 0x1~0xff.

```c
/* Query the core count and build the mask */
uint32_t core_num = 0;
rknn3_query(ctx, RKNN3_QUERY_CORE_NUMBER, &core_num, sizeof(core_num));
uint32_t core_mask = 0;
for (uint32_t i = 0; i < core_num; i++) core_mask |= (1 << i);

rknn3_config config;
memset(&config, 0, sizeof(rknn3_config));
config.run_core_mask = core_mask;
rknn3_model_init(ctx, &config);
```

> **Note**: For CNN models, `core_num=1` is recommended; set Transformer-class models as needed. Invoke `RKNN3_QUERY_CORE_MEM_SIZE` to query per-core memory usage and assist memory planning. RK3572 supports single core only (`core_mask=0x1`).
>
> **Multi-card distinction**: `core_num` / `core_mask` only determines how many NPU cores are used within one card; the number of cards in a multi-card model is determined by the number of segments and `stage_count`, and card selection is determined by `device_id`. Setting `core_mask` to 0xff does not automatically enable a second RK182X card.

4.1.2 Dynamic Shape

Applicable Scenarios

Dynamic shape allows the same model to switch input sizes at runtime. One model replaces multiple static models, saving Flash/DDR footprint. It suits scenarios with varying input sizes (such as variable-resolution detection and multi-tier image classification).

Prerequisites

- **The model must support dynamic shape**: If the original model is not dynamic-shape, RKNN3-Toolkit can extend it into a dynamic-shape model. However, the model must not contain operators or subgraph structures that restrict dynamic shape (for example, constant shapes cannot change); otherwise the conversion fails with an error. If extension is not supported, modify the model structure according to the error message and retrain. Using a model that is natively dynamic-shape is recommended.
- **Limited number of shapes**: Due to NPU hardware characteristics, a dynamic-shape model does not support arbitrary input shape changes. Users must explicitly enumerate a finite set of input shapes at conversion time. For multi-input models, each input must have the same number of shapes, and the shapes must correspond one-to-one in order; cross combinations are not supported.

Usage

- **Conversion side**: `rknn.config(dynamic_input=[...])` configures multiple sets of input shapes, each described by `rknn3_shape_info`. During quantization, the shape set with the largest total size is used. All shapes share one set of quantization parameters.

```
dynamic_shapes = [
    [[1, 3, 224, 224]],
    [[1, 3, 192, 192]],
    [[1, 3, 160, 160]],
]
rknn.config(mean_values=[[0,0,0]], std_values=[[255,255,255]],
            dynamic_input=dynamic_shapes)
```

- **Inference side**: The model uses Shape ID 0 by default. To switch, release the old memory first, then set the new ID, reallocate, and synchronize:

```
rknn3_destroy_mem(ctx, old_mem);              /* Release the old shape memory */
rknn3_set_shape(ctx, shape_id);               /* Switch the shape */
rknn3_create_mem(ctx, ...);                   /* Allocate memory for the new shape
*/
rknn3_mem_sync(ctx, mem, RKNN3_MEMORY_SYNC_TO_DEVICE);
```

Invoke `RKNN3_QUERY_DYNAMIC_SHAPE_CONFIG` / `RKNN3_QUERY_DYNAMIC_SHAPE_INFO` to query the list of supported shapes.

> **Limitation**: Runtime shape switching must follow the order "release old memory -> `rknn3_set_shape()` -> reallocate memory -> synchronize". Dynamic-shape models do not support weight sharing or context reuse (`rknn3_dup_context()`). Extreme size combinations may affect accuracy because all shapes share one set of quantization parameters.

4.1.3 Cacheable Memory Coherence

Applicable Scenarios

The CPU and NPU have independent caches. When they exchange data through shared memory with cache attributes, missing synchronization causes data incoherence and inference errors. The cacheable memory coherence mechanism uses `rknn3_mem_sync()` to explicitly flush the cache when the data flow direction changes, ensuring coherence.

Usage

Specify memory attributes through the flags of `rknn3_create_mem()`: `RKNN3_FLAG_MEMORY_CACHEABLE` (with cache) or `RKNN3_FLAG_MEMORY_NON_CACHEABLE` (without cache). For cacheable memory, synchronize according to the data flow direction when data passes between the CPU and NPU:

Data Flow Direction	Sync Mode	Timing
CPU write -> NPU read	`RKNN3_MEMORY_SYNC_TO_DEVICE`	Before inference (input)
NPU write -> CPU read	`RKNN3_MEMORY_SYNC_FROM_DEVICE`	After inference (output)

Usage Recommendations:

1. When the data flow direction is fixed, use one-way synchronization. This reduces redundant cache flush operations and improves inference performance.
2. When sending input data to the device, invoke `rknn3_mem_sync(..., RKNN3_MEMORY_SYNC_TO_DEVICE)`.
3. When reading output data from the device, invoke `rknn3_mem_sync(..., RKNN3_MEMORY_SYNC_FROM_DEVICE)`.

```
rknn3_tensor_mem* mem = rknn3_create_mem(ctx, size, 0, RKNN3_FLAG_MEMORY_CACHEABLE);
/* After filling in input data, synchronize to the device before inference */
rknn3_mem_sync(ctx, mem, RKNN3_MEMORY_SYNC_TO_DEVICE);
rknn3_run(ctx, ...);
/* After inference, synchronize output back to the CPU for reading */
rknn3_mem_sync(ctx, mem, RKNN3_MEMORY_SYNC_FROM_DEVICE);
```

4.1.4 Internal Memory Reuse

Applicable Scenarios

When deploying multimodal models (such as VLMs), the vision encoder and the LLM often execute serially. Reusing the same Internal memory between them significantly reduces overall on-device memory usage and effectively eases the bottleneck of limited memory resources.

The core idea is "time slicing + spatial reuse": after the vision model finishes inference and passes image features to the LLM, the Internal memory it occupied is released and reused for the LLM's intermediate computation. This avoids allocating separate memory spaces for both models.

This strategy is especially suitable for the following scenarios:

- **Multi-model pipeline deployment**: Combined inference tasks such as VLM and speech recognition + large language model (LLM).
- **Memory-constrained edge devices**: Devices that need to run multi-stage models within limited memory.
- **Multimodal interaction applications**: Applications that must maximize memory utilization while maintaining response speed.

Usage

Prerequisite: set `rknn3_config.user_mem_internal=1` so that internal memory is allocated by the user. Procedure (taking two serially executed models A and B as an example):

1. Invoke `RKNN3_QUERY_CORE_NUMBER` on both models to query the core counts, then invoke `RKNN3_QUERY_CORE_MEM_SIZE` to query the internal memory size of each core (allocate a `rknn3_core_mem_size` array sized by the core count to receive the results).
2. Iterate over the cores of both models, match them by `core_id` to find the parts running on the same physical core, and use `max(A_internal, B_internal)` as the shared size.
3. Total memory blocks = A core count + B core count - overlapping core count.
4. For reused cores, B's internal_mems pointer points directly to the memory already allocated by A; for non-reused cores, allocate separately (allocate uniformly with A's ctx for later unified release and to avoid double release).
5. Invoke `rknn3_set_internal_mem()` to set the memory for both models. The temporary pointer arrays can then be freed; the internal memory blocks themselves are released by `rknn3_destroy_mem()`.

The following is sample code implementing Internal memory reuse in a VLM:

```
...
int ret = -1;

uint32_t core_num_vision = 0;
uint32_t core_num_llm = 0;
// Get the core_num of the vision model
ret = rknn3_query(app_ctx->vision.rknn_ctx, RKNN3_QUERY_CORE_NUMBER,
&core_num_vision, sizeof(core_num_vision));
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_query failed! ret=%d\n", ret);
    return ret;
}
// Get the core_num of the LLM model
ret = rknn3_query(app_ctx->llm.rknn_ctx, RKNN3_QUERY_CORE_NUMBER, &core_num_llm,
sizeof(core_num_llm));
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_query failed! ret=%d\n", ret);
    return ret;
}

rknn3_core_mem_size* core_mem_sizes_vision =
(rknn3_core_mem_size*)malloc(sizeof(rknn3_core_mem_size) * core_num_vision);
if (!core_mem_sizes_vision) {
    printf("Failed to allocate memory for core_mem_sizes_vision\n");
    return ret;
}
rknn3_core_mem_size* core_mem_sizes_llm =
(rknn3_core_mem_size*)malloc(sizeof(rknn3_core_mem_size) * core_num_llm);
if (!core_mem_sizes_llm) {
    printf("Failed to allocate memory for core_mem_sizes_llm\n");
    return ret;
}
// Get the weight and internal memory usage of the vision model on each NPU core
ret = rknn3_query(app_ctx->vision.rknn_ctx, RKNN3_QUERY_CORE_MEM_SIZE,
core_mem_sizes_vision, sizeof(rknn3_core_mem_size) * core_num_vision);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_query core memory size failed! ret=%d\n", ret);
    return ret;
}
// Get the weight and internal memory usage of the LLM model on each NPU core
ret = rknn3_query(app_ctx->llm.rknn_ctx, RKNN3_QUERY_CORE_MEM_SIZE,
core_mem_sizes_llm, sizeof(rknn3_core_mem_size) * core_num_llm);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_query core memory size failed! ret=%d\n", ret);
    return ret;
}

int llm_to_vision[core_num_llm];
for (int i = 0; i < core_num_llm; i++)  { llm_to_vision[i] = -1; }

// Iterate over vision and LLM cores to find cores running on the same physical NPU
core
int core_num_same = 0;
for (int i = 0; i < core_num_vision; i++) {
    for (int j = 0; j < core_num_llm; j++) {
        if (core_mem_sizes_vision[i].core_id == core_mem_sizes_llm[j].core_id) {
            uint64_t internal_size =
std::max(core_mem_sizes_vision[i].internal_size,
core_mem_sizes_llm[j].internal_size);
            core_mem_sizes_vision[i].internal_size = internal_size;
```

```
            core_mem_sizes_llm[j].internal_size = internal_size;
            core_num_same ++;
            llm_to_vision[j] = i;
            break;
        }
    }
}

// Allocate storage for all internal memory block pointers to be assigned; total
blocks = vision core count + LLM core count - overlapping core count (the reused
part)
app_ctx->n_internal_mems = core_num_vision + core_num_llm - core_num_same;
app_ctx->internal_mems = (rknn3_tensor_mem**)calloc(app_ctx->n_internal_mems,
sizeof(rknn3_tensor_mem*));
if (!app_ctx->internal_mems) {
    printf("Failed to allocate memory for app_ctx->internal_mems array\n");
    return -1;
}
// Allocate a pointer array for each vision model core (used later to set the model
internal memory)
rknn3_tensor_mem** internal_mems_vision =
(rknn3_tensor_mem**)calloc(core_num_vision, sizeof(rknn3_tensor_mem*));
if (!internal_mems_vision) {
    printf("Failed to allocate memory for internal_mems_vision array\n");
    return -1;
}
// Allocate a pointer array for each LLM core (used later to set the model internal
memory)
rknn3_tensor_mem** internal_mems_llm = (rknn3_tensor_mem**)calloc(core_num_llm,
sizeof(rknn3_tensor_mem*));
if (!internal_mems_llm) {
    printf("Failed to allocate memory for internal_mems_llm array\n");
    return -1;
}

// Allocate and initialize the internal memory blocks for the vision model
int idx = 0;
for (uint32_t i = 0; i < core_num_vision; i++) {
    internal_mems_vision[i] = rknn3_create_mem(app_ctx->vision.rknn_ctx,
core_mem_sizes_vision[i].internal_size, core_mem_sizes_vision[i].core_id,
RKNN3_FLAG_MEMORY_CACHEABLE);
    if (!internal_mems_vision[i]) {
        return -1;
    }
    printf("Created user internal memory for core %d: size=%lu, virt_addr=%p,
phys_addr=0x%lx\n", core_mem_sizes_vision[i].core_id,
           internal_mems_vision[i]->size, internal_mems_vision[i]->virt_addr,
internal_mems_vision[i]->phys_addr);
    app_ctx->internal_mems[idx++] = internal_mems_vision[i];
}

// Allocate internal memory for the LLM (reused or newly allocated)
for (uint32_t i = 0; i < core_num_llm; i++) {
    if (llm_to_vision[i] != -1) {
        // Reuse: the LLM core and a vision model core are on the same physical
core; point directly to the allocated memory
        internal_mems_llm[i] = internal_mems_vision[llm_to_vision[i]];
        continue;
    }
    // Otherwise allocate separate memory for this LLM core; allocation uses
vision.rknn_ctx for unified release later (to avoid double release)
```

```
    internal_mems_llm[i] = rknn3_create_mem(app_ctx->vision.rknn_ctx,
core_mem_sizes_llm[i].internal_size, core_mem_sizes_llm[i].core_id,
RKNN3_FLAG_MEMORY_CACHEABLE);
    if (!internal_mems_llm[i]) {    // All allocated via vision.rknn_ctx for easy
release later
        return -1;
    }
    printf("Created user internal memory for core %d: size=%lu, virt_addr=%p,
phys_addr=0x%lx\n", core_mem_sizes_llm[i].core_id,
           internal_mems_llm[i]->size, internal_mems_llm[i]->virt_addr,
internal_mems_llm[i]->phys_addr);
    // Add the newly allocated memory blocks to the global management array
    app_ctx->internal_mems[idx++] = internal_mems_llm[i];
}

// Set the allocated internal memory for the vision model
ret = rknn3_set_internal_mem(app_ctx->vision.rknn_ctx, internal_mems_vision,
core_num_vision);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_set_internal_mem failed! ret=%d\n", ret);
    return ret;
}
// Set the allocated internal memory (including the reused part) for the LLM model
ret = rknn3_set_internal_mem(app_ctx->llm.rknn_ctx, internal_mems_llm,
core_num_llm);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_set_internal_mem failed! ret=%d\n", ret);
    return ret;
}

// Release the temporary pointer arrays (the internal memory blocks themselves are
allocated by rknn3_create_mem and must be released later with rknn3_destroy_mem)
free(internal_mems_vision);
free(internal_mems_llm);
free(core_mem_sizes_vision);
free(core_mem_sizes_llm);

return ret;
```

> **Note**: Reuse is safe only when the two models execute serially (only one runs at any moment); reuse is not allowed with concurrent execution.

4.2 LLM and MLLM Advanced Features

4.2.1 Prompt, Token, and Embedding Inputs

Applicable Scenarios

LLM / MLLM inference supports three types of text input, specified through `rknn3_llm_input.input_type`, each corresponding to a different data preparation stage. Choosing the type based on the actual data form reduces pre-processing overhead:

Input Type	Applicable Scenarios	Data Preparation Stage
`RKNN3_LLM_INPUT_PROMPT`	General text input; the SDK performs tokenizer segmentation	Natural language text
`RKNN3_LLM_INPUT_TOKEN`	Tokenization already done; skips the text->token conversion	token id sequence
`RKNN3_LLM_INPUT_EMBED`	Embedding already done; skips text->token->embedding	embedding vectors

Usage

All three types pass through `llm_input` (`rknn3_llm_tensor`) uniformly. Fill the `rknn3_llm_input` array and pass it to `rknn3_session_run()` , populating the corresponding fields by type. The three types are mutually exclusive; each `rknn3_llm_input` can carry only one.

- **PROMPT input**: Enter natural language text directly; the SDK performs tokenizer conversion. Recommended for most general scenarios.

```
rknn3_llm_tensor input_tensor = {.name = NULL, .prompt = prompt,
                                 .embed = NULL, .tokens = NULL,
                                 .n_tokens = 0, .enable_thinking = false};
input.input_type = RKNN3_LLM_INPUT_PROMPT;
input.llm_input  = input_tensor;
```

- **TOKEN input**: Enter a token id sequence directly. It suits scenarios where tokenization is already done and the text->token conversion should be skipped (such as reusing historical tokens or custom segmentation).

```
int32_t tokens[6] = {151628, 151629, 151630, 151631, 151632, 151633};
rknn3_llm_tensor input_tensor = {.name = NULL, .prompt = NULL,
                                 .embed = NULL, .tokens = tokens,
                                 .n_tokens = 6, .enable_thinking = false};
input.input_type = RKNN3_LLM_INPUT_TOKEN;
input.llm_input  = input_tensor;
```

- **EMBED input**: Enter embedding vectors directly (size: `n_tokens * embedding_dim * sizeof(float16)`). It skips the text->token->embedding pipeline and suits models without a traditional tokenization flow or with custom embeddings.

```
float16* embed = (float16*)malloc(n_tokens * embedding_dim * sizeof(float16));
/* Fill in the embed data */
rknn3_llm_tensor input_tensor = {.name = NULL, .prompt = NULL,
                                 .embed = embed, .tokens = NULL,
                                 .n_tokens = n_tokens, .enable_thinking = false};
input.input_type = RKNN3_LLM_INPUT_EMBED;
input.llm_input  = input_tensor;
```

> **Note**: For structure members, see 3.1.2. PROMPT/TOKEN/EMBED are mutually exclusive; each `rknn3_llm_input` can carry only one.

4.2.2 Multimodal and Aux Inputs

Applicable Scenarios

Besides text inputs (see 4.2.1), MLLM inference also supports multimodal inputs and auxiliary inputs, covering multimodal fusion and the extended input needs of special models:

Input Type	Applicable Scenarios	Data Preparation Stage
`RKNN3_LLM_INPUT_MULTIMODAL`	Multimodal input; supports image/audio/video individually or combined	Embeddings output by per-modality encoding models
`RKNN3_LLM_INPUT_AUX`	Other input types, used with the main input for special models (such as the 3 deepstack auxiliary inputs of Qwen3-VL)	Auxiliary tensors

Usage

- **MULTIMODAL input**: Use the `rknn3_llm_multimodal_tensor` structure (`rknn3_llm_input.multimodal_input`). Populate the `image` / `audio` / `video` substructures for the actual modalities and leave unused modalities unset. Each substructure contains the pointer to the embeddings output by the encoding model, the token count, the modality count, and text placeholder markers. Placeholders in the prompt use the `<image>` / `<audio>` / `<video>` tags, and their counts must match `n_image` / `n_audio` / `n_video`. Placeholder markers (such as `<|vision_start|>` or `<|vision_bos|>`) vary by model tokenizer; follow the special tokens of the actual model tokenizer.

```
rknn3_llm_multimodal_tensor tensor;
memset(&tensor, 0, sizeof(tensor));
tensor.prompt = "1.<image> 2.<audio> 3.<video> Please analyze the above content";
tensor.image.image_embed   = vision_output;     /* n_image × n_image_tokens ×
embedding_dim */
tensor.image.n_image_tokens = 196;
tensor.image.n_image        = 1;
tensor.image.image_start    = "<|vision_bos|>";
tensor.image.image_end      = "<|vision_eos|>";
tensor.image.image_content  = "<|IMAGE|>";
/* Fill the audio/video sub-structures in the same way */
input.input_type      = RKNN3_LLM_INPUT_MULTIMODAL;
input.multimodal_input = tensor;
```

- **AUX input**: Other input types that provide auxiliary tensors together with the main input, serving as extended inputs for special models. Take Qwen3-VL-2B as an example: besides 1 MULTIMODAL main input, it needs 3 deepstack auxiliary inputs, for 4 `rknn3_llm_input` entries in total. Each AUX carries auxiliary data through `rknn3_aux_tensor.mem` (allocated by `rknn3_create_mem()`):

```
/* Main input: MULTIMODAL, occupies llm_inputs[0] */
/* Auxiliary inputs: 3 AUX, occupy llm_inputs[1..3] */
rknn3_aux_tensor aux_input_tensor0;
rknn3_llm_input aux_input0;

aux_input_tensor0.mem = rknn3_create_mem(ctx, n * embedding_dim * sizeof(float16),
                                         0, RKNN3_FLAG_MEMORY_CACHEABLE);
aux_input0.input_type = RKNN3_LLM_INPUT_AUX;
aux_input0.aux_input  = aux_input_tensor0;
llm_inputs[1] = aux_input0;
/* aux_input1 and aux_input2 follow the same pattern */
```

> **Note**: The placeholder markers of each modality must match the special tokens of the model tokenizer. For MULTIMODAL structure members, see 3.1.2; for the full definition, see *RKNN3_Runtime_C_API_Reference* 5.5.25; for an example showing how to populate the structure, see 3.3.6. AUX cannot be used alone; it must be combined with the main input in the same `llm_inputs` array and passed to `rknn3_session_run()`.

4.2.3 Callback Mechanism

Applicable Scenarios

In LLM inference, stages such as tokenization, embedding lookup, sampling, and result output are delegated to user implementations through the callback mechanism, providing flexibility. Six types of callbacks are configured uniformly through the `RKLLMCallback` structure and registered with `rknn3_session_set_callback()`:

Callback	Purpose	Required
`result_callback`	Invoked after each token is generated; processes output (such as converting to text and printing)	Required
`tokenizer_callback`	Converts text to a token id sequence	Optional (default is used if unset)
`embed_callback`	Looks up embeddings by token id and fills the buffer	Optional (not needed in Tie Word mode)
`sampling_callback`	Custom sampling strategy (argmax/top-k/top-p/temperature)	Optional (default sampling is used if unset)
`output_callback`	Retrieves output tensors (such as logits); requires output_tensors to be preconfigured	Optional
`input_callback`	Modifies/fills input tensors before inference (such as Per-Layer Embeddings)	Optional

Each callback pairs with a `*_userdata` field to pass in context (such as a tokenizer instance or embedding_info).

```
RKLLMCallback callback;
memset(&callback, 0, sizeof(RKLLMCallback));
callback.result_callback    = result_callback;
callback.tokenizer_callback = tokenizer_callback;
callback.embed_callback     = embed_callback;
callback.sampling_callback  = sampling_callback;
rknn3_session_set_callback(session, &callback);
```

`result_callback` handles the different stages according to `LLMCallState` (`RKLLM_RUN_NORMAL` for normal generation, `RKLLM_RUN_FINISH` for completion, `RKLLM_RUN_ERROR` for errors, `RKLLM_RUN_PAUSE/RESUME` for pause/resume, and so on).

> **Note**: `output_callback` additionally requires `callback.output_tensors` and `n_output_tensors` to be set. For complete callback signatures and parameters, see *RKNN3_Runtime_C_API_Reference*.

4.2.4 Function Calling

Applicable Scenarios

Function Calling allows model output during inference to trigger preset function tools, enabling data interaction with external systems (such as weather queries, schedule reminders, and database retrieval) and extending the capability boundary of the LLM.

Usage

Invoke `rknn3_session_set_function_tools()` to set a group of tools (JSON describes function names and parameter definitions). During inference, the model decides whether to initiate a call according to the session template and provides the call request in the output. The calling flow involves two rounds of inference:

1. **Round 1** (role="user"): The model generates a function call request. Parse the output to obtain tool_call_list, perform the actual function invocation, and store the result as tool_prompt.
2. **Round 2** (role="tool"): Feed the function call result back to the model; the model processes it and returns the final result.

```
/* Set the function tools */
char* function_tools = R"([{"type":"function","function":{
    "name":"get_current_temperature","description":"...",
    "parameters":{"type":"object","properties":{...},"required":[...]}}}])";
rknn3_session_set_function_tools(session, function_tools);

/* Round 1: the user asks a question */
input.role = "user";
rknn3_session_run(session, inputs, 1, &param);
/* Parse the output -> invoke the actual function -> build tool_prompt */

/* Round 2: the tool returns the result */
input.role = "tool";
input_tensor.prompt = tool_prompt.c_str();
rknn3_session_run(session, inputs, 1, &param);
```

> **Note**: For a complete example, see `rknn3-runtime/examples/rknn3_session_test_demo/src/rknn3_session_test_function_call.cpp`.

4.2.5 KVCache Management

Applicable Scenarios

KVCache caches the key-value states of attention to accelerate autoregressive generation. RKNN3 provides three KVCache policies plus import/export capabilities, covering scenarios such as long conversations, resuming interrupted computation, and cross-session context saving.

Three Policies

Set through `rknn3_session_set_kvcache_policy()` (`rknn3_kvcache_policy`):

Policy	Behavior	Applicable Scenarios
`RKNN3_KVCACHE_POLICY_RECURRENT` (default)	KVCache is overwritten automatically; `n_keep` / `n_keep_aligned` keeps the system prompt from being overwritten	Long multi-turn conversations
`RKNN3_KVCACHE_POLICY_NORMAL`	Uses only max_context_len of KVCache; generation stops after that	Fixed-length context
`RKNN3_KVCACHE_POLICY_SAVE_CHECKPOINT`	Enables Checkpoint save mode (see 4.2.7)	Acceleration for Linear/Sliding Attention models

```
rknn3_kvcache_policy_param param;
memset(&param, 0, sizeof(param));
param.recurrent.n_keep         = 21;   /* Actual token count of the system prompt */
param.recurrent.n_keep_aligned = 32;   /* Aligned to 32 */
rknn3_session_set_kvcache_policy(session, RKNN3_KVCACHE_POLICY_RECURRENT, &param);
```

Import and Export

`rknn3_session_save_kvcache()` (save to file), `rknn3_session_load_kvcache_from_path()` (load from file), and `rknn3_session_load_kvcache_from_data()` (load from memory) apply to resuming interrupted computation and cross-session context saving. Typical flow: save after the first round of inference, load before the second round.

> **Note**: KVCache state is strongly tied to the model and runtime library version; do not mix it across different models/versions.

4.2.6 Paged Attention

Applicable Scenarios

A traditional contiguous KVCache reserves the full `max_context_len` buffer for each session, resulting in high memory overhead in multi-request concurrency. Paged Attention (paged KVCache) divides the KV Cache into pages of a fixed number of tokens and allocates/manages the cache at page granularity according to each request's actual length, significantly reducing

memory usage in multi-request scenarios. Whether paging is enabled is determined by the `tokens_per_page` configuration during model conversion (see 2.2.3.1); once enabled, the model is a "paged model". Page memory can be allocated automatically by the Runtime or managed explicitly by the application.

Conversion Stage

Configure the paging parameters in `attention_config` of `llm_config` in `rknn.config()`:

```
llm_config['attention_config'][0]['tokens_per_page'] = 1024  # Enable paging, 1024
tokens per page
llm_config['attention_config'][0]['request_num'] = 1         # Number of paged
requests; only 1 is supported currently
```

- A positive integer for `tokens_per_page` enables paging; a multiple of 64 is recommended. Larger pages yield better inference performance, but the performance gain will not be significant. The SlidingAttention type or non-positive values fall back to contiguous KVCache (`kvcache_buffer_len`).
- `request_num` is the number of paged requests; only 1 is supported in the current version.

On-Device Runtime Usage (Session API)

When deploying through the Session API, KVCache page memory is managed automatically inside the Session, and the application does not need to build page tables manually. To precisely control the initial memory footprint (for example, starting with a short context and growing later), use the page-level interfaces at the Runtime layer. Key points:

1. **Query page specifications**: For each physical core participating in the run, set the `core_id` field of `rknn3_kvcache_page_info` to the core ID and call `rknn3_query(ctx, RKNN3_QUERY_KVCACHE_PAGE_INFO, ...)`. The result returns, for each cache type, the `cache_type`, per-page byte size `page_size`, compile-time maximum page count `max_page_num`, and maximum concurrent request count `max_request_num`. A non-paged model returns `n_cache_types == 0`; in that case use the contiguous KVCache flow (see 4.2.5).
2. **Calculate the page count**: The FullAttention page count is computed as `ceil(max_ctx_len / tokens_per_page)` and must not exceed `max_page_num`; other cache types are allocated the full `max_page_num` capacity.
3. **Create page memory**: Call `rknn3_create_mem(ctx, page_size, core_id, RKNN3_FLAG_MEMORY_CACHEABLE)` for each page to create an independent `rknn3_tensor_mem` object. When the CPU needs to write or zero the page content, call `rknn3_mem_sync(..., RKNN3_MEMORY_SYNC_TO_DEVICE)` after writing.
4. **Bind page tables**: Assemble an `rknn3_kvcache_page_binding` for each `(core_id, cache_type, request_index)` combination (`n_pages` is the page count and `pages` points to the page array), then call `rknn3_set_kvcache_pages(ctx, bindings, n_bindings)`. The context must be idle when calling; the interface only replaces the listed bindings, and unlisted bindings remain unchanged.
5. **Unbind and release**: Submit `n_pages = 0` to unbind the corresponding binding; call `rknn3_destroy_mem` to destroy the page memory only after the unbind succeeds. Do not destroy a page while it is still referenced by a page table.

```
/* page_attr comes from RKNN3_QUERY_KVCACHE_PAGE_INFO; page_count is computed
   from max_ctx_len and tokens_per_page, and must not exceed page_attr-
>max_page_num. */
uint32_t page_count = (max_ctx_len + tokens_per_page - 1) / tokens_per_page;
rknn3_tensor_mem* pages[page_count];
for (uint32_t i = 0; i < page_count; ++i) {
    pages[i] = rknn3_create_mem(ctx, page_attr->page_size, core_id,
                                RKNN3_FLAG_MEMORY_CACHEABLE);
}
rknn3_kvcache_page_binding binding = {
    core_id, page_attr->cache_type, request_index,
    page_count, (const rknn3_tensor_mem* const*)pages, {0}
};
rknn3_set_kvcache_pages(ctx, &binding, 1);

/* After inference: unbind first (n_pages=0), then destroy the page memory */
rknn3_kvcache_page_binding unbind = {core_id, page_attr->cache_type,
                                     request_index, 0, NULL, {0}};
if (rknn3_set_kvcache_pages(ctx, &unbind, 1) == RKNN3_SUCCESS) {
    for (uint32_t i = 0; i < page_count; ++i)
        rknn3_destroy_mem(ctx, pages[i]);
}
```

> **Note**: After binding, the page memory remains owned by the caller and must stay valid until unbound. If validation fails, old bindings remain unchanged; if the page-table write fails, the submitted binding will not take effect, and the caller should handle the lifetime of the pages in that batch according to the return value. A paged model must not call `rknn3_set_kvcache_mem()` to set a contiguous KVCache. For complete interface and structure definitions, see Section 4.4.4 and Section 6.6 of the *RKNN3 Runtime C API Reference*.

4.2.7 KVCache Checkpoint

Applicable Scenarios

The Checkpoint policy saves KV Cache snapshots at fixed intervals. Generation can later resume from a checkpoint, avoiding re-prefilling the entire prefix. It takes effect only for Linear Attention / Sliding Attention models (such as the Qwen3.5 and Gemma4 series). Full Attention follows a separate path, and the setting has no effect on other models.

Usage

With the `RKNN3_KVCACHE_POLICY_SAVE_CHECKPOINT` policy, configure four parameters in `rknn3_kvcache_policy_param.save_checkpoint` (all must be aligned to 128):

Parameter	Description
`checkpoint_start_pos`	Base position, not saved itself; actual save positions = start + k×interval (k=1..max_count)
`checkpoint_interval`	Save interval; common values are 128/256/512/1024
`max_checkpoint_count`	Maximum number of saves; upper bound = floor((max_context_len - start)/interval)

Parameter	Description
`checkpoint_tail_overwrite`	false by default; when set to true, the last slot is overwritten in a rolling manner — suitable when memory is limited but the most recent restore point must be kept

```
rknn3_kvcache_policy_param param;
memset(&param, 0, sizeof(param));
param.save_checkpoint.checkpoint_start_pos      = 0;
param.save_checkpoint.checkpoint_interval       = 128;
param.save_checkpoint.max_checkpoint_count      = 2;
param.save_checkpoint.checkpoint_tail_overwrite = false;
rknn3_session_set_kvcache_policy(session, RKNN3_KVCACHE_POLICY_SAVE_CHECKPOINT,
&param);
```

> **Note**: When reusing only a fixed system prompt, `max_checkpoint_count<=2` is recommended. Validation rules: an error is raised directly if `start_pos > max_context_len - interval`; unaligned parameters are forced up to alignment; a `max_count` above the upper bound is forced down. Verify the effect through the log `load LINEAR_ATTN_CACHE kvcache_checkpoint pos: ...`.

4.2.8 LoRA

Applicable Scenarios

LoRA (Low-Rank Adaptation) allows adapters to be loaded/switched dynamically without modifying base model weights. It suits multi-task fine-tuned deployment: one base model + multiple LoRA adapters, switched per task, saving storage and memory. LoRA usage involves two stages: in the conversion stage, RKNN3-Toolkit converts the LoRA weights together with the base model into an RKNN model; in the on-device deployment stage, the Runtime interfaces load adapters and enable them on demand.

Conversion Stage

RKNN3-Toolkit provides the `load_lora`, `enable_lora`, and `disable_lora` interfaces for loading and controlling LoRA weights during model conversion. LoRA weights must be loaded after the model is loaded (`load_onnx` / `load_llm`) and before the model is built (`build`):

```
rknn.load_lora(lora_paths=[["./adapter_model.safetensors"]],
               lora_config_paths=[["./adapter_config.json"]],
               lora_patterns=[["lora0_pattern0"]],
               lora_distribute_strategy='best_perf')
rknn.enable_lora(enable_patterns=["lora0_pattern0"])
rknn.disable_lora(disable_patterns=["lora0_pattern0"])
```

On-Device Deployment

On the device side, load and control LoRA through the Runtime interfaces. `rknn3_lora_init()` initializes context-level LoRA support from a weight file path and loads the metadata. Invoke it after `rknn3_model_init()` and before any session uses LoRA. Afterwards, invoke `RKNN3_QUERY_LORA_NUM` and `RKNN3_QUERY_LORA_INFO` to query the loaded LoRA information, then invoke `rknn3_lora_load()` to load the specified adapter.

LoRA weights are loaded and managed at the context level and shared by multiple sessions under the same context. Whether they take effect for a session is controlled at two levels:

- **Context level** (takes effect for all inference under the context; updated once at invocation time):

Interface	Function
`rknn3_lora_init()` / `rknn3_lora_init_from_data()`	Initializes LoRA metadata from file/memory
`rknn3_lora_load()` / `rknn3_lora_unload()`	Loads/unloads adapter weights
`rknn3_lora_enable()` / `rknn3_lora_disable()`	Enables/disables at the context level

- **Session level** (controls only that session; updated on every `session_run`, without affecting other sessions):

Interface	Function
`rknn3_session_enable_lora()` / `rknn3_session_disable_lora()`	Enables/disables at the session level without reloading weights

On-device invocation example:

```
rknn3_lora lora_list[RKNN3_MAX_LORA_NUM];
memset(lora_list, 0, sizeof(lora_list));
rknn3_lora_init(ctx, "lora_weight_path");            /* Initialize the metadata;
must be called after model_init */
uint32_t n_lora = 0;
rknn3_query(ctx, RKNN3_QUERY_LORA_NUM, &n_lora, sizeof(n_lora));        /* Query the
number of LoRA adapters */
rknn3_query(ctx, RKNN3_QUERY_LORA_INFO, lora_list, sizeof(lora_list));  /* Query the
valid lora descriptors */
rknn3_lora_load(ctx, &lora_list[0]);                 /* Load the adapter weights */
rknn3_session_enable_lora(session, &lora_list[0]);   /* Enable at the session level
*/
```

> **Note**: Mixing the two levels of interfaces is not recommended: session-level settings override context-level settings, making the effective order indeterminate. LoRA weights support int4 / int6 / int8 / float16 quantization, with multi-core splitting strategies `best_perf` / `less_mem`.

4.2.9 Tie Word Embedding

Applicable Scenarios

Tie Word Embeddings is a weight-tying technique: the input embedding layer and the output lm_head layer share the same weight matrix. When enabled, the device side reuses the lm_head weights inside the model, and **the Host side does not need to load `embed.bin`**, saving Host memory and disk space.

Usage

- **Confirm support**: Check whether the `tie_word_embeddings` field in the HuggingFace `config.json` is true (common in the Qwen series).

- **Conversion configuration**: Set `llm_config['vocab'][0]['tie_embeddings'] = True`. After conversion, verify with `strings xxx.rknn | grep tie_word_embeddings`.

```
llm_config = DEFAULT_RKNN_LLM_CONFIG.copy()
llm_config['vocab'][0]['tie_embeddings'] = True
rknn.config(..., llm_config=llm_config)
```

- **Deployment differences**: In Tie mode, `embed.bin` is not loaded (no open/mmap) and `embed_callback` is not set; in non-Tie mode, both are required.

Mode	embed.bin	embed_callback	Host Memory
Tie	Not required	Not required	Lower
Non-Tie	Required	Required	Higher

> **Note**: Even if the model contains `tie_word_embeddings:true`, you can still set the environment variable `RKNN_ENABLE_TIE_WORD_EMBEDDINGS=0` to force non-Tie mode. However, embed.bin must then be loaded and embed_callback must be set, losing the memory advantage.

4.2.10 Pause, Resume, and State Query

Applicable Scenarios

A session can be paused/resumed during inference to insert other tasks during generation (such as handling user interruption or higher-priority tasks). State query monitors inference progress, performance statistics, and LoRA/KVCache status.

Usage

- **Pause**: `rknn3_session_pause()`.
- **Resume**: `rknn3_session_resume()`.

On pause, `result_callback` is invoked with `RKLLM_RUN_PAUSE`; on resume, with `RKLLM_RUN_RESUME`. Handle these states in the callback to implement business logic.

```
rknn3_session_pause(session);
/* Handle other tasks while paused */
rknn3_session_resume(session);
```

- **State query**: `rknn3_session_query_state()` queries the current state and stores the result in `RKLLMRunState`. Main fields:

Field	Description
`n_max_tokens`	Maximum number of inference tokens
`n_total_tokens`	Number of tokens already inferred
`n_prefill_tokens`	Number of tokens processed in the prefill stage
`n_decode_tokens`	Number of tokens generated in the generate stage

Field	Description
`n_output_tokens`	Number of model output tokens (= n_decode_tokens + 1)
`input_tokens`	Pointer to the input token array (users must not free it)
`output_tokens`	Pointer to the output token array (users must not free it)
`kvcache_policy`	Current KVCache policy
`n_loras_enabled`	Number of enabled LoRAs

> **Note**: For the complete field list, see *RKNN3_Runtime_C_API_Reference* 5.5.31.

4.2.11 Structured Output (Grammar)

Applicable Scenarios

Structured output (grammar-guided generation) constrains the model output to strictly match a specified format (JSON Schema / Regex / EBNF / any valid JSON / structural tags) during the decode stage of LLM inference, masking illegal tokens at the token level to guarantee structurally correct output and reduce post-processing tolerance cost. Typical scenarios include:

- Forcing the model to output structured data conforming to a business JSON Schema (e.g. `{"name":..., "age":...}`), eliminating regex extraction and field validation.
- Constraining the model output to regex formats such as email, date, UUID, hex color, etc.
- Using EBNF to constrain output to enumerated values (e.g. `yes` / `no` / `maybe`) or custom grammars.
- Using structural tags to constrain the tool-call output format of models such as Qwen3.5.

Basic Principles

The SDK integrates XGrammar to implement token-level constraints: it compiles user-provided JSON Schema / Regex / EBNF into BNF, generates a token bitmask before each decode step, sets illegal token logits to `-inf`, and then lets the sampler pick a token, so the output strictly conforms to the constraint. The constraint state is automatically maintained by the Runtime in the decode loop of `rknn3_session_run()`; users do not need to intervene in the sampling flow.

The Runtime always applies the grammar set by `rknn3_session_set_grammar()` starting from the first generated token of the current run. It does not detect whether the model has entered or exited thinking, nor does it automatically add a thinking prefix. `enable_thinking` only affects the model/chat template: when thinking is disabled, the grammar usually only needs to describe the final answer; when thinking is enabled, the caller must use a single grammar that covers the complete generation stream, including the thinking content, the ending marker, and the final answer.

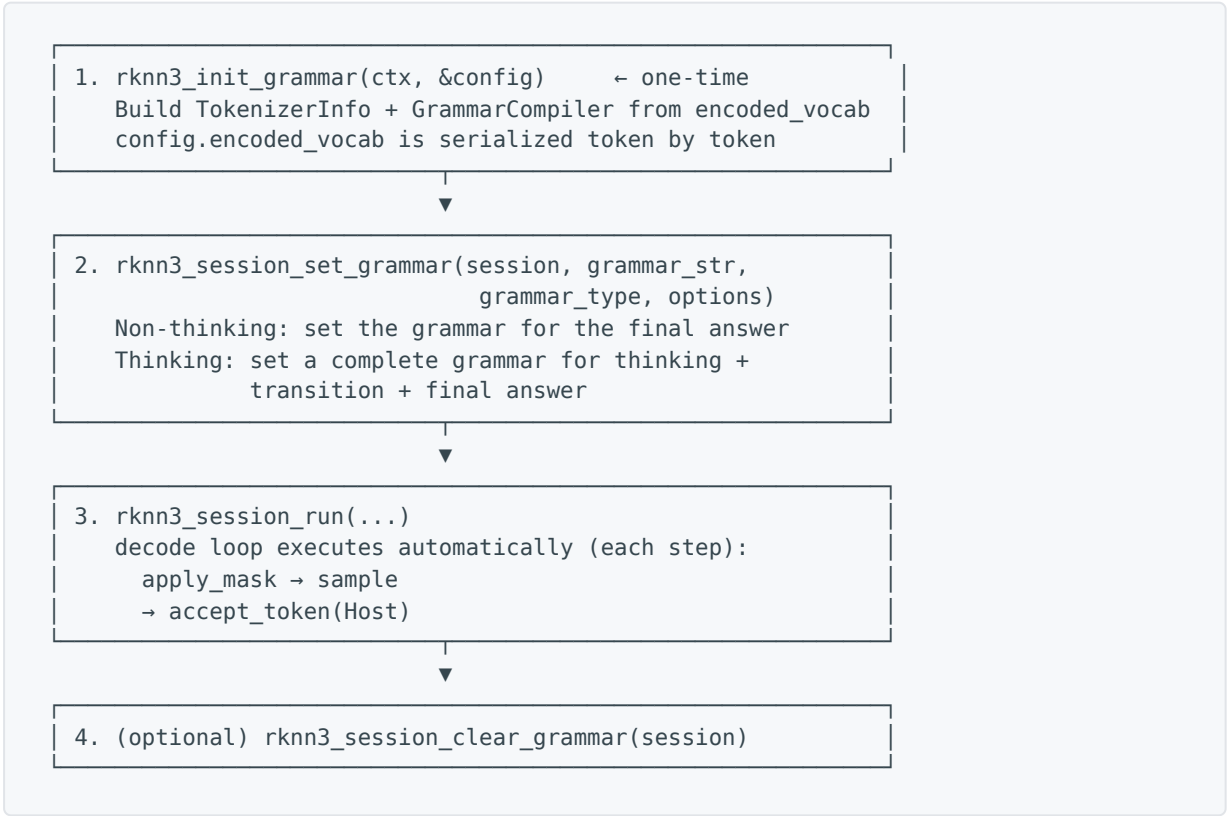
Supported Constraint Types

`rknn3_grammar_type`	Description	Typical Usage
`RKNN3_GRAMMAR_JSON_SCHEMA`	JSON Schema string, constrains output to JSON conforming to the schema	`{"type":"object","properties":{...}}`

`rknn3_grammar_type`	**Description**	**Typical Usage**
`RKNN3_GRAMMAR_EBNF`	EBNF grammar string, supports multi-rule composition and enums	`root ::= "yes" | "no" | "maybe"`
`RKNN3_GRAMMAR_BUILTIN_JSON`	Built-in any-valid-JSON, no schema required (pass `NULL` for `grammar_str`)	Constrain output to valid JSON without restricting fields
`RKNN3_GRAMMAR_REGEX`	Regular expression, constrains output to fully match the regex	`[0-9]{4}-[0-9]{2}-[0-9]{2}` (date)
`RKNN3_GRAMMAR_STRUCTURAL_TAG`	XGrammar Structural Tag JSON; combines constant strings, regex, and JSON Schema, and supports `tag` / `triggered_tags`	Constraints for thinking and tool-call formats used by models such as Qwen

Usage Flow

Grammar usage is a three-step process: context initialization (build vocabulary and compiler) -> session-level constraint setting (compile grammar) -> session inference (decode loop auto-mask); after inference, the constraint can be cleared to restore free generation. This version does not provide a separate interface for setting a thinking grammar. Thinking scenarios also use the existing `rknn3_session_set_grammar()` API, but the supplied grammar must cover the complete generation stream. The overall flow:



The relationship between `enable_thinking` and the grammar content is as follows:

`enable_thinking`	**Generation stream that `rknn3_session_set_grammar()` must describe**
`false`	The final answer; constraints apply directly from the first generated token

enable_thinking	Generation stream that rknn3_session_set_grammar() must describe
true	The complete output: thinking content + thinking end/transition marker + final answer

> **Note**: The Runtime does not infer the grammar phase from the model type, chat template, first token, or a generated </think> marker. If thinking is enabled but only a final-answer JSON Schema is set, that final-answer grammar is incorrectly applied starting from the first reasoning token.

API Description

The grammar interfaces are divided into three levels, select as needed:

- **Context level** (one-time, builds vocabulary and compiler):

API	Description
rknn3_init_grammar(ctx, &config)	Build the Host-side TokenizerInfo + GrammarCompiler from encoded_vocab; must be called before a session uses grammar; repeated calls return an error

Main fields of rknn3_grammar_config:

Field	Description
vocab_size	Number of tokens in the vocabulary
encoded_vocab	Serialized vocabulary blob: for each token, write int32_t len + len bytes of token string
encoded_vocab_len	Total blob size in bytes
n_stop_tokens / stop_tokens	EOS/stop token id list; generation terminates at these tokens under constraint

- **Session level** (auto-driven mask by the session_run decode loop, recommended):

API	Description
rknn3_session_set_grammar(session, str, type, options)	Compile grammar and bind to session; auto-mask during run; empty string with type not BUILTIN_JSON is equivalent to clear
rknn3_session_clear_grammar(session)	Clear grammar, restore free generation

rknn3_grammar_set_options fine-tunes the JSON output format; all fields are optional (initialize with the macro RKNN3_GRAMMAR_SET_OPTIONS_INIT; unset fields keep defaults):

Field	Description
any_whitespace	Whether arbitrary whitespace is allowed between JSON elements
indent	Indent width (-1 means no indent)
strict_mode	Strict mode switch
max_whitespace_cnt	Maximum number of whitespace characters

Field	Description
`any_order`	Whether JSON object keys are allowed in any order
`item_separator` / `key_value_separator`	Element separator / key-value separator
`root_rule_name`	Root rule name to specify for multi-rule EBNF

- **Handle level** (when users manually orchestrate the sampling loop):

API	Description
`rknn3_grammar_create(ctx)` / `rknn3_grammar_destroy(ctx, g)`	Create / destroy a grammar handle (requires `init_grammar` first)
`rknn3_grammar_set(g, str, type, options)` / `rknn3_grammar_clear(g)`	Set / clear the constraint
`rknn3_grammar_apply_mask(g, logits, vocab_size, output_idx)`	Apply the mask to logits (set illegal logits to `-inf`)
`rknn3_grammar_accept_token(g, token_id, output_idx)`	Notify grammar that the token has been sampled, advance the state machine
`rknn3_grammar_reset(g)`	Reset the state machine, restart (without recompiling)

> **Note**: The difference between session level and handle level--the session level is auto-driven by the `session_run` decode loop calling `apply_mask` / `accept_token`; the handle level is manually orchestrated by the user's own sampling loop (custom sampling / special inference flow), and in coprocessor mode it executes purely on the Host, not through the Device.

Usage Example

Take JSON Schema constraint as an example (require output JSON containing `name` / `age` fields). For tokenizer and `VocabInfo` acquisition, see 2.2.4 Model and Tokenizer Adaptation:

```
/* 1. Serialize encoded_vocab with the tokenizer: for each token write int32 length
+ token string */
int vocab_size = vocab_info.vocab_size;
std::vector<uint8_t> vocab_blob;
for (int i = 0; i < vocab_size; i++) {
    std::string piece = tokenizer->TokenToPiece(i);
    int32_t len = (int32_t)piece.size();
    vocab_blob.insert(vocab_blob.end(), (uint8_t*)&len, (uint8_t*)&len +
sizeof(int32_t));
    vocab_blob.insert(vocab_blob.end(), (uint8_t*)piece.data(),
(uint8_t*)piece.data() + piece.size());
}

/* 2. Initialize context-level grammar (one-time) */
std::vector<int32_t> stop_tokens;
for (int i = 0; i < vocab_info.n_special_eos_id; i++) {
    stop_tokens.push_back(vocab_info.special_eos_id[i]);
}
rknn3_grammar_config g_cfg = {};
g_cfg.vocab_size         = vocab_size;
g_cfg.encoded_vocab      = vocab_blob.data();
g_cfg.encoded_vocab_len  = (int32_t)vocab_blob.size();
g_cfg.n_stop_tokens      = (int32_t)stop_tokens.size();
g_cfg.stop_tokens        = stop_tokens.data();
rknn3_init_grammar(ctx, &g_cfg);

/* 3. Set the session-level constraint (JSON Schema) */
const char* schema =
    "{\"type\":\"object\","
    "\"properties\":{\"name\":{\"type\":\"string\"},"
    "\"age\":{\"type\":\"integer\",\"minimum\":0,\"maximum\":150}},"
    "\"required\":[\"name\",\"age\"]}";
rknn3_session_set_grammar(session, schema, RKNN3_GRAMMAR_JSON_SCHEMA, NULL);

/* 4. Normal session_run, the decode loop auto-masks */
rknn3_llm_infer_param infer = {};
infer.max_new_tokens = 256;
rknn3_llm_tensor input_tensor = {.name = NULL, .prompt = (char*)"Generate a JSON
with name and age.",
                                 .embed = NULL, .tokens = NULL, .n_tokens = 0,
.enable_thinking = false};
rknn3_llm_input input = {};
input.input_type = RKNN3_LLM_INPUT_PROMPT;
input.llm_input = input_tensor;
rknn3_session_run(session, &input, 1, &infer);
/* Output like: {"name":"Alice","age":30} */

/* 5. Clear the constraint, restore free generation */
rknn3_session_clear_grammar(session);
```

Other constraint type examples:

```
/* Regex: constrain to a date YYYY-MM-DD */
rknn3_session_set_grammar(session, "[0-9]{4}-[0-9]{2}-[0-9]{2}",
                          RKNN3_GRAMMAR_REGEX, NULL);

/* EBNF: constrain to one of yes/no/maybe */
const char* ebnf = "root ::= answer\nanswer ::= \"yes\" | \"no\" | \"maybe\"\n";
rknn3_session_set_grammar(session, ebnf, RKNN3_GRAMMAR_EBNF, NULL);

/* Built-in JSON: output any valid JSON object (pass NULL for grammar_str) */
rknn3_session_set_grammar(session, NULL, RKNN3_GRAMMAR_BUILTIN_JSON, NULL);
```

Thinking Mode: Compose the Complete Grammar in the Application

When thinking is enabled, no new API is required, and the final-answer grammar must not be passed to the Runtime by itself. The caller should use the existing `rknn3_session_set_grammar()` API to submit one grammar that covers the complete generation stream. With the XGrammar backend, the recommended approach is to use a Structural Tag `sequence` to describe the following in order:

```
thinking content -> thinking end/transition marker -> final answer
```

For example, the model outputs JSON containing an integer `answer` after generating `</think>\n\n`:

```
const char* full_thinking_json =
    "{\"type\":\"structural_tag\",\"format\":{"
    "\"type\":\"sequence\",\"elements\":["
    /* 1. Thinking: allow text of any length until </think> */
    "{\"type\":\"tag\",\"begin\":\"\","
        "\"content\":{\"type\":\"any_text\"},\"end\":\"</think>\"},"
    /* 2. Fixed separator between thinking and the final answer */
    "{\"type\":\"const_string\",\"value\":\"\\n\\n\"},"
    /* 3. Final answer */
    "{\"type\":\"json_schema\",\"json_schema\":{"
        "\"type\":\"object\","
        "\"properties\":{\"answer\":{\"type\":\"integer\"}},"
        "\"required\":[\"answer\"],\"additionalProperties\":false}}"
    "]}}";

int ret = rknn3_session_set_grammar(session, full_thinking_json,
                                    RKNN3_GRAMMAR_STRUCTURAL_TAG, NULL);
if (ret != RKNN3_SUCCESS) {
    /* Handle error */
}

rknn3_llm_tensor input_tensor = {
    .name = NULL,
    .prompt = "What is 2 + 2? Reply with JSON containing integer field answer.",
    .embed = NULL,
    .tokens = NULL,
    .n_tokens = 0,
    .enable_thinking = true,
};
/* Generation stream: ...reasoning...</think>\n\n{\"answer\":4} */
```

The empty `begin` in the example allows the grammar to accept arbitrary thinking text starting from the first generated token. This works both when the opening marker is already in the prompt/KVCache and when the model generates an opening marker before `</think>` (the marker is allowed but not required). If the model uses a different string or special token as the thinking boundary, adjust `begin`, `end`, and the separator to match the actual chat template.

Each call to `rknn3_session_set_grammar()` accepts only one grammar. Here, "compose" means describing the complete language within one Structural Tag JSON using nodes such as `sequence`, `tag`, `json_schema`, and `regex`; it does not mean directly concatenating multiple serialized JSON strings. After thinking is disabled, set a final-answer-only grammar again; otherwise, the complete grammar still requires a thinking end marker.

Constraining Output to a tool_call

Grammar can constrain the outer markers and argument JSON structure of a tool call, but it does not register or execute tools. The caller remains responsible for configuring tool definitions, the chat template, prompts, and output parsing. For the SDK Function Calling flow, see 4.2.4.

Different models use different function/tool_call serialization protocols. The following examples show two common formats covered by `rknn3_session_test_grammar.cpp`. Select the grammar that matches the model and its chat template; do not mix the formats.

1. **Native XML (Qwen3.5 / Qwen3-Coder)**

Both the function name and parameters are represented by XML tags:

```
<tool_call>
<function=get_weather>
<parameter=location>
Shanghai
</parameter>
</function>
</tool_call>
```

The corresponding Structural Tag string is shown below. The function name is constrained by a regex, and the example parameter is fixed to `location=Shanghai`:

```
const char* native_xml_tool_call_tag =
    "{\"type\":\"structural_tag\",\"format\":{\"type\":\"sequence\",\"elements\":["
    "{\"type\":\"const_string\",\"value\":\"<tool_call>\\n<function=\"},"
    "{\"type\":\"regex\",\"pattern\":\"[A-Za-z0-9_-]{1,64}\"},"
    "{\"type\":\"const_string\",\"value\":\">\\n\"},"
    "{\"type\":\"const_string\","
        "\"value\":\"<parameter=location>\\nShanghai\\n</parameter>\"},"
    "{\"type\":\"const_string\","
        "\"value\":\"\\n</function>\\n</tool_call>\"}"
    "]}}";
```

1. **JSON-in-XML / Hermes (Qwen3)**

The `<tool_call>` wrapper contains JSON with `name` and `arguments` fields:

```
<tool_call>
{"name": "get_current_temperature", "arguments": {"location": "Shanghai"}}
</tool_call>
```

The corresponding configuration uses `triggered_tags`: after `<tool_call>\n` is encountered, generation enters the specified tag, and a JSON Schema constrains `arguments`:

```
const char* json_in_xml_tool_call_tags =
    "{\"type\":\"structural_tag\",\"format\":{\"type\":\"triggered_tags\","
    "\"triggers\":[\"<tool_call>\\n\"],\"tags\":[{\"type\":\"tag\","
    "\"begin\":\"<tool_call>\\n"
        "{\\\"name\\\": \\\"get_current_temperature\\\", \\\"arguments\\\": \","
    "\"content\":{\"type\":\"json_schema\",\"json_schema\":{"
        "\"type\":\"object\",\"additionalProperties\":true}},"
    "\"end\":\"}\\n</tool_call>\"}],"
    "\"at_least_one\":true,\"stop_after_first\":true}}";
```

The strings above match the protocols used by the reference tests and demonstrate how to organize a grammar. In production, define an exact function name, parameter fields, required fields, and `additionalProperties` setting for each tool, and keep them consistent with the tool descriptions passed to `rknn3_session_set_function_tools()`. If the model uses special tokens or another wrapper, adjust the trigger, tag boundaries, and constant strings accordingly.

Thinking Disabled: Directly Constrain the Final tool_call

Select the Native XML or JSON-in-XML grammar above according to the model, and set `enable_thinking=false`. Because Grammar takes effect from the first generated token, the output can only follow the selected tool_call protocol:

```
/* Qwen3.5 / Qwen3-Coder: select Native XML. For Qwen3, use
 * json_in_xml_tool_call_tags instead.
 */
const char* selected_tool_call_grammar = native_xml_tool_call_tag;
int ret = rknn3_session_set_grammar(session, selected_tool_call_grammar,
                                    RKNN3_GRAMMAR_STRUCTURAL_TAG, NULL);

rknn3_llm_tensor input_tensor = {
    .name = NULL,
    .prompt = "Call get_weather for Shanghai. Reply only with the tool call.",
    .enable_thinking = false,
};
```

Thinking Enabled, Option 1: Manually Compose Thinking + Native XML

Place both the thinking prefix and the Native XML tool_call in a single `sequence`. This requires generation to enter `<tool_call>\n<function=...>` immediately after `</think>\n\n`:

```
const char* full_thinking_native_xml_tool_call =
    "{\"type\":\"structural_tag\",\"format\":{"
    "\"type\":\"sequence\",\"elements\":["
    /* Thinking region */
    "{\"type\":\"tag\",\"begin\":\"\","
        "\"content\":{\"type\":\"any_text\"},\"end\":\"</think>\"},"
    "{\"type\":\"const_string\",\"value\":\"\\n\\n\"},"
    /* Native XML tool_call region */
    "{\"type\":\"const_string\",\"value\":\"<tool_call>\\n<function=\"},"
    "{\"type\":\"regex\",\"pattern\":\"[A-Za-z0-9_-]{1,64}\"},"
    "{\"type\":\"const_string\",\"value\":\">\\n\"},"
    "{\"type\":\"const_string\","
        "\"value\":\"<parameter=location>\\nShanghai\\n</parameter>\"},"
    "{\"type\":\"const_string\","
        "\"value\":\"\\n</function>\\n</tool_call>\"}"
    "]}}";

int ret = rknn3_session_set_grammar(session, full_thinking_native_xml_tool_call,
                                    RKNN3_GRAMMAR_STRUCTURAL_TAG, NULL);

rknn3_llm_tensor input_tensor = {
    .name = NULL,
    .prompt = "Call get_weather for Shanghai. Reply only with the tool call.",
    .enable_thinking = true,
};
```

Thinking Enabled, Option 2: Complete Grammar + JSON-in-XML `triggered_tags`

For the JSON-in-XML/Hermes protocol, `triggered_tags` can be placed after the thinking prefix. To support multiple functions, add a matching `begin` and argument schema to `tags` for each function and configure triggers that uniquely dispatch to the corresponding tags:

```
const char* full_thinking_json_in_xml_tool_call =
    "{\"type\":\"structural_tag\",\"format\":{"
    "\"type\":\"sequence\",\"elements\":["
    /* Thinking region */
    "{\"type\":\"tag\",\"begin\":\"\","
        "\"content\":{\"type\":\"any_text\"},\"end\":\"</think>\"},"
    "{\"type\":\"const_string\",\"value\":\"\\n\\n\"},"
    /* tool_call region */
    "{\"type\":\"triggered_tags\","
        "\"triggers\":[\"<tool_call>\\n\"],"
        "\"tags\":[{\"type\":\"tag\","
            "\"begin\":\"<tool_call>\\n"
                "{\\\"name\\\": \\\"get_current_temperature\\\", \\\"arguments\\\":
\","
            "\"content\":{\"type\":\"json_schema\",\"json_schema\":{"
                "\"type\":\"object\",\"additionalProperties\":true}},"
            "\"end\":\"}\\n</tool_call>\"}],"
        "\"at_least_one\":true,\"stop_after_first\":true}"
    "]}}";

int ret = rknn3_session_set_grammar(session, full_thinking_json_in_xml_tool_call,
                                    RKNN3_GRAMMAR_STRUCTURAL_TAG, NULL);

rknn3_llm_tensor input_tensor = {
    .name = NULL,
    .prompt = "Call get_current_temperature with location=Shanghai. Reply only with
the tool call.",
    .enable_thinking = true,
};
```

Each trigger in `triggered_tags` must be an unambiguous prefix of the corresponding tag `begin`. Do not use a single permissive `triggered_tags` node to represent both "arbitrary thinking text" and "one mandatory tool_call":

- `at_least_one=false` allows free text before the trigger, but also allows generation to finish without any tool_call.
- `at_least_one=true` requires at least one tag, while disallowing leading free text within that `triggered_tags` node.
- Therefore, to preserve thinking while requiring at least one tool_call, use an outer `sequence`: describe thinking and `</think>\n\n` first, then enter `triggered_tags`.

Constraint Capabilities and Boundaries

Grammar improves output-format determinism, but it cannot guarantee that every inference produces a tool_call that is 100% consistent with the user's business intent. Distinguish structural validity from semantic correctness:

- Grammar only constrains whether generated tokens belong to the described language. For example, a regex-constrained function name or an `arguments` value constrained only as an arbitrary object does not ensure that the model selects the correct function or fills in `location=Shanghai`. Tool selection, argument values, and their consistency with the context still depend on model capability, tool descriptions, prompts, the chat template, and sampling configuration. Express fixed values as constants or enums in the grammar/schema, and perform business validation before invoking a tool.

- When a run completes its grammar normally, the output structure conforms to the corresponding Structural Tag/JSON Schema. However, reaching the `max_new_tokens` or context limit, a user stop/abort, or an inference error can still leave an incomplete thinking section or tool_call. The caller must inspect the run status, then parse and validate the complete output again.
- `triggered_tags` only constrains a matching tag after its trigger is encountered. Whether a tool_call is mandatory depends on `at_least_one` and the surrounding structure. With a permissive configuration, the model may never trigger a tool_call.
- The thinking opening marker, end marker, newlines, and special tokens must match the actual model template. Even if a mismatched grammar compiles successfully, it may degrade generation quality, leave no natural token choices, or prevent completion within the token budget.

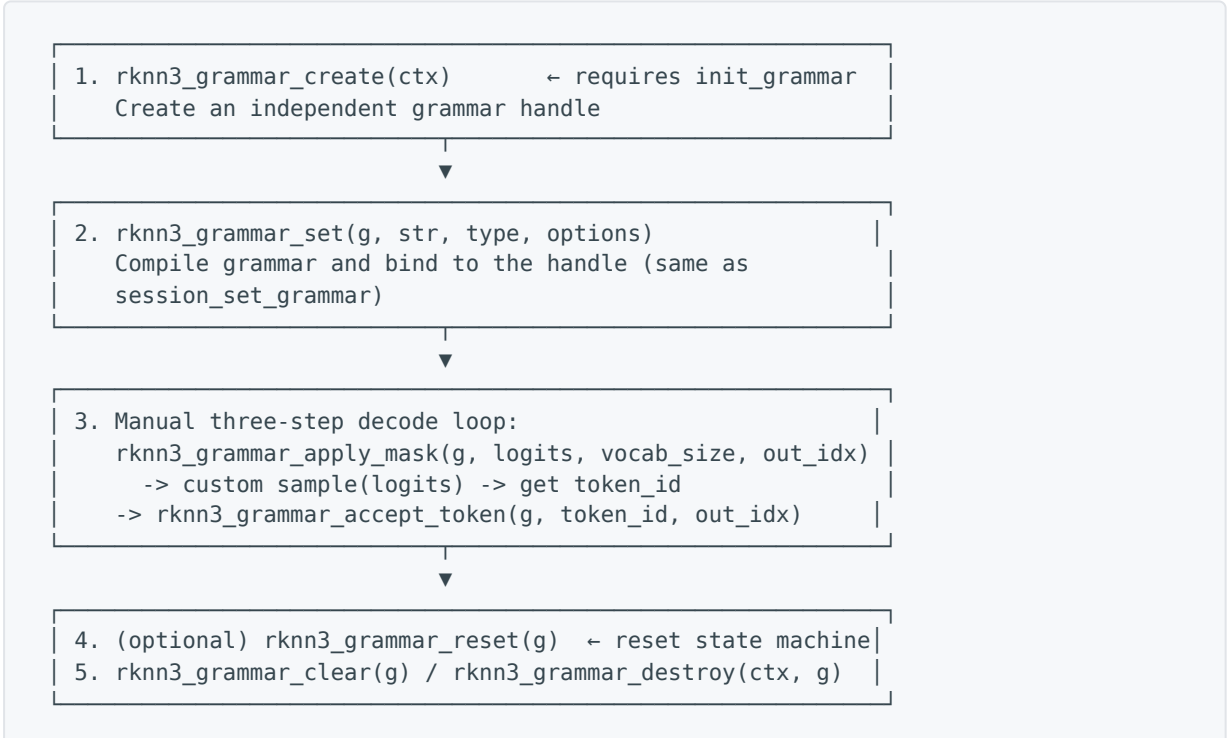
Therefore, Grammar makes normally completed output conform to a machine-parseable structural constraint; it does not replace the model's tool-call decision or business-level correctness validation. In production, optimize the model and prompts, and validate the function name, argument schema, argument values, and permissions before executing a tool.

Handle-Level Manual Driving Mode

When a custom sampling strategy or special inference flow is needed (such as implementing beam search, top-k/top-p sampling, or logits post-processing yourself), the session-level auto-mask cannot intervene in the user's custom sampling path; in this case use the handle-level APIs to manually orchestrate the sampling loop. In coprocessor mode, the handle level executes purely on the Host, not through the Device; the user must obtain the logits of each decode step and drive the flow in the order `apply_mask -> custom sample -> accept_token`.

Usage Flow

The handle-level manual driving flow:



Usage Example

Manual sampling loop example (JSON Schema constraint, logits obtained from a custom inference path):

```
/* 1. Create a grammar handle (requires rknn3_init_grammar first) */
rknn3_grammar* grammar = rknn3_grammar_create(ctx);

/* 2. Compile grammar and bind to the handle */
const char* schema =
    "{\"type\":\"object\",\"properties\":{\"name\":{\"type\":\"string\"}}}";
rknn3_grammar_set(grammar, schema, RKNN3_GRAMMAR_JSON_SCHEMA, NULL);

/* 3. Manual decode loop: apply_mask -> custom sample -> accept_token */
for (int step = 0; step < max_new_tokens; step++) {
    /* Obtain the current step logits from the custom inference path */
    float16* logits     = ...;  /* current step logits, length = vocab_size */
    int32_t  vocab_size  = ...;

    /* 3a. Apply the mask: illegal token logits are set to -inf */
    rknn3_grammar_apply_mask(grammar, logits, vocab_size, 0);

    /* 3b. Custom sampling (greedy / top-k / top-p, etc.) */
    int32_t token_id = my_sample(logits, vocab_size);

    /* 3c. Notify grammar that this token was sampled, advance the state machine;
           failure means the sampling is inconsistent with the constraint */
    if (rknn3_grammar_accept_token(grammar, token_id, 0) != 0) {
        printf("xgrammar AcceptToken rejected token_id=%d\n", token_id);
        break;
    }

    /* End generation when a stop token is hit */
    if (is_stop_token(token_id, stop_tokens, n_stop_tokens)) break;
}

/* 4. Reset the state machine, start the next constrained generation (without
recompiling) */
rknn3_grammar_reset(grammar);

/* 5. Destroy the handle (clear first, then destroy) */
rknn3_grammar_clear(grammar);
rknn3_grammar_destroy(ctx, grammar);
```

> **Note**: The handle level and session level are two driving modes--the session level has the `rknn3_session_run` decode loop automatically call `apply_mask` / `accept_token`; the handle level has the user orchestrate the sampling loop. Both share the same TokenizerInfo / GrammarCompiler built by `rknn3_init_grammar` on the context, so switching driving modes does not require re-initialization. The three-step call order must not be reversed: `apply_mask` must be called before sampling to mask illegal tokens, and `accept_token` must be called after sampling to advance the state machine; otherwise the state machine and output become desynchronized.
>
> **Note**: - `rknn3_init_grammar` can only be called once per context; repeated calls destroy the old compiler, leaving the grammar handles of existing sessions dangling, and return an error. - `encoded_vocab` must cover all tokens in the vocabulary; missing tokens cause an incomplete mask, and the model may output tokens that violate the constraint. - Single-step decode overhead is about +0.5~1.0 ms/token (for reference); the first `set_grammar` compilation takes <100 ms (for reference); it is recommended to set it once before `session_run`. - Grammar state is rebuilt as a fresh matcher automatically at the entry of `session_run`; no manual reset is needed for multi-turn `keep_history` inference. To switch constraint types (e.g. JSON -> Regex), call `rknn3_session_set_grammar` directly without clearing first. - Current models all have a single output; when `llm_output_count > 1`, only the mask of `output_idx=0` is effective. - Grammar is currently incompatible with speculative decoding; disable speculative decoding when grammar is enabled.

4.3 Extensions and Security

4.3.1 Custom Operators

Overview

RKNN3 provides a custom operator extension mechanism that allows users to implement custom operators executing on the CPU cores of the RK182X coprocessor. This extends operator implementations, reduces data transfer back to the host, lowers the host SoC load, and simplifies application development. Typical scenarios include post-processing operations such as bounding box decoding and NMS for object detection models (YOLOv5/v6/v8), and intermediate operators in the model graph that RKNN3 does not yet support.

Two Operator Types

Based on position and purpose in the model, custom operators fall into two types:

Type	plugin_type	Position	Inputs and Outputs
Post-process operator	`RKNN3_OP_PLUGIN_TYPE_POSTPROCESS`	End of the model	Customized by the plugin's `get_output_num` / `get_attrs`
Custom CPU operator	`RKNN3_OP_PLUGIN_TYPE_CUSTOM_OP`	Any position in the compute graph	Determined by the model graph

Both types share the same plugin interface (`rknn3_custom_op` structure), compilation method, and loading API (`rknn3_register_custom_ops_plugins()`). They differ in the `plugin_type` field and the corresponding callback responsibilities.

Plugin Interface Definition

A custom operator implements the interface defined by the `rknn3_custom_op` structure, which is defined in `rknn3_api.h`:

```
typedef struct _rknn3_custom_op
{
    const char *op_type;                    // Operator type name
    rknn3_op_plugin_type plugin_type;       // Plugin type
    rknn3_op_target_type target;            // Execution backend (CPU)
    uint32_t version;                       // Version number
    const char *author;                     // Author information
    const char *description;                // Description

    // Lifecycle callbacks
    int (*init)(rknn3_custom_op_context *op_ctx);     // [Optional] Initialization
    int (*prepare)(rknn3_custom_op_context *op_ctx,   // [Optional] Preparation
                   rknn3_tensor *inputs, uint32_t n_inputs,
                   rknn3_tensor *outputs, uint32_t n_outputs);
    int (*compute)(rknn3_custom_op_context *op_ctx,   // [Required] Compute
                   rknn3_tensor *inputs, uint32_t n_inputs,
                   rknn3_tensor *outputs, uint32_t n_outputs);
    int (*deinit)(rknn3_custom_op_context *op_ctx);   // [Optional] De-
initialization

    // Post-process plugin specific interfaces
    int (*get_output_num)(rknn3_custom_op_context *op_ctx);  // [Optional] Get the
output count
    int (*get_attrs)(rknn3_custom_op_context *op_ctx,        // [Optional] Get input
and output attributes
                     rknn3_tensor_attr *input_attrs, uint32_t n_inputs,
                     rknn3_tensor_attr *output_attrs, uint32_t n_outputs);
} rknn3_custom_op;
```

Key Data Structures:

- `rknn3_custom_op_context`: Operator context, containing the RKNN3 context handle, a private data pointer (`priv_data`, managed by the framework; users must not modify it), a user data pointer (`user_data`, managed by the user), and more. Custom CPU operators can also read operator attributes through its `get_param` interface.
- `rknn3_tensor`: Tensor structure, containing memory information (`rknn3_tensor_mem`) and attribute information (`rknn3_tensor_attr`). Plugins read inputs from `inputs[i].mem->virt_addr` and write outputs to `outputs[j].mem->virt_addr`.
- `rknn3_tensor_attr`: Tensor attributes, including shape / dtype / layout and quantization parameters.

Plugin Type and Execution Backend Enums:

```
typedef enum _rknn3_op_plugin_type {
    RKNN3_OP_PLUGIN_TYPE_POSTPROCESS = 0,  // Post-process operator (at the end of
the model)
    RKNN3_OP_PLUGIN_TYPE_CUSTOM_OP = 1,    // Custom CPU operator (any position in
the graph)
} rknn3_op_plugin_type;

typedef enum _rknn3_op_target_type {
    RKNN3_OP_TARGET_TYPE_CPU = 1,          // CPU backend
} rknn3_op_target_type;
```

The rest of this section describes implementation steps, compilation/loading, and examples by operator type: 4.3.1.1 post-process operators and 4.3.1.2 custom CPU operators.

4.3.1.1 POST-PROCESS OPERATORS

Post-process operators sit at the end of the model. They move post-processing computation (such as YOLO bounding box decoding and NMS) from the host SoC to the coprocessor, reducing data transfer back to the host and lowering the host load. The plugin customizes the output count and attributes through `get_output_num` / `get_attrs`.

4.3.1.1.1 Plugin Implementation Steps

The custom operator plugin development flow is shown in the figure below:

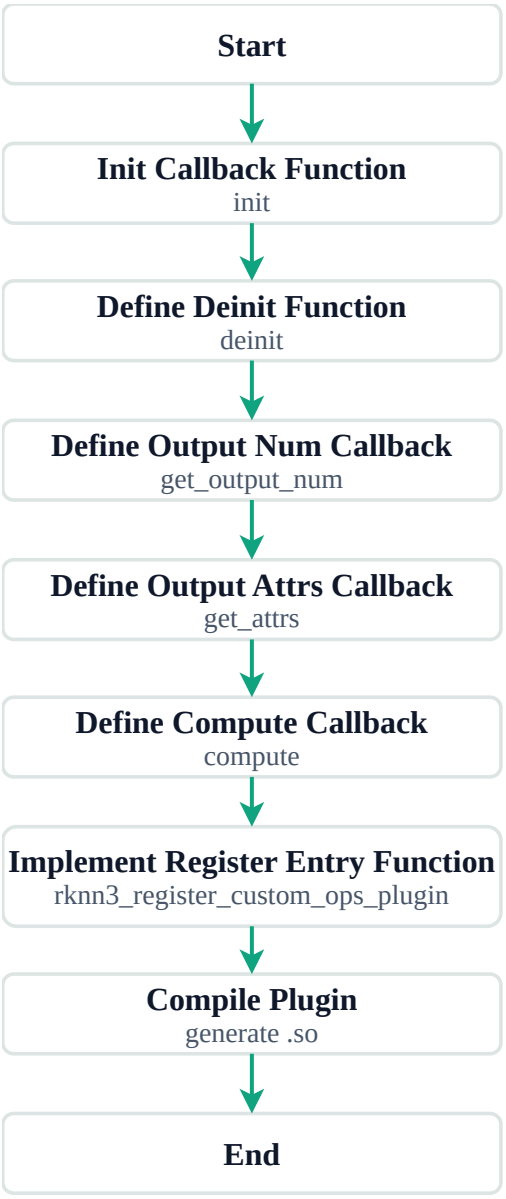


Figure 4-1 Custom operator plugin development flow

As shown, plugin implementation completes each callback in turn, assembles them into the operator structure for registration, and finally compiles them into a `.so`. The steps below use a YOLO post-process plugin (`YoloPostprocess`) as an example:

- **Implement the initialization callback**: `init` is invoked at operator initialization. It allocates private data (such as thresholds and model dimensions) and stores it in `op_ctx->priv_data` for later callbacks.

```
static int yolo_postprocess_plugin_init(rknn3_custom_op_context *op_ctx)
{
    rknn3_yolo_postprocess_context *pp_ctx = malloc(sizeof(...));
    memset(pp_ctx, 0, sizeof(...));
    pp_ctx->conf_threshold = 0.25;
    pp_ctx->nms_threshold  = 0.45;
    op_ctx->priv_data = pp_ctx;
    return 0;
}
```

- **Define the deinitialization function**: `deinit` is invoked at operator destruction. It releases private data and other resources allocated in `init`.

```
static int yolo_postprocess_plugin_deinit(rknn3_custom_op_context *op_ctx)
{
    if (op_ctx->priv_data) {
        free(op_ctx->priv_data);
        op_ctx->priv_data = NULL;
    }
    return 0;
}
```

- **Define the output tensor callback**: `get_output_num` returns the number of output tensors of the operator. The output count of a post-process plugin is defined by the plugin itself.

```
static int yolo_postprocess_plugin_get_output_num(rknn3_custom_op_context *op_ctx)
{
    return 1;   /* Output 1 tensor: detection results */
}
```

- **Define the output attribute callback**: `get_attrs` defines attributes such as shape, dtype, and aligned_size of the output tensors.

```
static int yolo_postprocess_plugin_get_attrs(rknn3_custom_op_context *op_ctx,
                                             rknn3_tensor_attr *input_attrs,
uint32_t n_inputs,
                                             rknn3_tensor_attr *output_attrs,
uint32_t n_outputs)
{
    output_attrs[0].n_dims  = 3;
    output_attrs[0].shape[0] = input_attrs[0].shape[0];  /* batch */
    output_attrs[0].shape[1] = 256;   /* Maximum number of detections */
    output_attrs[0].shape[2] = 6;     /* score, class_id, x1, y1, x2, y2 */
    output_attrs[0].dtype    = RKNN3_TENSOR_FLOAT32;
    return 0;
}
```

- **Define the compute callback**: `compute` is the required callback. It executes the operator's actual computation logic, reads inputs from `inputs[i].mem->virt_addr`, and writes the processed results to `outputs[j].mem->virt_addr`.

```
static int yolo_postprocess_plugin_compute(rknn3_custom_op_context *op_ctx,
                                           rknn3_tensor *inputs, uint32_t n_inputs,
                                           rknn3_tensor *outputs, uint32_t
n_outputs)
{
    int result_count = post_process(op_ctx->priv_data, inputs, n_inputs, outputs,
n_outputs);
    return (result_count >= 0) ? 0 : -1;
}
```

- **Implement the registration entry function**: Assemble the callbacks above into an `rknn3_custom_op` structure (specifying `op_type`, `plugin_type`, `target`, and the callback pointers), then register through the fixed-name entry function `rknn3_register_custom_ops_plugin()`. The framework invokes it repeatedly by `op_index` to retrieve the registered operators. One `.so` can register multiple operators.

```
static rknn3_custom_op yolo_postprocess_op = {
    .op_type        = "YoloPostprocess",
    .plugin_type    = RKNN3_OP_PLUGIN_TYPE_POSTPROCESS,
    .target         = RKNN3_OP_TARGET_TYPE_CPU,
    .init           = yolo_postprocess_plugin_init,
    .compute        = yolo_postprocess_plugin_compute,
    .deinit         = yolo_postprocess_plugin_deinit,
    .get_output_num = yolo_postprocess_plugin_get_output_num,
    .get_attrs      = yolo_postprocess_plugin_get_attrs,
};

static rknn3_custom_op* registered_ops[] = { &yolo_postprocess_op, NULL };

rknn3_custom_op* rknn3_register_custom_ops_plugin(int op_index)
{
    if (op_index < 0 || registered_ops[op_index] == NULL) {
        return NULL;
    }
    return registered_ops[op_index];
}
```

- **Compile the plugin**: Compile the C source above into a `.so` shared library. For compilation requirements and scripts, see 4.3.1.1.2.

4.3.1.1.2 Plugin Compilation, Loading, and Usage

Compilation Requirements

- Pure C only (C++ classes/templates/namespaces/exceptions are not supported).
- The compiler is the RISC-V toolchain `riscv64-unknown-elf-gcc` (dedicated to the RK182X CPU cores).
- Key compilation options: `-mcpu=c908v -mrvv-v0p10-compatible -march=rv64gcv -mabi=lp64d -O2 -g2 -mcmodel=medany -fpic -fno-plt`.
- Link options: `-nostartfiles -nostdlib -static-libgcc -e 0 -Wl,-r,-z,max-page-size=1024`. The output is a `.so`, trimmed with `riscv64-unknown-elf-strip`.

```
CC=riscv64-unknown-elf-gcc
STRIP=riscv64-unknown-elf-strip
CFLAGS="-mcpu=c908v -mrvv-v0p10-compatible -march=rv64gcv -mabi=lp64d -O2 -g2 \
        -mcmodel=medany -fpic -fno-plt -D_POSIX_C_SOURCE=1 -I${RKNPU3_INCLUDE}"
LDFLAGS="-mcpu=c908v -O2 -g2 -Wl,-r,-z,max-page-size=1024 -fpic \
         -nostartfiles -nostdlib -static-libgcc -e 0"
${CC} ${LDFLAGS} -o libpostprocess_yolov5_rk182x.so *.o
${STRIP} --strip-unneeded -R .hash libpostprocess_yolov5_rk182x.so
```

Loading and Usage

Load plugins with `rknn3_register_custom_ops_plugins()`, which must be invoked after `rknn3_model_init()` and before `rknn3_run()`. The third parameter `size` is a reserved field; pass `0` for now. After loading, the framework rebuilds the operator dispatch table automatically, and `rknn3_run()` invokes the plugin compute automatically.

API interface:

```
int rknn3_register_custom_ops_plugins(rknn3_context ctx,
                                      const char* plugin_path,
                                      int64_t size);
```

Usage flow (using the YOLOv5 post-process plugin as an example):

```
/* 1. Initialize and load the model */
rknn3_init(&ctx, NULL);
rknn3_load_model_from_path(ctx, "yolov5.rknn", "yolov5.weight");
rknn3_model_init(ctx, &config);

/* 2. Load the post-process plugin (must be called after model_init) */
const char* plugin_path = "./libpostprocess_yolov5_rk182x.so";
ret = rknn3_register_custom_ops_plugins(ctx, plugin_path, 0);
if (ret != RKNN3_SUCCESS) {
    printf("Plugin load failed! ret=%d\n", ret);
    return -1;
}

/* 3. Query the plugin output information */
rknn3_input_output_num io_num;
rknn3_query(ctx, RKNN3_QUERY_POSTPROCESS_IN_OUT_NUM, &io_num, sizeof(io_num));
rknn3_tensor_attr output_attrs[io_num.n_output];
for (int i = 0; i < io_num.n_output; i++) {
    output_attrs[i].index = i;
    rknn3_query(ctx, RKNN3_QUERY_POSTPROCESS_OUTPUT_ATTR,
                &output_attrs[i], sizeof(rknn3_tensor_attr));
}

/* 4. Run inference (the framework invokes the plugin compute automatically) */
rknn3_run(ctx, inputs, n_inputs, outputs, n_outputs);

/* 5. Read the plugin output (already contains the post-processed detection boxes)
*/
float* result = (float*)outputs[0].mem->virt_addr;
```

> **Note**: The same context can load plugins multiple times; a later load replaces an earlier one with the same `op_type` (the old plugin is dlclosed first, invoking deinit). After a post-process plugin is loaded, use `RKNN3_QUERY_POSTPROCESS_IN_OUT_NUM` / `RKNN3_QUERY_POSTPROCESS_OUTPUT_ATTR` to query the output count and attributes, and allocate output memory accordingly. This compilation and loading method applies to all custom operators, including custom CPU operators.

4.3.1.1.3 Examples

The YOLOv5 post-processing example is located at `rknn3-model-zoo/examples/yolov5/cpp/libpostprocess_rk182x/` and includes both Anchor-based (yolov5) and Anchor-free (yolov8) implementations. Typical configuration: `MAX_OBJ_NUM=256`, `OBJ_CLASS_NUM=80`, `NMS_THRESH=0.45`, `BOX_THRESH=0.25`; output format `[N, 256, 6]` (score, class_id, x1, y1, x2, y2).

4.3.1.2 CUSTOM CPU OPERATORS

Custom CPU operators (`RKNN3_OP_PLUGIN_TYPE_CUSTOM_OP`) can sit anywhere in the model compute graph, replacing or supplementing operators that RKNN3 does not support. During toolchain conversion, nodes that cannot be mapped to the NPU are preserved as CustomOperator and executed at runtime on the coprocessor CPU cores. Main differences from post-process operators:

Aspect	Post-Process Operator	Custom CPU Operator
Position	End of the model	Any position in the compute graph
Inputs and outputs	Customized by the plugin's `get_output_num` / `get_attrs`	Determined by the model graph; the plugin does not implement these two callbacks
Operator attributes	None	From the original framework (ONNX), read through `get_param`

4.3.1.2.1 Plugin Implementation Steps

The implementation steps of a custom CPU operator are similar to those of a post-process operator (init / deinit / compute / registration entry), with two key differences:

- **Do not implement `get_output_num` / `get_attrs`**: Inputs and outputs are determined by the model graph; no plugin customization is needed.
- **Read operator attributes through `get_param`**: Operator attributes come from the original framework (ONNX). `rknn3_custom_op_context` contains `rknn_ctx`, `priv_data` (do not modify), `user_data` (managed by the user), and `get_param`. `get_param` supports 6 dtypes: `i` (int32), `f` (float), `s` (char[]), `is` (int32[]), `fs` (float[]), and `ss` (char[] NUL-separated).

The `compute` callback typically has three steps: read attributes -> fetch inputs and convert layout/type (unpack NC1HWC2+FP16 into NCHW+FP32) -> compute and write back (pack back into NC1HWC2).

```
/* op_type must exactly match the operator type name in the model (case-sensitive)
*/
static rknn3_custom_op my_op = {
    .op_type     = "RknnCustomOpExam",
    .plugin_type = RKNN3_OP_PLUGIN_TYPE_CUSTOM_OP,
    .target      = RKNN3_OP_TARGET_TYPE_CPU,
    .init        = my_op_init,
    .compute     = my_op_compute,
    .deinit      = my_op_deinit,
};
static rknn3_custom_op* registered_ops[] = { &my_op, NULL };
```

4.3.1.2.2 Plugin Compilation, Loading, and Usage

The compilation requirements, loading API, and usage flow of custom CPU operators are identical to those of post-process operators; see 4.3.1.1.2. The only difference is that after loading, there is no need to query `RKNN3_QUERY_POSTPROCESS_*` (the inputs and outputs of a custom CPU operator are determined by the model graph; the regular `RKNN3_QUERY_IN_OUT_NUM` / `RKNN3_QUERY_OUTPUT_ATTR` queries suffice).

4.3.1.2.3 Examples

For a complete example, see `rknn3-runtime/examples/rknn3_custom_op_demo/` (RknnCustomOpExam: `y=clamp(x*scale+shift, min, max)`). It demonstrates the full flow of reading attributes with `get_param` and executing computation with `compute`. Use `fp16_to_fp32` / `fp32_to_fp16` from `float16.h` for FP16/FP32 conversion.

4.3.2 Encrypted Model Deployment

The RKNN SDK supports encrypting RKNN models offline and decrypting them at runtime.

> Model encryption is only one part of model protection and cannot completely eliminate the risk of the model being cracked. Users are advised to prevent third parties from directly accessing the model and implement overall protection at the system level.

Applicable Scenarios

Protect model intellectual property and prevent unauthorized copying and distribution of `.rknn` / `.weight` files. A hybrid encryption architecture is used:

- **AES-256-CTR** (symmetric): Encrypts the model body. High performance and supports random-access decryption.
- **RSA-OAEP** (asymmetric): Protects the AES key for secure distribution.

Key system: a user key (AES-256, generated by the user, encrypts and decrypts the model files) plus an RK key pair (RSA, preloaded by RK; the public key encrypts the user key, and the private key built into the chip decrypts it).

Usage

- **Encryption**: Encrypt the model with `rknn3_model_encrypt.py`. It outputs the encrypted model and the key envelope:

```
python3 rknn3_model_encrypt.py \
    --model model.rknn --weight model.weight \
    --rk-pubkey rk_public_key.pem --output-dir ../encrypted
# Output: model.rknn.enc, model.weight.enc, user_key_iv.enc (RSA-encrypted key
envelope)
```

- **Loading**: Set the decryption key after `rknn3_init()` and before `rknn3_load_model_from_path()`. The runtime decrypts automatically, and the subsequent flow is identical to that of unencrypted models:

```
rknn3_init(&ctx, NULL);
rknn3_set_decrypt_key_from_path(ctx, "user_key_iv.enc");   /* Must be called before
load_model */
rknn3_load_model_from_path(ctx, "model.rknn.enc", "model.weight.enc");
rknn3_config config = {0};
config.run_core_mask = 0xff;  /* Configure according to the actual core count */
rknn3_model_init(ctx, &config);
/* Subsequent inference is identical to unencrypted models */
```

> **Note**: Security features include two-layer encryption; the key can be decrypted only by the private key built into the chip; weights are transmitted as ciphertext (no plaintext is exposed on PCIe / USB / Ethernet links); and the decrypted key exists only briefly in memory and is cleared after use. For a complete example, see `rknn3-runtime/examples/rknn3_encrypt_demo/`.

5. Issue Diagnosis and Optimization

This chapter describes diagnosis methods and optimization techniques for common issues in model deployment, covering three core issue categories—accuracy, performance, and memory—as well as common error handling. The overall troubleshooting approach is: first confirm environment and version consistency, then verify model correctness on the simulator, and finally locate Runtime or deployment issues on the device.

5.1 Accuracy Issue Diagnosis

Applicable Scenarios

Inference results deviate excessively from the original model after model conversion or quantization, and you need to determine whether the cause is a configuration error, quantization error, pre-processing and post-processing error, or a Runtime implementation issue. Accuracy troubleshooting is divided by quantization workflow into internal quantization accuracy diagnosis and RKQuantizer quantization accuracy diagnosis; the two differ slightly in their troubleshooting stages.

5.1.1 Internal Quantization Accuracy Diagnosis

Accuracy troubleshooting for the internal quantization workflow follows the progressive sequence of "simulator FP16 accuracy -> simulator quantized accuracy -> connected-board accuracy -> Runtime accuracy"; correctness at each stage is a prerequisite for the next. The troubleshooting steps are shown in the figure below:

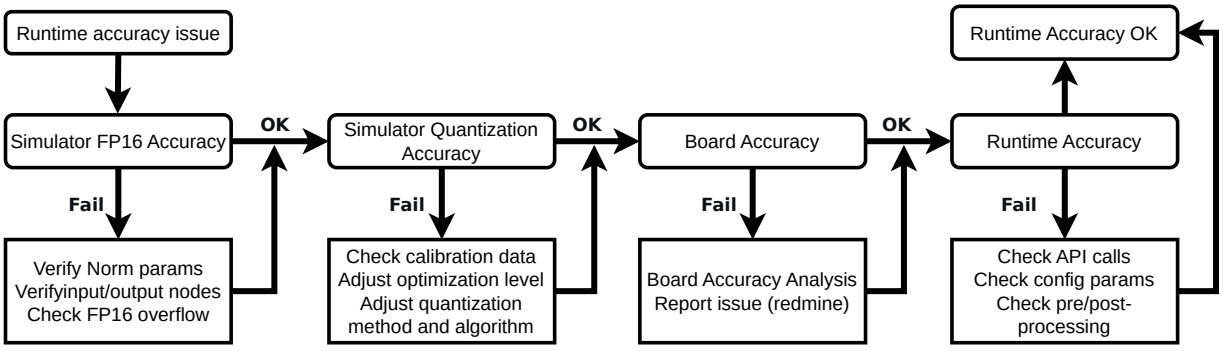


Figure 5-1 Internal quantization accuracy troubleshooting steps

Cosine distance serves as the basic criterion for judging accuracy, and final verification must be performed on a dataset. The methods for each stage are described below in order.

Stage 1: Simulator FP16 Accuracy Troubleshooting

Run inference on the simulator with quantization disabled (FP16 by default) to confirm basic correctness. Focus on the following:

- **Verify normalization parameters**: `mean_values` / `std_values` must match those used during training. Note the mismatch between the BGR order read by cv2 and the NHWC default values. For an N-channel input, use `[[c1, c2, ..., cn]]`; for multiple inputs, use `[[...], [...]]`, with each sub-list corresponding to one input:

```
rknn.config(mean_values=[[128, 128, 128, 128]],   # 4-channel single input
            std_values=[[1, 1, 1, 1]],
            target_platform='rk1820', core_num=1)
```

- **Verify input and output nodes**: The input/output node names of `load_onnx()`, `input_size_list` (input sizes), and `data_format` must match the model definition.
- **Check for FP16 overflow**: If an intermediate tensor exceeds -65504~65504, use accuracy analysis to check for `inf`; this can be resolved by modifying the model structure, such as adding a BN layer. If FP16 inference results already differ significantly from the original model and it is not a configuration or overflow issue, the Runtime accuracy may be poor; go directly to Stage 4 for Runtime accuracy troubleshooting.

Stage 2: Simulator Quantized Accuracy Troubleshooting

After FP16 correctness is confirmed, enable quantization and start troubleshooting. Focus on the following:

- **Check calibration data**: The calibration set must cover the data distribution of the actual application scenario, typically 20~200 sets. Calibration data can be images (jpg/png, read with OpenCV) or npy files (read with numpy); npy is recommended for inputs that are not RGB/BGR images. Each line in `dataset.txt` is one set of inputs; for multiple inputs, separate them with spaces:

```
sampleA_in0.npy sampleA_in1.npy sampleA_in2.npy
sampleB_in0.npy sampleB_in1.npy sampleB_in2.npy
```

For 3-channel image input with quantization data in image format, confirm whether the model input is RGB or BGR, and set `quant_img_RGB2BGR` accordingly. This parameter only controls whether channels are converted when calibration set images are read during quantization; it does not affect inference: if the model was trained with RGB, set it to `False` or leave it unset, and feed RGB input during inference; if the model was trained with BGR, set it to `True` and feed BGR input during inference. If quantization data is in npy format, it is recommended to leave this parameter unset to avoid confusion.

- **Adjust the optimization level**: `optimization_level` defaults to 3 (speed-prioritized). Lowering it (for example to 0) disables optimizations that affect accuracy.
- **Adjust the quantization method and algorithm**: If accuracy is insufficient, optimize in the following order:
- **Switch the quantization method**: For w8a8, change from `layer` to `channel`; for w4a16, change from `channel` to `group32`.
- **Switch the quantization algorithm**: `normal` -> `kl_divergence` (effective when feature distributions are uneven) -> `mmse` (iteratively clips weights; greatly increases time and memory).
- **Hybrid quantization**: Use higher precision for sensitive layers (for example w8a16) and lower precision for the rest (for example w4a16); specify the quantization type of each layer via `op_quantized_dtype`.

Quantization Algorithms and Calibration Data:

Algorithm	Characteristics	Recommended Data Volume
`normal` (default)	Standard calibration, fast	20~200 sets
`kl_divergence`	Better improvement when feature distributions are uneven; slightly slower than normal	20~200 sets
`mmse`	Iteratively clips weights of intermediate layers; may improve accuracy but greatly increases time and memory	20~50 sets

Start with `normal`; if accuracy is unsatisfactory, try `kl_divergence` or `mmse`. The quantization algorithm only affects the conversion stage and does not affect runtime performance.

If some layers in the `single` column show severe accuracy loss that cannot be improved, the numerical distribution of the weights may be unsuitable for quantization; the model structure must be modified or the model retrained.

Stage 3: Connected-Board Accuracy Troubleshooting

After the simulator works correctly, run connected-board inference (specify `target` + `core_mask`) and compare the results with the simulator results. Focus on the following:

- **Connected-board accuracy analysis**: If differences are large, perform connected-board accuracy analysis (`rknn.inference(accuracy_analysis=True)`, requires `profile_mode=True`) to compare per-layer output differences between the simulator and the board. Focus on checking whether pre-processing and post-processing are consistent with the simulator (normalization, channel order, data type), and the C API input/output configuration (whether normalization is applied twice, channel order, `rknn3_create_mem()` size and data format).
- **Report the issue (redmine)**: If differences persist and cannot be eliminated, report the accuracy analysis results to the NPU team (redmine).

Stage 4: Runtime Accuracy Troubleshooting

If connected-board inference is correct but on-device deployed inference is abnormal, perform on-device accuracy analysis and check the `single_sim` column in `runtime_error` (low cos or high euc values are highlighted in yellow or red). Focus on the following:

- **Check API calls**: Verify that the API call order and parameter passing are correct.
- **Check configuration parameters**: Verify that configurations such as `core_mask`, memory allocation, and data types match the model.
- **Check pre-processing and post-processing**: Verify that input pre-processing (normalization, channel order, data type) and output post-processing (dequantization, business logic) are consistent with the simulator.

If abnormal `single_sim` values cause the `entire` results to diverge further from the simulator, the Runtime implementation of that layer may be faulty; report it to the NPU team.

5.1.2 RKQuantizer Quantization Accuracy Diagnosis

The output of the RKQuantizer quantization workflow is already a fake-quantized model (weights are fake-quantized), so FP16 accuracy troubleshooting is not needed. Accuracy troubleshooting follows the progressive sequence of "quantized accuracy -> connected-board accuracy -> Runtime accuracy". The troubleshooting steps are shown in the figure below:

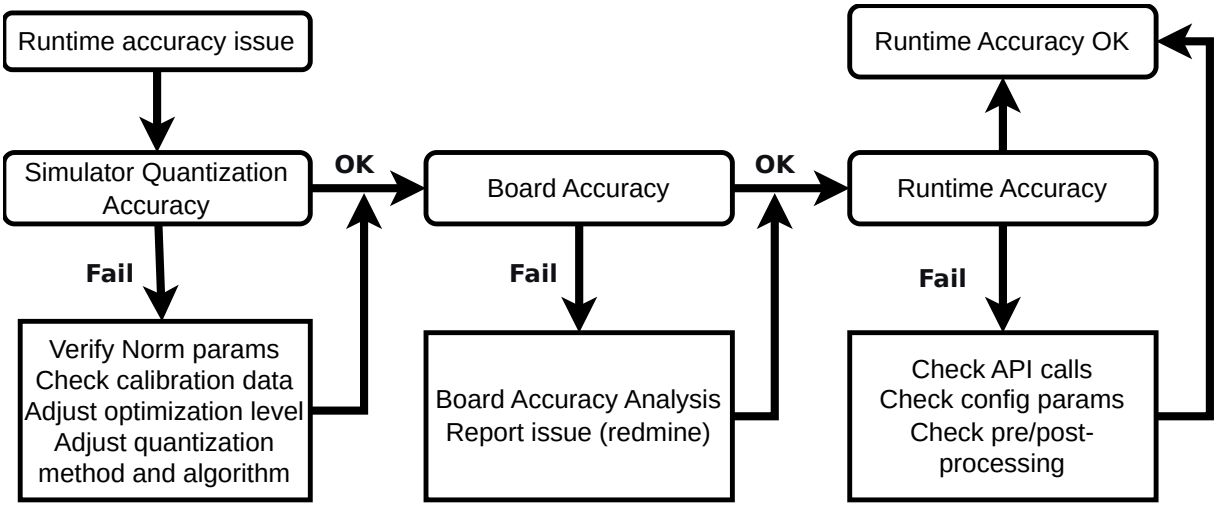


Figure 5-2 RKQuantizer quantization accuracy troubleshooting steps

Stage 1: Quantized Accuracy Troubleshooting

For models quantized by RKQuantizer, start directly with quantized accuracy troubleshooting. Focus on the following:

- **Verify normalization parameters**: `mean_values` / `std_values` must match those used during training. Note the mismatch between the BGR order read by cv2 and the NHWC default values.
- **Check calibration data**: The GRQ algorithm of RKQuantizer requires calibration data to collect the activation distribution of each layer and fine-tune the weights. Calibration data is provided as JSON files and must cover the data distribution of the actual application scenario, typically 20~200 samples. For the specific format and preparation method of calibration data, see 2.3.4 RKQuantizer Quantization Workflow.
- **Adjust the optimization level**: In the RKNN conversion stage, `optimization_level` of `rknn.config()` defaults to 3 (speed-prioritized). Lowering it (for example to 0) disables optimizations that affect accuracy. Note that when converting with the RKQuantizer quantization workflow, `quantized_algorithm` must be set to `normal`.
- **Adjust the quantization method and algorithm**: RKQuantizer supports two algorithms, GRQ and Normal, with GRQ as the first choice. If accuracy is insufficient, adjust `quantized_dtype` (w4a16 / w6a16 / w8a16) or use hybrid precision (`layer_quant_config` / `auto_hybrid_rate`); see 2.3.4 for details.

LLM / MLLM/VIT Quantization Recommendations: Use `quantized_dtype='w4a16'` and `quantized_method='group32'`; to further improve w4a16 accuracy, apply RKQuantizer GRQ quantization first.

If some layers in the `single` column show severe accuracy loss that cannot be improved, the numerical distribution of the weights may be unsuitable for quantization; the model structure must be modified or the model retrained.

Stage 2: Connected-Board Accuracy Troubleshooting

Same as the connected-board accuracy troubleshooting for internal quantization; see Stage 3 in 5.1.1.

Stage 3: Runtime Accuracy Troubleshooting

Same as the Runtime accuracy troubleshooting for internal quantization; see Stage 4 in 5.1.1.

5.2 Performance Analysis and Optimization

Applicable Scenarios

Model inference speed falls short of expectations, and you need to determine whether the cause is an NPU compute bottleneck, a bandwidth bottleneck, CPU operator overhead, or time spent on pre-processing/post-processing and data transfer, and then optimize accordingly.

Performance Analysis Tools

Tool/Interface	Function	Usage
`rknn.eval_perf(log_level=)`	Toolkit performance evaluation; outputs Op Time Ranking	Requires `config(profile_mode=True)` first
`rknn3_profile_ops(ctx, ..., log_level)`	Per-layer operator performance on device	log_level 0 for summary / 1 for per operator / 2 for Cycle and bandwidth
`rknn3_session_query_state()`	LLM session performance statistics	Queries `RKLLMRunState`
Lite `session_run` return value	LLM Python performance statistics	Returns `[n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time]`

Key LLM performance metrics: TTFT (first-token latency), TPOT (time per token), Prefill TPS, and Decode TPS. Use `llm_start_time` / `llm_end_time` plus the first-token time to calculate the time and throughput of each stage.

Performance Optimization Techniques

- **Multi-core configuration**: Use multiple cores (`core_num>1`) for compute-intensive models, and a single core for small networks (multi-core switching requires CPU involvement and can be slower). `core_mask` must match the `core_num` used at conversion.
- **Quantization acceleration**: Quantization improves both bandwidth and compute utilization, and is the fundamental means of performance optimization.
- **Offload CPU operators to the NPU**: Most performance issues stem from CPU operators (size limits exceeded/unsupported/hardware limitations). Identify CPU operators with `eval_perf` / `profile_ops`, then replace them with NPU-supported equivalent operators or adjust their sizes.
- **Graph-level optimization**: Hardware fusion (Conv+Relu/Clip/Add), replacing algorithm-equivalent subgraphs with a single OP, and merging same-type operators.
- **Operator-level optimization**: DDR alignment (align Channel to Int8=32/FP16=16, and HW to 4), avoid inefficient operators (Transpose/Reshape/Resize/Sigmoid/Tanh), and optimize convolution sizes (utilization is higher when Channel<256 and output HW>=32).

- **LLM-specific optimization**: `keep_one_logit` reduces lm_head computation in the prefill stage, `kvcache_dtype` selects Int4/Int8 to reduce KVCache access, and multi-core configuration accelerates prefill.

Performance Analysis Baseline

Performance data is meaningful only under **fixed frequency** (data fluctuates significantly without it). On the board, use `opp_dump` to check CPU/NPU frequencies, and `srv_set_rate` / `npu_set_rate` to set frequencies. Use `rknn-smi info -w` to check the NPU load; a low load indicates the NPU is waiting, so optimize pre-processing and post-processing or use multi-threading.

> **Note**: To identify NPU operator bottlenecks, compare DDR Cycles vs NPU Cycles: DDR > NPU indicates a bandwidth bottleneck, otherwise a compute bottleneck. Deployment time consists of three parts: the user application (pre-processing and post-processing, which can be accelerated with the Matmul API), input/output transfer (in coprocessor mode, choose appropriate tensor data types/layout), and inference (average multiple runs).

5.3 Memory Analysis and Optimization

Applicable Scenarios

On-device memory is limited; when multiple models coexist or large models are deployed, memory becomes insufficient, requiring analysis of the memory usage composition and optimization. For LLM models, memory optimization is especially critical because KVCache consumes a large amount of memory.

Memory Analysis Tools

Tool/Interface	Function
`rknn.eval_memory()`	Toolkit memory evaluation; outputs Device Memory and the Per-Core Memory Allocation table
`rknn3_query(ctx, RKNN3_QUERY_CORE_MEM_SIZE, ...)`	Queries weight/internal usage of each NPU core on device
`rknn3_profile_mem(ctx)`	Prints per-core Command/Weight/Internal/KVCache and device memory statistics on device

The Per-Core Memory Allocation output of `eval_memory` includes Command (register configuration), Weight (weights), Internal (intermediate tensors), KVCache (LLM-specific), and Total.

Memory Composition

Runtime memory = weights + internal tensors + register configuration + input/output tensors; LLMs additionally have KVCache. Memory is allocated at `rknn3_model_init()` / `rknn3_session_init()`.

Memory Optimization Techniques

- **Internal memory reuse**: In multimodal serial inference (vision -> LLM), internal memory on the same core can be reused (see 4.1.4); the total number of blocks = number of vision cores + number of LLM cores - number of overlapping cores.

- **Multi-session sharing**: Multiple sessions under the same context share weights/internal/LoRA, with only KVCache being independent; this saves memory compared with multiple contexts.
- **KVCache optimization**: `kvcache_buffer_len` (32-aligned; set as needed), `kvcache_dtype` (memory usage: FP16 > Int8_to_F16 > Int4_to_F16), and `position_embeddings_host_storage` (stores on the Host side to save device memory).
- **Tie Word Embedding**: Reuses lm_head weights, saves `embed.bin`, and reduces Host-side memory (see 4.2.9).
- **llm_head device designation**: `llm_head.device` can designate lm_head to the main SoC (rk3588/rk3576) to reduce coprocessor memory.
- **core_mask control**: `core_num` / `core_mask` determine how many cores the weights/internal are distributed across, directly affecting Total memory; configure as needed rather than using all cores.

> **Note**: Dynamic-shape models do not support weight sharing or context reuse (`rknn3_dup_context()`), so memory reuse is limited. Use `rknn.eval_memory()` to check Total on-device memory and assess whether it is sufficient.

5.4 FAQ and Error Handling

Version and Compatibility

- **Version inconsistency**: rknn3-toolkit, the communication services (rknn3_transfer_proxy / rknn3-server), and the Runtime library must be the same version. Otherwise, model conversion may succeed but the model fails to run on the board, possibly with `protocol fingerprint mismatch` or an `Invalid RKNN format` error. Version query commands:

```
pip list | grep rknn3-toolkit                              # Toolkit version
strings xxx.rknn | grep rknn3-toolkit                      # Version used for model
conversion
strings librknn3_api.so | grep -i "librknn3_api version"   # Runtime version
strings rknn3_transfer_proxy | grep "Transfer version"     # Coprocessor mode
communication service version
rknn3-server -v                                            # Non-coprocessor mode
```

- **Cross-platform incompatibility**: Models for RK1820 / RK1828 / RK3572 are mutually incompatible; re-convert for the target platform.

Tool Installation

- **Installation failure due to dependency conflicts**: When all dependencies are installed but versions do not match, the pip environment check blocks installation. Add `--no-deps` to skip it: `pip install rknn3-toolkit-*.whl --no-deps`. You must then ensure the runtime dependencies are complete yourself.
- **No ARM Linux version**: RKNN3-Toolkit is only provided for x86_64; there is no ARM Linux version.

Model Loading Failure

Error	Cause	Solution
`Error parsing message`	ONNX model corrupted	Re-download and verify MD5
`Not support input data type 'float16'`	PyTorch float16 weights not supported	Change to float32
Error with custom output nodes	Invalid outputs node names	Verify the node names
Error with operators of more than 4 dimensions	Not supported yet	Remove or simplify manually
Dynamic graph error (such as NonZero)	Causes a dynamic graph	Replace/remove the OP

It is recommended to preprocess the model with `onnx-simplifier` before conversion.

Model Conversion Issues

- **onnxsim simplification failure**: onnx-simplifier may fail to simplify models with failed shape inference or dynamic graphs. In this case, convert directly from the original ONNX; onnxsim is not mandatory.
- **mean_values channel count error** (such as `expect N`): When the model input is not a 3-channel image (for example 1x32 non-image data), the number of `mean_values` channels must match the actual input channels; if the model does not need mean/std, leave them unset.
- **RKNN model larger than the original model**: This is normal. In addition to weights and graph structure, an RKNN model contains NPU register configuration information and may split OPs to improve runtime efficiency.

Incorrect Inference Results

- **Differences between simulator and board**: Small differences caused by hardware driver variations are normal; report large differences to the NPU team.
- **Large differences between connected-board and on-device inference**: Check the consistency of pre-processing, data types, and layout (NCHW / NHWC).
- **Inference takes >20s with incorrect results**: Usually an NPU hang; update the Toolkit / Runtime.
- **Connected-board error with 3-dimensional input**: NPU support for 3 dimensions is incomplete; change to 4 dimensions or update to a newer version.
- **Results differ between runs**: An on-device NPU kernel driver bug; update the driver and the Toolkit.
- **All-nan results with `do_quantization=False`**: An intermediate layer exceeds the FP16 range; add a BN layer.
- **QAT model inconsistency**: Set `engine='qnnpack'` for PyTorch inference.

Simulator and Connected-Board Inference

- **Distinguishing the three inference methods**: Simulator inference (RKNN3-Toolkit simulator on Linux x86_64; output may not match the board), connected-board inference (the PC calls the Toolkit Python API while the model runs on the board NPU), and on-device

inference (the board calls the C API to run inference). Simulator results are for preliminary verification only; for accuracy/performance, refer to connected-board or on-device results.

- **Connected-board slower than on-device**: Connected-board inference incurs PC↔board data transfer overhead; the actual NPU inference performance is measured with the on-device C API.
- **Viewing connected-board logs**: Model initialization and inference run on the board, so logs are generated on the board. Enter the board system via the serial port, run `export RKNN_LOG_LEVEL=5`, then start connected-board debugging; on-device error messages appear in the serial console window.
- **data_format default value**: In connected-board mode, `rknn.inference()` defaults to `NHWC`, while ONNX 4-dimensional input defaults to NCHW; inference internally converts NCHW→NHWC. If `data_format='NCHW'` is set explicitly, no conversion is performed.

Model Evaluation Issues

- **`Invalid RKNN format`**: On the board, this error usually means `target_platform` in `rknn.config()` is unset or incorrect, or the Toolkit and Runtime versions are incompatible. Check target_platform and ensure version consistency.
- **Inference time inconsistent with eval_perf**: `rknn.inference()` connects to the board via PC + adb and incurs fixed data transfer overhead; for the real frame rate, test directly on the board with the C API.
- **Resize OP causes accuracy drop**: The NPU does not yet support hardware-level Resize; the conversion tool converts Resize to ConvTranspose, which loses accuracy. Avoid Resize where possible (replace it with ConvTranspose and retrain), or set `optimization_level=2` in `rknn.config()` to run Resize on the CPU (no accuracy loss but reduced performance).
- **perf_debug affects performance data**: Enabling `perf_debug` in `rknn.init_runtime()` adds debug code and may disable parallelism, so the measured time is higher than with it disabled. perf_debug is for locating layers that account for a large share of the time, not for absolute performance evaluation.

core_mask Mismatch

Error `core_mask ff is not match with npu core number 1!`: The `run_core_mask` of `rknn3_model_init()` does not match the `core_num` used at conversion. Configure according to the correct core number: 0x1~0xff for RK1820 / RK1828, and only 0x1 for RK3572.

Insufficient Memory

- **Killed during quantization**: Toolkit quantization consumes a large amount of memory; increase the PC memory or virtual memory/swap partition.
- **Insufficient on-device memory**: Check Total with `rknn.eval_memory()` and optimize using the techniques in 5.3.

Return Value Checking

Check the return value of every API call; `RKNN3_SUCCESS` (=0) indicates success. Unified pattern:

```c
int ret = rknn3_init(&ctx, NULL);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_init failed! ret=%d\n", ret);
    return -1;
}
```

`rknn3_session_init()` returns NULL on failure. Resource release order: destroy the session first (`rknn3_session_destroy()`), then destroy the context (`rknn3_destroy()`); for general model scenarios, also release self-allocated tensor memory (`rknn3_destroy_mem()`) before destroying the context.

C API Usage

- **`rknn3_create_mem()` allocation size**: Input memory is generally allocated using `aligned_size` and `core_id` from `rknn3_tensor_attr`, ensuring the size and alignment meet NPU requirements.
- **Input data filling order**: Fill 4-dimensional input in `NHWC` order (`[batch, height, width, channel]`); fill non-4-dimensional input according to the original model shape, and set `data_format` to `undefined`.

Logging and Debugging

- Connected-board inference logs are on the board; enter the board system via the serial port and run `export RKNN_LOG_LEVEL=5` to obtain detailed errors.
- Use `RKNN_LOG_LEVEL=4` for on-device performance profiling.
- Connected-board initialization stuck after Docker restart: rknn3_transfer_proxy exited abnormally, leaving the RKNN Server connection state unreset; restart the development board.

LLM and Session

- **Slow first token (high TTFT)**: The prefill stage processes the entire input sequence and is compute-intensive. Reduce `max_context_len`, or enable KVCache Checkpoint (only for Linear/Sliding Attention models) to speed up long-prefix recovery.
- **Multi-turn conversation gets progressively slower**: Context growth enlarges the KVCache. Use the `RKNN3_KVCACHE_POLICY_RECURRENT` policy for automatic overwrite, and set `n_keep` / `n_keep_aligned` to keep the system prompt from being overwritten.
- **`rknn3_session_init()` returns NULL**: Check whether `vocab_info` is filled in correctly (`vocab_size` / `special_eos_id`, etc.), whether `max_context_len` is too large and causes insufficient memory, and whether the model is loaded correctly.
- **`result_callback` not triggered**: Confirm that the callback is registered via `rknn3_session_set_callback()` and that the `RKLLMCallback` structure is initialized correctly (fill it in after zeroing with `memset`).

Memory and Performance

- **Insufficient memory when loading an LLM model**: An overly large `kvcache_buffer_len` increases KVCache memory; set it according to actual needs (32-aligned). Choose `Int4_to_F16` or `Int8_to_F16` for `kvcache_dtype` for lower usage than

`Float16`; `position_embeddings_host_storage=True` stores position embeddings on the Host side to save device memory.

- **Insufficient memory with multiple models coexisting**: Multimodal serial inference can use Internal memory reuse (see 4.1.4); `llm_head.device` can offload lm_head computation to the main SoC to reduce coprocessor memory.

Deployment Modes

- **Cannot connect to the board in coprocessor mode**: Confirm that `rknn3_transfer_proxy` is started on the board, the PCIe / USB / Ethernet connection is working, and `device_id` is correct (query it via `rknn3_find_devices()` or `get_devices_id()`).
- **Whether RK3572 and RK182X models are interchangeable**: No. RKNN models for different platforms are mutually incompatible; re-convert for the target platform (`target_platform` parameter).

Model Encryption

- **Encrypted model fails to load**: Confirm that `rknn3_set_decrypt_key_from_path()` is called before `rknn3_load_model_from_path()`; the key envelope file (`user_key_iv.enc`) must be paired with the encrypted model; the key envelope is generated by encryption with the corresponding `rk_public_key.pem` and must not be mixed with others.

Multimodal

- **Garbled output after image input in MLLM**: Check whether the placeholder tokens (`image_start` / `image_end` / `image_content`) match the special tokens of the model tokenizer, and whether the dimensions of `image_embed` (`n_image_tokens × embedding_dim`) and the count (`n_image`) match the number of `<image>` tags in the prompt.

> **Note**: The error `Meet unsupport xxx operator` means the Runtime does not support that operator; update the toolchain to the latest version. The HTTP error codes of rkllm3-server follow the OpenAI specification (401 authentication_error, 400 invalid_request_error, 501 not_supported_error); for the complete error codes, see *RKLLM3_Server_User_Guide_and_API_Reference*.